



Baocao DA 1 - aaaa

NDT non destructive testing (Trường Đại học Sư phạm Kỹ Thuật Thành phố Hồ Chí Minh)



Scan to open on Studeersnel

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ
MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



BÁO CÁO MÔN HỌC

BÁO CÁO ĐỒ ÁN 1

**HỆ THỐNG THEO DÕI SỨC KHỎE VÀ PHÁT
HIỆN TẾ NGÃ Ở NGƯỜI CAO TUỔI ỨNG
DỤNG MACHINE LEARNING**

GVHD: TS. Phạm Văn Khoa

SVTH: Nguyễn Hoàng Đăng Duy -
22119171

Hồ Sỹ Nguyên - 22119203

TP. Hồ Chí Minh 07/2025

LỜI MỞ ĐẦU

Lời mở đầu này là nơi em muốn bày tỏ lòng biết ơn sâu sắc nhất đến những cá nhân đã góp phần quan trọng vào sự thành công của đề tài "Xây dựng hệ thống IoT giám sát sức khỏe và phát hiện té ngã cho người cao tuổi."

Trước tiên và trên hết, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến **Tiến sĩ Phạm Văn Khoa**, người thầy tận tâm và nhiệt huyết đã trực tiếp hướng dẫn chúng em trong suốt quá trình thực hiện đồ án chuyên ngành 1. Những chỉ dẫn quý báu, những góp ý chi tiết và sự đồng hành không ngừng nghỉ của thầy không chỉ giúp chúng em định hình rõ ràng hướng đi, mà còn là nguồn động lực to lớn giúp chúng em vượt qua mọi khó khăn, thử thách trong quá trình nghiên cứu và triển khai đề tài này. Không có sự hỗ trợ và kiến thức chuyên môn sâu rộng từ thầy, chắc chắn chúng em khó có thể đạt được những kết quả như hôm nay.

Một lần nữa, em xin trân trọng cảm ơn tất cả những đóng góp, sự hỗ trợ và tin tưởng quý báu mà em đã nhận được. Em tin rằng những nỗ lực và kết quả của đồ án này sẽ không chỉ là một cột mốc quan trọng trong hành trình học tập của chúng em mà còn là một nền tảng vững chắc, mở ra cánh cửa cho những nghiên cứu sâu rộng và ứng dụng thực tiễn hơn nữa trong tương lai.

Trân trọng

Mục lục

CHƯƠNG 1: GIỚI THIỆU.....	1
1.1 Đặt vấn đề.....	1
1.2 Giải quyết vấn đề.....	1
1.3 Mục tiêu.....	1
1.4 Giới hạn.....	3
1.5 Bố cục trình bày của đề tài.....	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	4
2.1. Tìm hiểu vi điều khiển ESP32 S3.....	4
2.1.1 Giới thiệu.....	4
2.1.2 Cấu trúc vi điều khiển.....	4
2.2 Cảm biến nhiệt độ MAX30205.....	9
2.2.1 Giới thiệu.....	9
2.2.2 Cấu hình chân.....	9
2.2.3 Thông số điện (Electrical Characteristics).....	11
2.2.4 Nguyên lý hoạt động.....	12
2.3 Cảm biến nhịp tim và nồng độ Oxy trong máu MAX30102.....	14
2.3.1 Mô tả chung.....	14
2.3.2 Cấu hình chân.....	14
2.3.3 Thông số điện.....	15
2.3.4 Nguyên lý hoạt động.....	17
2.4 Cảm biến gia tốc và góc quay MPU6050.....	18
2.4.1 Mô tả chung.....	18
2.4.2 Sơ đồ chân.....	19
2.4.3 Đặc điểm kỹ thuật điện.....	20
2.4.4 Nguyên lý hoạt động.....	21
2.5 Machine Learning.....	23
2.5.1 Giới thiệu.....	23
2.5.2 Mạng nơ-ron tích chập (CNNs).....	25
2.5.3 TensorFlow Lite.....	34
CHƯƠNG 3: TÌNH TOÁN VÀ THIẾT KẾ MÔ HÌNH.....	39
3.1 Giới thiệu mô hình.....	39
3.2 Thiết kế tính toán từng khối.....	40
3.2.1 Cảm biến nhiệt độ MAX30205.....	40

3.2.2 Cảm biến nhịp tim và nồng độ Oxy trong máu MAX30102.....	42
3.2.3 Cảm biến gia tốc và góc quay MPU6050.....	46
3.2.4 Mô hình Deep Learning.....	51
3.2.5 Khối điều khiển trung tâm.....	55
CHƯƠNG 4: GIỚI THIỆU PHẦN MỀM.....	59
4.1 Ngôn ngữ lập trình.....	59
4.1.1 Python.....	59
4.1.2 C/C++.....	61
4.2 Phần mềm viết chương trình điều khiển (Arduino IDE).....	64
4.2.1. Giới thiệu chung về Arduino IDE.....	64
4.2.2. Đặc điểm và chức năng chính của Arduino IDE.....	65
4.2.3. Cấu trúc chương trình trong Arduino IDE.....	66
4.3 Giao diện người dùng trên Blynk.....	67
4.3.1 Giới thiệu nền tảng Blynk.....	67
4.3.2 Ứng dụng trong đồ án.....	68
4.3.3 Cấu trúc giao diện Blynk đã xây dựng.....	68
CHƯƠNG 5: KẾT QUẢ THI CÔNG.....	70
5.1. Hình ảnh sản phẩm thực tế:.....	70
5.2. Kết quả thực nghiệm.....	71
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	73
6.1. Những vấn đề đã giải quyết trong đồ án.....	73
6.2. Ưu điểm, khuyết điểm của mô hình.....	73
6.3. Hướng phát triển.....	73
Phụ lục:.....	74
Tài liệu tham khảo.....	84

Mục lục hình ảnh

Hình 2.1 Cấu trúc vi một vi điều khiển ESP32-S3-WROOM.....	4
Hình 2.2 Sơ đồ khối của một vi điều khiển ESP32-S3-DevKitC-1.....	6
Hình 2.3 Mô tả sơ đồ chân của một vi điều khiển ESP32-S3-DevKitC-1.....	6
Hình 2.4 Sơ đồ mạch của cảm biến MAX30205.....	9
Hình 2.5 Cấu hình chân của cảm biến MAX30205.....	9
Hình 2.6 Cấu hình chân của cảm biến MAX30102.....	14
Hình 2.7 Sơ đồ chân của cảm biến MPU-6050.....	18
Hình 2.8 Hình ảnh các chiều của cảm biến MPU-6050.....	19
Hình 3.1 Sơ đồ khối tổng quát của mô hình.....	38
Hình 3.2 Cảm biến MAX30205.....	39
Hình 3.3 Kết nối MAX30205 với ESP32.....	40
Hình 3.4 Lưu đồ giải thuật hàm updateTemperature().....	41
Hình 3.5 Cảm biến MAX30102.....	41
Hình 3.6 Kết nối MAX30102 với ESP32.....	42
Hình 3.7 Lưu đồ giải thuật của hàm checkRealtimeHeartRate().....	43
Hình 3.8 Lưu đồ giải thuật hàm updateSpO2Measurement().....	44
Hình 3.9 Cảm biến MPU6050.....	45
Hình 3.10 Kết nối MPU6050 với ESP32.....	46
Hình 3.11 Chiều Vec-tơ gia tốc của cảm biến MPU6050.....	47
Hình 3.12 Lưu đồ giải thuật hàm checkMotionAndStartCollection().....	48
Hình 3.13a Lưu đồ giải thuật của hàm collectDataForAI().....	49
Hình 3.13b Lưu đồ giải thuật của hàm collectDataForAI().....	50
Hình 3.14 Cấu trúc mô hình Deep Learning.....	52
Hình 3.15 Biểu đồ sự thay đổi độ chính xác trên tập huấn luyện và đánh giá theo epoch.....	53
Hình 3.16 Sơ đồ kết nối chân toàn bộ mô hình.....	54
Hình 3.17 Lưu đồ giải thuật hàm manageSensorModes().....	55
Hình 3.18 Lưu đồ giả thuật của hàm checkVitalSignsAlerts().....	56
Hình 3.19 Lưu đồ giải thuật của chương trình chính.....	57
Hình 4.1 Giao diện của ứng dụng Blynk đã xây dựng.....	67
Hình 5.1 Sản phẩm khi được người dùng đeo lên người.....	69
Hình 5.2 Vị trí của các thành phần trong hệ thống.....	70

CHƯƠNG 1: GIỚI THIỆU

1.1 Đặt vấn đề

Trong những năm gần đây, tình trạng già hóa dân số đang diễn ra với tốc độ nhanh chóng trên phạm vi toàn cầu, trong đó có Việt Nam. Tỷ lệ người cao tuổi ngày càng tăng kéo theo nhu cầu chăm sóc sức khỏe và giám sát y tế cho nhóm đối tượng này trở thành một vấn đề cấp thiết mà xã hội đặc biệt quan tâm. Té ngã là một trong những nguyên nhân hàng đầu gây chấn thương, gãy xương, thậm chí tử vong ở người cao tuổi, đồng thời làm giảm sút nghiêm trọng chất lượng cuộc sống nếu không được phát hiện và xử lý kịp thời. Việc giám sát thủ công hoặc phụ thuộc hoàn toàn vào người thân, nhân viên y tế thường không đảm bảo kịp thời, đặc biệt trong bối cảnh nhiều người cao tuổi sống một mình hoặc thiếu sự theo dõi thường xuyên.

Sự phát triển của Internet of Things (IoT) kết hợp với trí tuệ nhân tạo (AI) và đặc biệt là các thuật toán Machine Learning đã mở ra những cơ hội mới trong việc xây dựng các hệ thống giám sát sức khỏe thông minh. Các hệ thống này cho phép thu thập, xử lý dữ liệu từ các cảm biến theo thời gian thực, đưa ra các phân tích và cảnh báo tức thì, giúp phát hiện sớm các vấn đề sức khỏe hoặc sự cố nguy hiểm như té ngã. Các giải pháp dựa trên IoT và Machine Learning không chỉ giúp người chăm sóc mà còn mang đến độ chính xác và khả năng hoạt động liên tục, nâng cao đáng kể hiệu quả chăm sóc sức khỏe cho người cao tuổi.

Xuất phát từ nhu cầu thực tiễn này, nhóm nghiên cứu thực hiện đề tài “Hệ Thống Theo Dõi Sức Khỏe Và Phát Hiện Té Ngã Ở Người Cao Tuổi Ứng Dụng Machine Learning” với mục tiêu xây dựng một giải pháp toàn diện giúp giám sát các chỉ số sức khỏe quan trọng như nhịp tim, nhiệt độ cơ thể và phát hiện các tình huống té ngã theo thời gian thực. Hệ thống được thiết kế dựa trên nền tảng IoT, sử dụng bộ vi điều khiển ESP32 kết hợp với các cảm biến gia tốc, nhịp tim, nhiệt độ để thu thập dữ liệu, đồng thời áp dụng các thuật toán Machine Learning nhằm phân tích tín hiệu và nhận diện sự kiện té ngã với độ chính xác cao. Dữ liệu và cảnh báo sẽ được truyền về ứng dụng giám sát từ xa như Blynk, cho phép người thân hoặc nhân viên y tế kịp thời can thiệp khi có tình huống nguy hiểm xảy ra.

1.2 Giải quyết vấn đề

Hệ thống được thiết kế với các thành phần chính nhằm thu thập dữ liệu và phát hiện té ngã theo thời gian thực. Cụ thể, hệ thống bao gồm:

- **Cảm biến gia tốc (MPU6050):** Module gia tốc 3 trục này được sử dụng để theo dõi chuyển động và phát hiện té ngã thông qua việc phân tích các đột biến gia tốc hoặc thay đổi tư thế. Dữ liệu gia tốc

sẽ là đầu vào quan trọng cho thuật toán học máy để phân tích hành vi người dùng.

- **Cảm biến nhịp tim và SpO₂ (MAX30102):** Đây là cảm biến quang học dùng để đo nhịp tim và độ bão hòa oxy trong máu (SpO₂). Thiết bị này tích hợp các đèn LED và photodiode để đo tín hiệu thay đổi của dòng máu, giúp giám sát liên tục tình trạng tim mạch của người cao tuổi. Thông tin về nhịp tim bất thường có thể cảnh báo dấu hiệu nguy hiểm.
- **Cảm biến nhiệt độ cơ thể (MAX30205):** MAX30205 là cảm biến nhiệt độ tiếp xúc có độ chính xác cao, được thiết kế chuyên biệt để đo nhiệt độ da người trong các ứng dụng theo dõi sức khỏe và thiết bị y tế. Cảm biến hoạt động bằng cách tiếp xúc trực tiếp với bề mặt da, từ đó cung cấp giá trị nhiệt độ với sai số thấp chỉ $\pm 0.1^{\circ}\text{C}$ trong dải nhiệt độ cơ thể người (từ 37°C đến 39°C).
- **Bộ xử lý trung tâm (ESP32):** Hệ thống sử dụng vi điều khiển ESP32 (một SoC tích hợp Wi-Fi/Bluetooth) để thu thập dữ liệu từ các cảm biến, xử lý tín hiệu sơ bộ và truyền kết quả. ESP32 được lựa chọn nhờ chi phí thấp và khả năng tiêu thụ điện năng thấp. Phần cứng được thiết kế gọn nhẹ, có thể đeo trên người, đảm bảo hoạt động ổn định liên tục và tiết kiệm năng lượng cho thời gian sử dụng dài hạn.
- **Mô hình Machine Learning:** Dữ liệu thu được từ các cảm biến sẽ được phân tích bằng mô hình học máy (ví dụ: cây quyết định, mạng nơ-ron) để nhận diện các hoạt động như đi bộ, đứng, nằm, và đặc biệt là té ngã. Việc kết hợp nhiều cảm biến với thuật toán Machine Learning giúp tăng độ chính xác và giảm tỷ lệ báo động giả trong việc phát hiện té ngã.
- **Ứng dụng giám sát từ xa (Blynk):** Nền tảng IoT Blynk (ứng dụng di động/web) được sử dụng để hiển thị liên tục các giá trị nhịp tim, thân nhiệt và trạng thái vận động. Khi hệ thống phát hiện té ngã hoặc dấu hiệu sức khỏe nguy hiểm, nó sẽ gửi cảnh báo khẩn cấp đến người thân hoặc bác sĩ qua ứng dụng, cho phép họ phản ứng kịp thời.

Như vậy, ý tưởng cốt lõi của đề tài là xây dựng một thiết bị đeo tích hợp đa cảm biến và sử dụng thuật toán AI để giám sát liên tục sức khỏe người cao tuổi và phát hiện chính xác sự kiện té ngã. Thiết bị này sẽ hoạt động độc lập, liên tục đo đạc các chỉ số (nhịp tim, thân nhiệt, gia tốc), phân tích dữ liệu tại chỗ bằng mô hình học máy để phát hiện rủi ro, và cảnh báo khẩn cấp ngay khi cần thiết. Nhờ đó, hệ thống giúp giảm thiểu các biến chứng do té ngã gây ra và đảm bảo an toàn cho người dùng.

Trong khi nhiều đề tài nghiên cứu trước đây thường tách rời hai chức năng giám sát sức khỏe và phát hiện té ngã thành hai hệ thống độc lập—một bên chỉ tập trung đo đạc nhịp tim, thân nhiệt hay SpO₂ và bên kia chỉ phát hiện té ngã dựa trên ngưỡng gia tốc cố định—đề tài này cung cấp một giải pháp tích hợp toàn diện. Thay vì chỉ cảnh báo khi gia tốc vượt qua một ngưỡng cố định, hệ

thống của chúng tôi khai thác dữ liệu từ nhiều cảm biến đồng thời và ứng dụng mô hình Machine Learning được huấn luyện trên tập dữ liệu thực tế. Nhờ vậy, tỷ lệ báo động giả được giảm thiểu đáng kể, đồng thời khả năng phát hiện té ngã thực sự được nâng cao.

1.3 Mục tiêu

Đề tài hướng tới xây dựng một hệ thống IoT theo dõi sức khỏe người cao tuổi với các mục tiêu chính sau:

- Giám sát liên tục các thông số sức khỏe và vận động: Hệ thống thu thập và theo dõi các chỉ số nhịp tim, nhiệt độ cơ thể và trạng thái vận động (đi bộ, đứng, nằm, té ngã) của người dùng theo thời gian thực.
- Độ chính xác cao trong phát hiện té ngã: Áp dụng học máy để nhận diện hành vi té ngã một cách chính xác với độ chính xác mục tiêu trên 90%. Phát hiện kịp thời các trường hợp té ngã nguy hiểm.
- Hiển thị thông tin và cảnh báo thời gian thực: Các chỉ số sức khỏe được hiển thị liên tục trên ứng dụng Blynk. Khi phát hiện té ngã hoặc dấu hiệu bất thường (như nhịp tim tăng cao), hệ thống gửi thông báo khẩn cấp tức thì đến điện thoại của người thân hoặc nhân viên y tế.
- Theo dõi từ xa: Cho phép người giám sát (gia đình, bác sĩ) theo dõi tình trạng của người cao tuổi từ xa qua thiết bị di động hoặc máy tính kết nối Internet.
- Thiết bị đeo tiện lợi và tiết kiệm năng lượng: Thiết bị có thiết kế gọn nhẹ để đeo được trên người, hoạt động ổn định trong thời gian dài với mức tiêu thụ điện năng thấp (nhờ sử dụng ESP32). Điều này đảm bảo người dùng thoải mái khi mang theo và kéo dài thời gian vận hành trước khi cần sạc lại.

Kết quả mong muốn của đề tài là xây dựng được một hệ thống thông minh, hoạt động ổn định, dễ triển khai và sử dụng, có khả năng giám sát liên tục tình trạng sức khỏe và phát hiện chính xác té ngã của người cao tuổi. Giải pháp này không chỉ góp phần giảm thiểu rủi ro do chậm trễ trong phát hiện tai nạn mà còn nâng cao chất lượng cuộc sống cho người cao tuổi, giảm gánh nặng cho gia đình và đội ngũ y tế, đồng thời khẳng định tiềm năng ứng dụng rộng rãi của công nghệ IoT và Machine Learning trong lĩnh vực chăm sóc sức khỏe cộng đồng.

1.4 Giới hạn

Do giới hạn về thời gian và kinh phí, hệ thống trong đề tài tập trung vào thu thập các dữ liệu cơ bản (nhịp tim, thân nhiệt, gia tốc), phát hiện té ngã và hiển thị thông tin. Các tính năng nâng cao như gọi cấp cứu tự động khi có tai nạn hoặc lưu trữ dài hạn trên máy chủ đám mây chưa được triển khai trong giai đoạn hiện tại.

1.5 Bố cục trình bày của đề tài

Chương 1: GIỚI THIỆU

Chương 2: CƠ SỞ LÝ THUYẾT

Chương 3: TÍNH TOÁN VÀ THIẾT KẾ MÔ HÌNH

Chương 4: GIỚI THIỆU PHẦN MỀM

Chương 5: KẾT QUẢ THI CÔNG

Chương 6 : KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

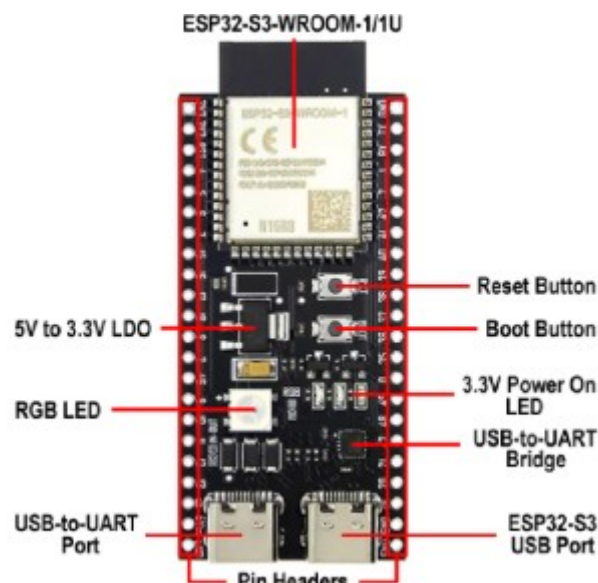
2.1. Tìm hiểu vi điều khiển ESP32 S3

2.1.1 Giới thiệu

Một vi điều khiển (microcontroller) là một con chip tích hợp nhỏ gọn, được thiết kế để điều khiển các thiết bị điện tử thông qua việc xử lý tín hiệu đầu vào và điều khiển đầu ra. Khác với máy tính thông thường, vi điều khiển thường được tích hợp vào bên trong các thiết bị như máy giặt, tủ lạnh, đồng hồ thông minh hay các hệ thống IoT, nhằm thực hiện các nhiệm vụ cụ thể một cách tự động và tiết kiệm năng lượng. Chúng thường bao gồm bộ xử lý (CPU), bộ nhớ (RAM, Flash), các cổng giao tiếp (UART, SPI, I²C ...), và các chân vào/ra (GPIO) trên cùng một con chip.

ESP32-S3-WROOM-1 là một module vi điều khiển hiệu năng cao do hãng Espressif Systems phát triển, thuộc dòng ESP32 mới nhất và được tối ưu cho các ứng dụng IoT kết hợp trí tuệ nhân tạo (AIoT). Con vi điều khiển này tích hợp bộ xử lý lõi kép Xtensa LX7 với tốc độ lên tới 240 MHz, hỗ trợ cả Wi-Fi và Bluetooth 5.0, cùng khả năng mở rộng bộ nhớ flash và PSRAM giúp nó xử lý các tác vụ phức tạp như nhận dạng âm thanh, xử lý tín hiệu sinh học, hay phát hiện hành vi theo thời gian thực. Bên cạnh đó, ESP32-S3 còn nổi bật với khả năng hỗ trợ tăng tốc phần cứng cho các tác vụ machine learning thông qua các lệnh vector chuyên dụng, giúp cải thiện hiệu suất đáng kể trong các ứng dụng như phát hiện té ngã, nhận diện giọng nói hay điều khiển thiết bị thông minh. Nhờ vào tính linh hoạt, hiệu năng cao và khả năng giao tiếp không dây mạnh mẽ, ESP32-S3-WROOM-1 là lựa chọn lý tưởng cho các hệ thống IoT hiện đại, đặc biệt là những hệ thống cần xử lý tín hiệu phức tạp hoặc yêu cầu phản ứng theo thời gian thực.

2.1.2 Cấu trúc vi điều khiển



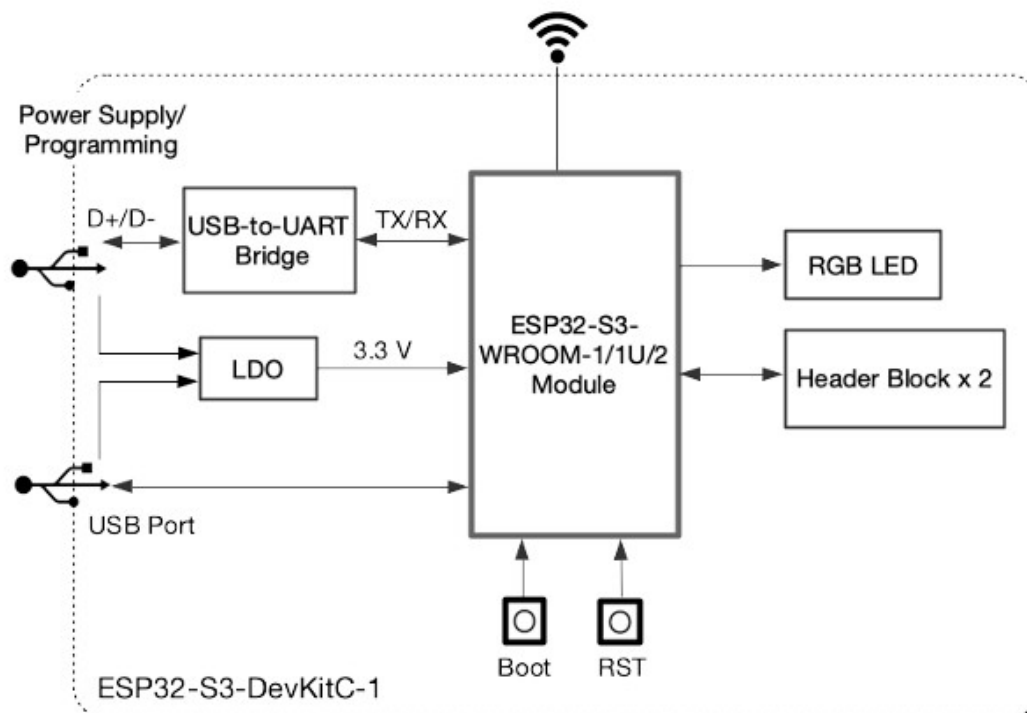
Hình 2.1 Cấu trúc vi một vi điều khiển ESP32-S3-WROOM

Các thành phần chính:

- Module chính: ESP32-S3-WROOM-1 là dòng module vi điều khiển tích hợp Wi-Fi và Bluetooth Low Energy (BLE) do Espressif phát triển. WROOM-1: Sử dụng anten PCB tích hợp sẵn trên module. Tất cả các phiên bản đều được trang bị khả năng xử lý tín hiệu số, hỗ trợ mạng nơ-ron và các ứng dụng đòi hỏi tính toán cao như trí tuệ nhân tạo (AI) và học máy (ML).
- Bộ điều chỉnh điện áp 5V xuống 3.3V (LDO): Đây là mạch nguồn hạ áp từ 5V đầu vào xuống 3.3V – mức điện áp hoạt động tiêu chuẩn của ESP32-S3. Mạch này đảm bảo chip và các thành phần ngoại vi nhận được nguồn ổn định, phù hợp cho vận hành liên tục
- Các hàng chân kết nối (Pin Headers): Board mạch cung cấp đầy đủ các chân GPIO (ngoại trừ những chân dành riêng cho bộ nhớ flash SPI) được đưa ra thông qua các hàng chân (header) hai bên. Những chân này phục vụ cho mục đích giao tiếp, lập trình và mở rộng với các thiết bị ngoại vi như cảm biến, màn hình, relay, v.v.
- Cổng USB-to-UART: Đây là cổng Micro-USB chính, được sử dụng để: Cấp nguồn 5V cho board, Giao tiếp nối tiếp (UART) giữa máy tính và vi điều khiển, tải firmware vào chip thông qua bộ chuyển USB-to-UART tích hợp. Cổng này rất quan trọng trong giai đoạn lập trình và gỡ lỗi hệ thống.
- Nút Boot: Là nút tải chương trình (Download Button). Khi nhấn giữ nút này đồng thời nhấn Reset, vi điều khiển sẽ chuyển sang chế độ tải firmware qua cổng UART. Đây là bước cần thiết trong quá trình lập trình chip lần đầu hoặc khi cập nhật firmware thủ công.
- Nút Reset: Nút này được sử dụng để khởi động lại toàn bộ hệ thống. Khi nhấn, nó sẽ thiết lập lại vi điều khiển mà không cần rút nguồn.
- Cổng USB OTG: Đây là cổng USB tốc độ cao, tuân thủ chuẩn USB 1.1. Cổng này hỗ trợ nhiều chức năng: Cấp nguồn cho board, giao tiếp trực tiếp với chip ESP32-S3 để lập trình và debug, hỗ trợ giao tiếp JTAG để gỡ lỗi chuyên sâu, có thể được sử dụng như thiết bị USB ngoại vi hoặc host tùy theo cấu hình phần mềm.
- Bộ chuyển USB-to-UART: Cầu chuyển đổi tín hiệu từ USB sang UART được tích hợp trên board, cho phép truyền dữ liệu tốc độ cao lên đến 3 Mbps. Thành phần này đóng vai trò quan trọng trong việc lập trình, giao tiếp và điều khiển chip từ máy tính.

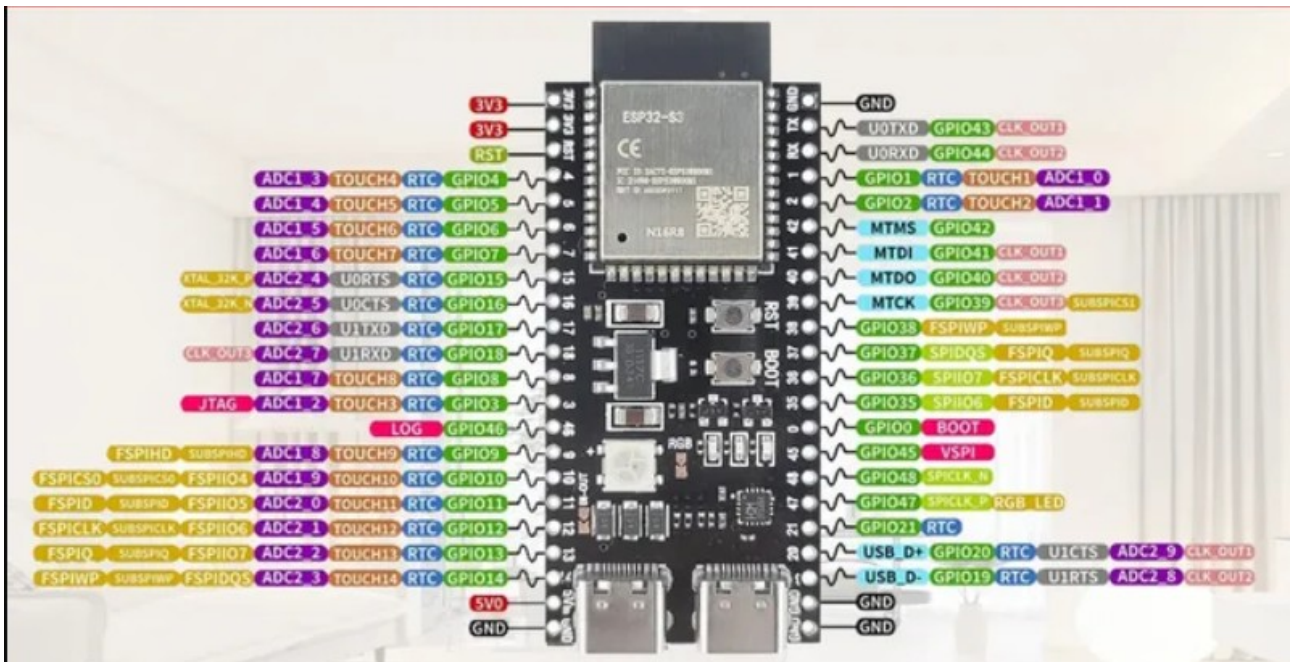
- Đèn LED RGB: Board có tích hợp một đèn LED RGB điều khiển được, sử dụng chân GPIO38. Đèn LED này có thể phát ra các màu sắc khác nhau, hữu ích trong việc hiển thị trạng thái hoặc báo hiệu theo yêu cầu người dùng.
- Đèn báo nguồn 3.3V (3.3V Power On LED): Khi board được cấp nguồn qua cổng USB, đèn LED này sẽ sáng để báo hiệu rằng điện áp 3.3V đã sẵn sàng. Đây là chỉ báo trực quan cho biết board đã được cấp nguồn thành công

Sơ đồ khối:



Hình 2.2 Sơ đồ khối của một vi điều khiển ESP32-S3-DevKitC-1

Mô tả sơ đồ chân



Hình 2.3 Mô tả sơ đồ chân của một vi điều khiển ESP32-S3-DevKitC-1

*Tổng quan về hệ thống chân (Pins)

Bo mạch có hai hàng chân (headers): J1 và J3

Gồm 44 chân, hỗ trợ nhiều chức năng:

- I/O đa năng
- Nguồn cấp
- UART, SPI, ADC, Touch, USB, v.v.

KHOẢNG CHÂN J1 (J1 Header - bên trái bo mạch)

STT	Tên chân	Loại	Mô tả chức năng
1-2	3V3	P	Cấp nguồn 3.3V
3	RST	I	Chân reset (EN)
4-7	GPIO4- GPIO7	I/O/T	GPIO, RTC_GPIO, Touch4-7, ADC1_CH3-6
8	GPIO15	I/O/T	U0RTS, ADC2_CH4, XTAL_32K_P
9	GPIO16	I/O/T	U0CTS, ADC2_CH5, XTAL_32K_N
10	GPIO17	I/O/T	UART1 TXD, ADC2_CH6
11	GPIO18	I/O/T	UART1 RXD, ADC2_CH7, CLK_OUT3
12	GPIO8	I/O/T	Touch8, ADC1_CH7, SPI CS
13	GPIO3	I/O/T	Touch3, ADC1_CH2

STT	Tên chân	Loại	Mô tả chức năng
14	GPIO46	I/O/T	GPIO thông thường
15	GPIO9	I/O/T	Touch9, ADC1_CH8, FSPIHD
16	GPIO10	I/O/T	Touch10, ADC1_CH9, SPI CS0
17	GPIO11	I/O/T	Touch11, ADC2_CH0, FSPI D
18	GPIO12	I/O/T	Touch12, ADC2_CH1, SPI CLK
19	GPIO13	I/O/T	Touch13, ADC2_CH2, SPI Q
20	GPIO14	I/O/T	Touch14, ADC2_CH3, SPI WP
21	5V	P	Cấp nguồn 5V
22	G	G	Ground (mass)

Bảng 2.1 Mô tả chân ở khối J1-bên trái bo mạch

KHỐI CHÂN J3 (J3 Header - bên phải bo mạch)

STT	Tên chân	Loại	Mô tả chức năng
1	G	G	Ground
2	TX (GPIO43)	I/O/T	UART0 TXD, CLK_OUT1
3	RX (GPIO44)	I/O/T	UART0 RXD, CLK_OUT2
4	GPIO1	I/O/T	Touch1, ADC1_CH0
5	GPIO2	I/O/T	Touch2, ADC1_CH1
6	GPIO42	I/O/T	JTAG MTMS
7	GPIO41	I/O/T	JTAG MTDI, CLK_OUT1
8	GPIO40	I/O/T	JTAG MTDO, CLK_OUT2
9	GPIO39	I/O/T	JTAG MTCK, CLK_OUT3, SPI CS
10	GPIO38	I/O/T	LED RGB (v1.1), SPI WP
11	GPIO37	I/O/T	SPI Q
12	GPIO36	I/O/T	SPI CLK
13	GPIO35	I/O/T	SPI D
14	GPIO0	I/O/T	RTC_GPIO0
15	GPIO45	I/O/T	GPIO thông thường

STT	Tên chân	Loại	Mô tả chức năng
16	GPIO48	I/O/T	LED RGB (v1.0), SPICLK_N
17	GPIO47	I/O/T	SPICLK_P
18	GPIO21	I/O/T	RTC_GPIO21
19	GPIO20	I/O/T	U1CTS, ADC2_CH9, USB D+
20	GPIO19	I/O/T	U1RTS, ADC2_CH8, USB D-
21-22	G	G	Ground

Bảng 2.2 Mô tả chân ở khối J3-bên phải bo mạch

2.2 Cảm biến nhiệt độ MAX30205

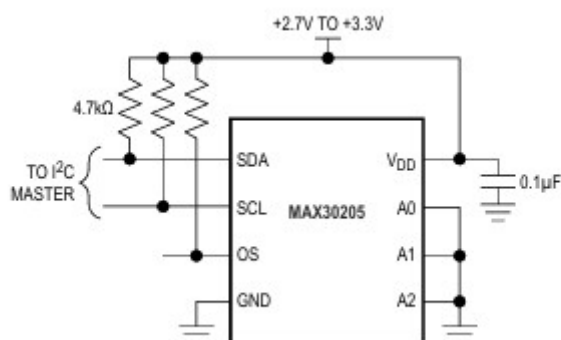
2.2.1 Giới thiệu

Cảm biến nhiệt độ MAX30205 đo chính xác nhiệt độ và cung cấp đầu ra cảnh báo/ngắt/tắt máy khi quá nhiệt. Thiết bị này chuyển đổi các phép đo nhiệt độ sang dạng số thông qua bộ chuyển đổi tương tự-số (ADC) sigma-delta có độ phân giải cao. Độ chính xác đáp ứng tiêu chuẩn nhiệt kế lâm sàng ASTM E1112 khi được hàn trên PCB cuối.

Giao tiếp thông qua giao diện nối tiếp 2 dây tương thích I²C . Giao diện này chấp nhận các lệnh ghi byte, đọc byte, gửi byte và nhận byte để đọc dữ liệu nhiệt độ và cấu hình hoạt động của đầu ra tắt máy khi quá nhiệt (open-drain).

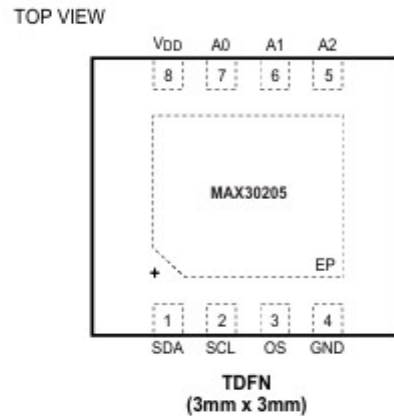
MAX30205 có ba chân chọn địa chỉ, hỗ trợ tối đa 32 địa chỉ I²C . Cảm biến hoạt động với điện áp 2.7V–3.3V, dòng tiêu thụ thấp 600μA, và giao diện I²C có bảo vệ chống treo, phù hợp cho các ứng dụng y tế và thiết bị đeo.

Thiết bị đóng gói TDFN 8 chân, hoạt động trong dải nhiệt độ 0°C đến +50°C.



Hình 2.4 Sơ đồ mạch của cảm biến MAX30205

2.2.2 Cấu hình chân



Hình 2.5 Cấu hình chân của cảm biến MAX30205

SDA (Pin 1)

Chức năng: Đường dữ liệu I²C (Serial Data Input/Output).

Đặc điểm:

- Hoạt động kiểu open-drain, nghĩa là không tự tạo mức logic cao, cần điện trở kéo lên (pull-up) để hoạt động
- Có khả năng truyền và nhận dữ liệu trên bus I²C.
- Điện áp làm việc: 0–3.3V (high impedance khi không hoạt động).

SCL (Pin 2)

Chức năng: Đường xung nhịp I²C (Serial Clock Input).

Đặc điểm:

- Cũng là open-drain, cần điện trở kéo lên (pull-up) để tạo mức logic cao.
- Được master điều khiển để đồng bộ truyền dữ liệu trên SDA.
- Điện áp làm việc: 0–3.3V (high impedance khi không hoạt động).

OS (Pin 3)

Chức năng: Ngõ ra cảnh báo quá nhiệt (Overtemperature Shutdown Output).

Đặc điểm:

- Kiểu open-drain, cần kéo lên bằng điện trở để đọc trạng thái.
- Khi cảm biến phát hiện nhiệt độ vượt ngưỡng, chân này được kích hoạt để cảnh báo hoặc điều khiển tắt hệ thống.

GND (Pin 4)

Chức năng: Chân nối đất (Ground).

Đặc điểm:

- Điểm tham chiếu điện áp cho toàn bộ mạch.
- Kết nối với mass của hệ thống.

A2 (Pin 5)

Chức năng: Bit địa chỉ phụ I²C (I²C Slave Address Input).

Đặc điểm:

- Được dùng để thiết lập địa chỉ I²C (cùng với A1, A0).
- Có thể kết nối tới: GND, VDD, SDA hoặc SCL để chọn địa chỉ.
- Không được để hở (floating).

A1 (Pin 6)

Chức năng: Bit địa chỉ phụ I²C.

Đặc điểm:

- Tương tự A2, giúp xác định địa chỉ thiết bị trên bus I²C.
- Có thể nối GND, VDD, SDA hoặc SCL.
- Không được để hở, điện áp hoạt động từ 0–3.3V.

A0 (Pin 7)

Chức năng: Bit địa chỉ phụ I²C cuối cùng.

Đặc điểm:

- Cùng với A2 và A1, tạo thành 3-bit địa chỉ phụ, cho phép nhiều thiết bị giống nhau hoạt động trên cùng bus I²C.
- Kết nối GND, VDD, SDA hoặc SCL.
- Không được để hở, điện áp hoạt động từ 0–3.3V.

VDD (Pin 8)

Chức năng: Chân cấp nguồn 3.3V cho IC.

Đặc điểm:

- Nguồn cung cấp chính, yêu cầu tụ lọc 0.1μF nối giữa VDD và GND để giảm nhiễu.
- Điện áp hoạt động 3.3V.

EP (Exposed Pad - không đánh số)

Chức năng: Pad tản nhiệt hoặc nối mass ở đáy package.

Đặc điểm:

- Cần nối với GND để giảm nhiễu và hỗ trợ tản nhiệt cho IC.

2.2.3 Thông số điện (Electrical Characteristics)

Thermometer Error (Độ sai số đo nhiệt độ)

- Đây là độ chính xác của cảm biến trong các dải nhiệt độ khác nhau.
- Giá trị:
 - +37°C đến +39°C: Sai số $\pm 0.1^\circ\text{C}$ (rất chính xác, phù hợp đo thân nhiệt).
 - +35°C đến +41°C: Sai số $\pm 0.15^\circ\text{C}$.
 - +20°C đến +50°C: Sai số $\pm 0.2^\circ\text{C}$.
 - 0°C đến +70°C: Sai số $\pm 0.3^\circ\text{C}$.
 - -40°C đến +105°C: Sai số $\pm 0.5^\circ\text{C}$.
- → Độ chính xác cao nhất trong khoảng nhiệt độ cơ thể (35–41°C).

ADC Repeatability (Độ lặp lại phép đo)

- Độ phân giải ADC 16-bit cho mỗi lần đọc dữ liệu (1 sigma).
- Độ lặp lại tốt giúp giảm nhiễu và sai số ngẫu nhiên.

Temperature Data Resolution (Độ phân giải dữ liệu)

- 0.00390625°C/bit (1/256°C), tương đương 16-bit.
- Giúp đo nhiệt độ rất chi tiết (phù hợp cho y tế).

Conversion Time (Thời gian chuyển đổi)

- 44 - 50ms cho một lần đo.
- Tốc độ này phù hợp để theo dõi liên tục mà không tốn quá nhiều năng lượng.

First Conversion Completed (Thời gian đo đầu tiên): 50ms sau khi khởi động hoặc reset.

Quiescent Supply Current (Dòng tiêu thụ tĩnh - I_{DD})

- Ở điều kiện hoạt động: 1.0μA đến 2.5μA (rất thấp).
- Giúp tiết kiệm năng lượng, thích hợp cho thiết bị đeo (wearable).

OS Delay (Độ trễ cảnh báo quá nhiệt): Thời gian từ khi phát hiện nhiệt độ vượt ngưỡng tới khi ngõ OS kích hoạt: 35μs.

POR Threshold (Ngưỡng reset nguồn): 1.6V – IC tự reset nếu điện áp VDD giảm dưới mức này.

Input Logic Levels (Ngưỡng mức logic I²C)

- Mức cao (VIH): tối thiểu 1.26V.
- Mức thấp (VIL): tối đa 0.54V.
- Tương thích chuẩn I²C với điện áp 3.3V.

Dòng tiêu thụ ở chân I²C (Input Leakage Current): $\pm 0.2\mu\text{A}$ (rất nhỏ, giúp giảm hao pin).

OS Output Characteristics (Ngõ ra cảnh báo quá nhiệt)

- Dòng rò: $\pm 0.2\mu\text{A}$.
- Điện áp mức thấp (VOL) khi kéo dòng 4mA: 0.4V max.

Ý nghĩa tổng quát:

- Độ chính xác rất cao trong vùng nhiệt độ cơ thể ($\pm 0.1^\circ\text{C}$).
- Tiêu thụ dòng siêu thấp, phù hợp thiết bị đeo.
- Đáp ứng nhanh (44ms mỗi phép đo) và hỗ trợ giao tiếp I²C chuẩn 3.3V.
- Có ngõ cảnh báo quá nhiệt (OS) và tự reset khi sụt nguồn.

2.2.4 Nguyên lý hoạt động

Chức năng chính:

- Đo nhiệt độ của môi trường hoặc cơ thể (qua tiếp xúc) bằng cách cảm nhận nhiệt độ của chính die bên trong IC.
- Nhiệt độ được chuyển đổi sang tín hiệu số 16-bit nhờ bộ ADC sigma-delta ($\Sigma\Delta$) có độ phân giải $0.00390625^\circ\text{C/bit}$.
- Giao tiếp với vi điều khiển qua giao diện I²C (2 dây: SDA, SCL) để đọc giá trị nhiệt độ hoặc cấu hình.
- Có chân OS (Overtemperature Shutdown) để phát cảnh báo hoặc ngắt hệ thống khi nhiệt độ vượt ngưỡng.

Quy trình đo và xử lý dữ liệu:

- Cảm biến nhiệt độ đo trực tiếp nhiệt độ của chip (die).
- Nhiệt độ được số hóa nhờ ADC 16-bit, sau đó lưu vào thanh ghi Temperature (00h).
- Vi điều khiển có thể truy xuất dữ liệu qua I²C bằng các lệnh read/write.
- Có thể thiết lập ngưỡng cảnh báo (TOS – nhiệt độ quá cao, THYST – ngưỡng ngắt cảnh báo) qua thanh ghi tương ứng.

Chân OS sẽ:

- Hoạt động ở Comparator mode (như một thermostat): OS kích hoạt khi nhiệt độ $> \text{TOS}$, tắt khi $< \text{THYST}$.
- Hoặc Interrupt mode: OS giữ trạng thái cho đến khi vi điều khiển đọc thanh ghi để xóa cờ.

Các chế độ hoạt động quan trọng:

- Normal Mode: Đo liên tục và cập nhật thanh ghi nhiệt độ mỗi $\sim 44\text{ms}$.
- Shutdown Mode: IC tiêu thụ dòng cực thấp ($\leq 3.5\mu\text{A}$) nhưng vẫn giữ dữ liệu trong thanh ghi.

- One-Shot Mode: Kích hoạt một phép đo đơn lẻ (tiết kiệm năng lượng), sau đó IC trở lại shutdown.

Giao tiếp và cấu hình:

- Sử dụng I²C tốc độ tối đa 400kHz.
- Có 3 chân chọn địa chỉ (A0, A1, A2) để hỗ trợ nhiều thiết bị trên cùng bus (32 địa chỉ).
- Có các thanh ghi chính:
 - Temperature Register (00h) – chỉ đọc, chứa giá trị nhiệt độ 16-bit.
 - Configuration Register (01h) – cấu hình các chế độ (shutdown, one-shot, interrupt/comparator, định dạng dữ liệu, fault queue...).
 - THYST (02h) và TOS (03h) – lưu giá trị ngưỡng nhiệt độ để điều khiển chân OS.

Ứng dụng thực tế

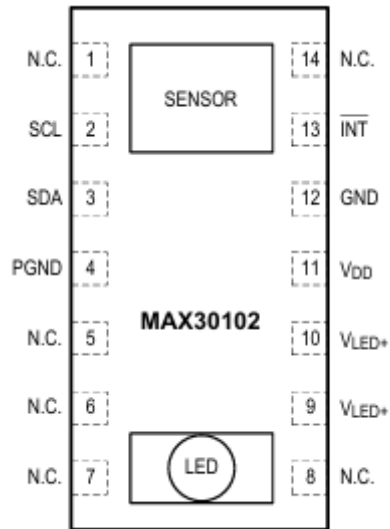
- Đo đo chính xác ($\pm 0.1^{\circ}\text{C}$ từ 37°C đến 39°C), IC này thường được dùng trong:
- Thiết bị theo dõi sức khỏe (wearable, IoT).
- Máy đo thân nhiệt, thiết bị y tế.
- Giám sát nhiệt độ trong hệ thống nhạy cảm (có cảnh báo OS).

2.3 Cảm biến nhịp tim và nồng độ Oxy trong máu MAX30102

2.3.1 Mô tả chung

MAX30102 là một module đo độ bão hòa oxy trong máu và nhịp tim tích hợp. Nó bao gồm các đèn LED nội bộ, cảm biến quang, các yếu tố quang học và điện tử với độ ồn thấp có khả năng loại bỏ ánh sáng xung quanh. MAX30102 cung cấp một giải pháp hệ thống hoàn chỉnh để đơn giản hóa quy trình thiết kế cho các thiết bị di động và đeo trên người. MAX30102 hoạt động trên nguồn cung cấp 1.8V đơn và nguồn cung cấp 3.3V riêng cho các đèn LED nội bộ. Giao tiếp diễn ra qua giao diện tương thích I²C chuẩn. Module có thể được tắt qua phần mềm với dòng chờ bằng không, cho phép các đường nguồn luôn được cung cấp điện.

2.3.2 Cấu hình chân



Hình 2.6 Cấu hình chân của cảm biến MAX30102

*Phân tích chức năng các chân của IC

Chân 1, 5, 6, 7, 8, 14 - N.C. (No Connection)

- Đây là các chân không kết nối mạch điện bên trong IC, nhưng vẫn cần hàn vào pad PCB để tăng độ bền cơ học và tản nhiệt.
- Không nên cấp điện hay nối tín hiệu vào các chân này.
- Một số datasheet khuyến nghị nối các chân này với mass thông qua pad để chống nhiễu cơ học.

Chân 2 - SCL (I²C Clock Input)

- Đây là chân xung nhịp cho giao tiếp I²C, nhận tín hiệu clock từ vi điều khiển (master).
- Dùng để đồng bộ hóa dữ liệu trên SDA.
- Cần điện trở pull-up (thường 4.7kΩ-10kΩ) kéo lên VDD để bus I²C hoạt động đúng.

Chân 3 - SDA (I²C Data, Bidirectional - Open-Drain)

- Chân truyền và nhận dữ liệu cho giao tiếp I²C.
- Là ngõ ra kiểu open-drain, cần điện trở pull-up lên VDD.
- Hoạt động song công (bidirectional) để giao tiếp với vi điều khiển (MCU).

Chân 4 - PGND (Power Ground)

- Là mass nguồn dành cho khối driver LED.
- Thường tách riêng với GND (Analog Ground) để giảm nhiễu cho mạch analog.

Chân 9 & 10 - VLED+ (LED Power Supply, Anode Connection)

- Nguồn cấp cho LED (cực dương/anode).
- Cần tụ bypass (thường $0.1\mu\text{F}$ và/hoặc $10\mu\text{F}$) nối từ VLED+ xuống PGND để lọc nhiễu, ổn định điện áp.
- Thường nối tới nguồn riêng cho LED (ví dụ 3.3V hoặc 5V) để không gây ảnh hưởng mạch chính.

Chân 11 - VDD (Analog Power Supply Input)

- Nguồn cấp chính cho mạch analog và logic của IC.
- Thường yêu cầu tụ bypass $0.1\mu\text{F}$ gần chân IC, nối về GND để giảm nhiễu.

Chân 12 - GND (Analog Ground)

- Mass cho phần mạch logic và analog.
- Nên nối về điểm mass chung, tách biệt với PGND nếu cần giảm nhiễu.

Chân 13 - INT (Interrupt, Active-Low, Open-Drain)

- Chân ngắt đầu ra, kích hoạt mức thấp (active-low).
- Thông báo cho vi điều khiển (MCU) khi có sự kiện (ví dụ: dữ liệu sẵn sàng hoặc lỗi).
- Cần điện trở pull-up nối với VDD để hoạt động.

Lưu ý khi thiết kế mạch:

- Các chân SDA, SCL và INT là open-drain, nên cần điện trở kéo lên VDD (thường $4.7\text{k}\Omega$ – $10\text{k}\Omega$).
- Tách PGND và GND trên layout để giảm nhiễu, chỉ nối tại một điểm (star ground).
- Đặt tụ bypass gần các chân VDD và VLED+ để ổn định nguồn.

2.3.3 Thông số điện

Điện áp và nguồn cấp

- VDD (Analog Power Supply): 1.7V – 2.0V (typical 1.8V) – dùng cho mạch logic và analog.
- VLED+ (LED Power Supply): 3.1V – 5.0V (typical 3.3V) – nguồn riêng cấp cho LED hồng ngoại và LED đỏ.
- GND – PGND: Tách riêng để giảm nhiễu (PGND cho driver LED, GND cho mạch logic).

Dòng tiêu thụ

- SpO2 và HR mode (IR + RED): 600 – 1200 μA (ở PW = 215 μs , 50sps).
- Chế độ chỉ IR: 600 – 1200 μA .
- Shutdown mode: 0.7 μA (typical) – 10 μA (max).

Giới hạn tuyệt đối (Absolute Maximum Ratings)

- VDD tới GND: -0.3V đến +2.2V.
- VLED+ tới PGND: -0.3V đến +6V.
- Tất cả chân khác tới GND: -0.3V đến +6V.
- Dòng liên tục qua mỗi chân: $\pm 20\text{mA}$.
- Nhiệt độ hoạt động: -40°C đến $+85^{\circ}\text{C}$.
- Nhiệt độ lưu trữ: -40°C đến $+105^{\circ}\text{C}$.

Đặc tính cảm biến và ADC

- Độ phân giải ADC: 18-bit sigma-delta ADC.
- Tần số lấy mẫu ADC: 50sps đến 3200sps (lập trình được).
- Dải tín hiệu ADC (Full-scale): $2\mu\text{A}$ đến $16\mu\text{A}$ (lập trình qua thanh ghi).
- Thời gian tích hợp (tùy Pulse Width): $69\mu\text{s}$ – $411\mu\text{s}$ (tương ứng 15-bit đến 18-bit độ phân giải).

Đặc tính LED

- LED IR:
 - Bước sóng đỉnh: 870 – 900nm (typical 880nm).
 - Điện áp thuận VF: 1.4V (ở 20mA).
 - Công suất bức xạ: $\sim 6.5\text{mW}$ (ở 20mA).
- LED RED:
 - Bước sóng đỉnh: 650 – 670nm (typical 660nm).
 - Điện áp thuận VF: 2.1V (ở 20mA).
 - Công suất bức xạ: $\sim 9.8\text{mW}$ (ở 20mA).

Dòng LED lập trình: 0 – 50mA (qua thanh ghi, 8-bit, max 51mA ở 0xFF).

Photodiode

- Dải nhạy sáng (QE > 50%): 600 – 900nm.
- Diện tích nhạy sáng: 1.36mm^2 ($1.38 \times 0.98\text{mm}$).

Giao tiếp số (I²C và ngắt)

- Tốc độ I²C: 0 – 400kHz.
- Ngưỡng điện áp logic:
- $V_{IH} \geq 0.7 \times V_{DD}$ (High).
- $V_{IL} \leq 0.3 \times V_{DD}$ (Low).
- INT (Interrupt): Active-Low, open-drain, cần điện trở pull-up.

2.3.4 Nguyên lý hoạt động

a. Cấu tạo chính

IC MAX30102 tích hợp: 2 LED (đỏ 660nm và hồng ngoại 880nm) để chiếu ánh sáng xuyên qua hoặc phản xạ từ da (thường ở ngón tay).

Photodiode nhạy quang (600–900nm) để thu ánh sáng phản xạ/truyền qua mô.

Mạch khuếch đại, lọc và ADC 18-bit.

Bộ hủy nhiễu ánh sáng môi trường (Ambient Light Cancellation – ALC).

Khối giao tiếp I²C và bộ nhớ FIFO (32 mẫu).

b. Cách đo nhịp tim và SpO₂

Nguyên tắc quang học: Khi chiếu ánh sáng đỏ và hồng ngoại vào mô, hemoglobin (Hb) và oxyhemoglobin (HbO₂) hấp thụ ánh sáng khác nhau theo bước sóng.

Tín hiệu phản xạ biến thiên theo nhịp tim và nồng độ oxy máu (SpO₂):

- LED phát xung theo chu kỳ ngắn (69μs–411μs) với dòng điều khiển (0–50mA).
- Photodiode thu ánh sáng phản xạ, chuyển thành dòng điện.
- Dòng này được khuếch đại, lọc (loại bỏ nhiễu DC và ánh sáng môi trường) và đưa vào ADC sigma-delta 18-bit.

c. Xử lý tín hiệu trong IC

Ambient Light Cancellation (ALC): triệt tiêu dòng DC do ánh sáng ngoài môi trường (có thể đến 200μA).

ADC và Digital Filter: chuyển tín hiệu quang thành dữ liệu số (50 – 3200 mẫu/giây).

Dữ liệu được lưu tạm trong FIFO 32 mẫu, giảm tải cho MCU.

d. Giao tiếp và điều khiển

MCU giao tiếp qua I²C (400kHz) để:

- Đọc dữ liệu nhịp tim/SpO₂ từ FIFO.
- Cấu hình dòng LED, tốc độ lấy mẫu, độ phân giải ADC, chế độ đo (HR mode, SpO₂ mode).
- Đọc nhiệt độ chip để bù nhiệt (ảnh hưởng bước sóng LED).

Chân INT (Interrupt, active-low) báo MCU khi: Có dữ liệu mới, FIFO đầy gần hết, đo nhiệt độ xong, hoặc có lỗi ALC.

e. Các chế độ hoạt động chính

Heart Rate Mode: chỉ dùng LED đỏ để lấy tín hiệu PPG đo nhịp tim.

SpO₂ Mode: dùng cả LED đỏ và IR để tính tỷ lệ hấp thụ và xác định nồng độ oxy.

Multi-LED Mode: hỗ trợ slot thời gian linh hoạt (tối đa 4 kênh) cho các ứng dụng tùy chỉnh.

Shutdown Mode: giảm dòng tiêu thụ xuống ~0.7μA khi không hoạt động.

f. Nguyên tắc cốt lõi:

MAX30102 đo biến thiên cường độ ánh sáng phản xạ từ mô (thay đổi theo nhịp tim và SpO₂), xử lý và số hóa tín hiệu, sau đó gửi về MCU qua I²C. MCU sẽ dùng thuật toán (thường chạy ngoài IC) để tính toán nhịp tim và nồng độ oxy.

2.4 Cảm biến gia tốc và góc quay MPU6050

2.4.1 Mô tả chung

MPU6050 là một cảm biến chuyển động tích hợp do InvenSense (nay thuộc TDK) sản xuất, kết hợp gia tốc kế 3 trục (3-axis accelerometer) và con quay hồi chuyển 3 trục (3-axis gyroscope) trong một vi mạch duy nhất. Thiết bị này cho phép đo gia tốc tuyến tính, vận tốc góc và có thể suy ra các thông số về tư thế (orientation) hoặc phát hiện chuyển động của đối tượng.

Cảm biến sử dụng bộ chuyển đổi tương tự-số (ADC) 16-bit để cung cấp dữ liệu chính xác, với dải đo có thể lựa chọn:

- Gia tốc: $\pm 2\text{ g}$, $\pm 4\text{ g}$, $\pm 8\text{ g}$, $\pm 16\text{ g}$.
- Vận tốc góc: $\pm 250\text{ }^\circ/\text{s}$, $\pm 500\text{ }^\circ/\text{s}$, $\pm 1\,000\text{ }^\circ/\text{s}$, $\pm 2\,000\text{ }^\circ/\text{s}$.

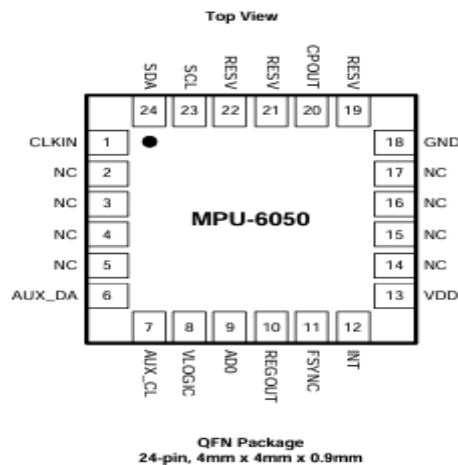
MPU6050 giao tiếp với vi điều khiển thông qua chuẩn I²C với tốc độ lên đến 400 kHz, dễ dàng kết nối với các hệ thống nhúng như ESP32 hoặc Arduino. Ngoài các cảm biến chính, MPU6050 còn tích hợp:

- Bộ xử lý chuyển động kỹ thuật số (DMP) giúp xử lý dữ liệu cảm biến, tính toán góc nghiêng, phát hiện chuyển động hoặc kết hợp dữ liệu từ cảm biến khác (sensor fusion).
- Cảm biến nhiệt độ nội bộ để hiệu chỉnh dữ liệu đo.

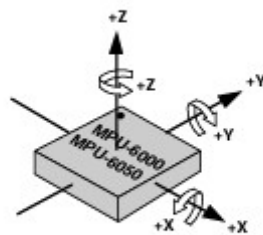
Với kích thước nhỏ gọn ($4 \times 4 \times 0,9\text{ mm}$), mức tiêu thụ điện năng thấp và khả năng cung cấp dữ liệu chuyển động chính xác theo thời gian thực, MPU6050 được sử dụng rộng rãi trong các ứng dụng như:

- Thiết bị đeo theo dõi sức khỏe.
- Hệ thống phát hiện té ngã.
- Robot, máy bay không người lái, hoặc điều khiển cử chỉ.

2.4.2 Sơ đồ chân



Hình 2.7 Sơ đồ chân của cảm biến MPU-6050



Hình 2.8 Hình ảnh các chiều của cảm biến MPU-6050

MPU-6050 sử dụng gói QFN 24 chân, trong đó bao gồm các chân chức năng chính liên quan đến nguồn, giao tiếp, ngắt, và điều khiển. Dưới đây là mô tả chi tiết chức năng từng chân:

Chân 1 (CLKIN): Clock ngoài tùy chọn. Nếu không sử dụng clock ngoài, chân này nên được nối xuống GND. Khi có tín hiệu clock ngoài, chip có thể đồng bộ với nguồn xung nhịp hệ thống.

Chân 6 (AUX_DA): Đường dữ liệu I²C dành cho cảm biến phụ kết nối ngoài với MPU-6050. Được sử dụng khi muốn mở rộng thêm cảm biến qua MPU.

Chân 7 (AUX_CL): Đường xung clock I²C dành cho cảm biến phụ bên ngoài. Là cặp tín hiệu với AUX_DA để hỗ trợ giao tiếp I²C phụ.

Chân 8 (VLOGIC): Nguồn cung cấp mức logic cho các chân giao tiếp. Cho phép MPU-6050 hoạt động linh hoạt với nhiều mức điện áp logic khác nhau. Chỉ có ở phiên bản MPU-6050.

Chân 9 (AD0): Dùng để chọn địa chỉ I²C của thiết bị. Nếu AD0 nối GND, địa chỉ I²C là 0x68; nếu nối VDD, địa chỉ là 0x69. Chức năng này cho phép sử dụng hai MPU-6050 trên cùng một bus I²C.

Chân 10 (REGOUT): Chân cấp ra điện áp đã được điều chỉnh nội bộ. Thường nối với tụ lọc 0.1 μ F để đảm bảo độ ổn định cho hệ thống cấp nguồn bên trong.

Chân 11 (FSYNC): Chân đồng bộ khung hình (frame sync). Dùng để đồng bộ tín hiệu với các cảm biến hoặc hệ thống ngoài. Nếu không sử dụng, nên nối GND.

Chân 12 (INT): Chân ngắt. Thường được sử dụng để báo cho vi điều khiển biết khi có dữ liệu mới hoặc sự kiện xảy ra (ví dụ: vượt ngưỡng, chuyển động phát hiện...).

Chân 13 (VDD): Nguồn cấp chính cho MPU-6050. Điện áp hoạt động phổ biến là 3.3V.

Chân 18 (GND): Chân nối đất, đảm bảo mạch hoạt động ổn định về điện áp và chống nhiễu.

Chân 20 (CPOUT): Chân đầu ra của mạch charge pump. Cần được nối với một tụ điện ngoài (thường 2.2 nF) để duy trì hoạt động ổn định của nguồn nội.

Chân 23 (SCL): Đường xung clock của giao tiếp I²C. Điều khiển tốc độ truyền dữ liệu giữa MPU và vi điều khiển.

Chân 24 (SDA): Đường dữ liệu của giao tiếp I²C. Truyền dữ liệu song công giữa MPU và vi điều khiển trung tâm.

Các chân 2-5, 14-17, 19, 21-22 (NC/RESV): Là các chân không kết nối (No Connect) hoặc dự phòng (Reserved). Không nên sử dụng hoặc kết nối các chân này trong thiết kế mạch ứng dụng.

2.4.3 Đặc điểm kỹ thuật điện

Nguồn cấp:

VDD (Nguồn chính): 2.375V – 3.46V

VLOGIC (MPU-6050): 1.71V – VDD

Xác định mức logic cho giao tiếp I²C (hoạt động ở cả 1.8V lẫn 3.3V)

Dòng tiêu thụ:

Chế độ hoạt động	Dòng tiêu thụ
Gyro + Accel + DMP	3.9 mA
Accel only	500 μ A

Chế độ hoạt động	Dòng tiêu thụ
Accel ở chế độ tiết kiệm (low power, 1.25Hz)	10 μ A
Toàn bộ chip idle	5 μ A

Sensor đặc trưng:

Gyroscope:

- Dải đo: ± 250 , ± 500 , ± 1000 , ± 2000 $^{\circ}/s$
- Độ phân giải: 16-bit ADC
- Sai số nhạy: $\pm 3\%$ tại $25^{\circ}C$
- ZRO (Zero Rate Output): ± 20 $^{\circ}/s$

Accelerometer:

- Dải đo: $\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$
- Độ phân giải: 16-bit ADC
- Nhạy: 16384 LSB/g ($\pm 2g$), 2048 LSB/g ($\pm 16g$)

Nhiệt độ:

- Tích hợp cảm biến nhiệt: $-40^{\circ}C \rightarrow +85^{\circ}C$
- Độ nhạy: ~ 340 LSB/ $^{\circ}C$
- Offset tại $35^{\circ}C$: khoảng -521 LSB

2.4.4 Nguyên lý hoạt động

a. Tổng quan về nguyên lý hoạt động

MPU-6050 là một cảm biến đo chuyển động 6 trục tích hợp:

- 3 trục Gia tốc kế (accelerometer) $\rightarrow$ đo gia tốc tuyến tính (linear acceleration)
- 3 trục Con quay hồi chuyển (gyroscope) $\rightarrow$ đo vận tốc góc (angular velocity)
- Bộ xử lý chuyển động số (DMP) $\rightarrow$ xử lý tín hiệu và hợp nhất dữ liệu (sensor fusion)
- ADC nội 16-bit, FIFO, Cảm biến nhiệt độ, và Giao tiếp I²C

b. Thành phần và hoạt động chi tiết

Con quay hồi chuyển 3 trục (Gyroscope)

- Chức năng: đo vận tốc góc quanh 3 trục X, Y, Z (đơn vị $^{\circ}/s$).
- Dựa vào hiện tượng Coriolis: khi xoay, phần tử MEMS bị lệch, tạo ra dao động được phát hiện bởi các cảm biến điện dung.
- Dao động này được khuếch đại, lọc và chuyển đổi thành tín hiệu điện áp tương ứng với vận tốc quay.

- ADC 16-bit nội bộ sẽ số hóa tín hiệu thành giá trị số để lưu trữ vào thanh ghi hoặc FIFO.
- Dải đo có thể cấu hình: ± 250 , ± 500 , ± 1000 , ± 2000 °/s
Tốc độ lấy mẫu: lên đến 8kHz

Gia tốc kế 3 trục (Accelerometer)

- Chức năng: đo gia tốc tuyến tính (g) theo 3 trục X, Y, Z.
- Dựa vào sự lệch vị trí của "proof mass" (khối đo trong MEMS) khi có gia tốc → tạo ra chênh lệch điện dung.
- Tín hiệu điện dung được chuyển đổi, lọc và số hóa qua ADC 16-bit.
- Dải đo: $\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$
Độ phân giải: lên đến 16,384 LSB/g tại $\pm 2g$

Bộ xử lý chuyển động số DMP (Digital Motion Processor)

- DMP là một bộ xử lý tín hiệu số tích hợp, hoạt động độc lập với MCU.
- Nó thu thập dữ liệu từ gyro và accel → thực hiện xử lý cảm biến hợp nhất (sensor fusion) xuất kết quả như:
 - Quaternions
 - Euler angles (yaw, pitch, roll)
 - Pedometer, gesture detection, shake/tap detection
- Ưu điểm:
 - Giảm tải tính toán cho vi điều khiển
 - Tiết kiệm điện năng (MCU có thể vào chế độ sleep)
 - Cho phép ứng dụng cập nhật ở tần số thấp (ví dụ 5Hz), trong khi DMP xử lý dữ liệu tại 200Hz hoặc cao hơn

Bộ đệm FIFO

- Dung lượng: 1024 Byte
- Cho phép ghi các dữ liệu gyro, accel, nhiệt độ, cảm biến phụ → giúp MCU đọc dữ liệu theo lô (burst read).
- Khi FIFO đầy hoặc có dữ liệu mới, INT sẽ báo về MCU qua chân INT (pin 12).

Giao tiếp I²C chính

- Dùng để cấu hình thanh ghi và đọc dữ liệu.
- Hoạt động ở tốc độ 400kHz (Fast Mode)
- Slave address tùy theo chân AD0 (pin 9):
 - AD0 = 0 → địa chỉ 0x68
 - AD0 = 1 → địa chỉ 0x69

Giao tiếp I²C phụ (AUX_I²C)

- Gồm 2 chân: AUX_DA (pin 6) và AUX_CL (pin 7)
- Dùng để giao tiếp với cảm biến ngoài như từ kế (magnetometer), tạo thành hệ thống 9 DOF (Degree of Freedom)
- Có 2 chế độ:
 - Master Mode: MPU đọc dữ liệu cảm biến phụ, xử lý qua DMP.
 - Pass-Through Mode: MCU điều khiển cảm biến phụ trực tiếp (tạm thời nối đường I²C chính và phụ)

Bộ lọc số (Digital Low-Pass Filter - DLPF)

- Có thể cấu hình để lọc nhiễu tín hiệu gyro/accel trước khi xuất.
- Tùy theo ứng dụng, bạn có thể chọn:
- Tốc độ đáp ứng nhanh (ít lọc, phù hợp cử động nhanh)
- Hoặc lọc mạnh (giảm nhiễu trong đo tĩnh)
- Cảm biến nhiệt độ
- Cho biết nhiệt độ của chip (vì ảnh hưởng đến độ nhạy)
- Kết quả đọc được thông qua thanh ghi cảm biến

c. Chu trình hoạt động tổng quát

Sau khi cấp nguồn, MPU-6050 khởi động và dùng đồng hồ nội.

Dữ liệu từ gyro và accel được lấy mẫu liên tục → qua ADC 16-bit.

Dữ liệu có thể:

- Ghi vào FIFO
- Xuất qua I²C
- Xử lý bởi DMP (nếu dùng)

Ngắt có thể được kích hoạt khi:

- Có dữ liệu mới
- FIFO đầy
- Lỗi I²C phụ

MCU đọc dữ liệu theo chu kỳ (hoặc theo ngắt) → dùng cho ứng dụng như: điều khiển chuyển động, phát hiện cử chỉ, cân bằng robot, đo hướng,...

2.5 Machine Learning

2.5.1 Giới thiệu

Trong kỷ nguyên công nghệ số hiện nay, lượng dữ liệu được tạo ra và lưu trữ tăng lên theo cấp số nhân. Để khai thác và xử lý khối lượng dữ liệu khổng lồ này, các phương pháp truyền thống dựa trên lập trình tuyến tính (rule-based programming) trở nên kém hiệu quả. Thay vào đó, những công nghệ mới như Machine Learning (Học máy) và Deep Learning (Học sâu) đã ra đời, trở thành nền tảng quan trọng của trí tuệ nhân tạo (AI) hiện đại.

a. Machine Learning (Học máy)

Machine Learning là một nhánh của trí tuệ nhân tạo (AI) tập trung vào việc phát triển các thuật toán và mô hình cho phép máy tính tự học từ dữ liệu mà không cần lập trình rõ ràng từng bước. Thay vì viết các quy tắc cụ thể để xử lý mọi tình huống, lập trình viên cung cấp dữ liệu và cho phép hệ thống tìm ra các quy luật hoặc mô hình tiềm ẩn, từ đó đưa ra dự đoán hoặc quyết định.

Các mô hình học máy thường được phân loại thành ba nhóm chính:

- Học có giám sát (Supervised Learning): Hệ thống được huấn luyện với dữ liệu đã gán nhãn (label). Thuật toán học cách ánh xạ từ đầu vào (input) sang đầu ra (output). Ví dụ: dự đoán nhịp tim bất thường dựa trên dữ liệu cảm biến sức khỏe.
- Học không giám sát (Unsupervised Learning): Hệ thống xử lý dữ liệu chưa gán nhãn và tự tìm các cấu trúc tiềm ẩn, chẳng hạn như phân cụm dữ liệu (clustering) hoặc giảm chiều (dimensionality reduction).
- Học tăng cường (Reinforcement Learning): Hệ thống học thông qua thử-sai, nhận phản hồi dưới dạng phần thưởng (reward) hoặc hình phạt (penalty) để tối ưu hóa hành động theo thời gian.

Ứng dụng của Machine Learning rất đa dạng, bao gồm nhận diện khuôn mặt, dự báo xu hướng thị trường, phân tích hành vi người dùng, và trong các hệ thống IoT, nó giúp xử lý và phân tích dữ liệu cảm biến hiệu quả.

b. Deep Learning (Học sâu)

Deep Learning là một nhánh đặc biệt của Machine Learning, lấy cảm hứng từ mạng nơ-ron nhân tạo (Artificial Nơ-ron Network – ANN) mô phỏng hoạt động của não người. Khác với các phương pháp học máy truyền thống thường yêu cầu trích xuất đặc trưng (feature extraction) thủ công, Deep Learning có khả năng tự động học các đặc trưng phức tạp từ dữ liệu thô thông qua nhiều tầng (layers) của mạng nơ-ron.

Mạng học sâu thường có cấu trúc gồm:

- Lớp đầu vào (Input Layer): Tiếp nhận dữ liệu (ví dụ: hình ảnh, tín hiệu cảm biến).
- Các lớp ẩn (Hidden Layers): Gồm nhiều tầng xử lý phi tuyến (non-linear transformations) giúp trích xuất các đặc trưng ở nhiều cấp độ.
- Lớp đầu ra (Output Layer): Tạo kết quả cuối cùng như phân loại (classification) hoặc dự đoán (regression).

Deep Learning đặc biệt hiệu quả trong các bài toán phức tạp, nơi dữ liệu có độ lớn và độ phức tạp cao, chẳng hạn như:

- Nhận dạng hình ảnh và giọng nói.
- Phân tích chuỗi thời gian (ví dụ: dữ liệu y tế và tín hiệu sinh học).
- Hệ thống gợi ý và dự đoán trong IoT và Big Data.

c. Mối quan hệ giữa Machine Learning và Deep Learning

Machine Learning và Deep Learning có mối quan hệ mật thiết. Deep Learning thực chất là một nhánh nâng cao của Machine Learning, nổi bật nhờ khả năng học từ dữ liệu lớn (big data) và tối ưu hóa với sức mạnh tính toán (GPU, TPU). Nếu như Machine Learning truyền thống phù hợp với bài toán có dữ liệu vừa và nhỏ, hoặc yêu cầu giải thích rõ ràng, thì Deep Learning lại vượt trội khi xử lý các bài toán phức tạp, dữ liệu lớn và đa dạng.

d Tầm quan trọng trong nghiên cứu và ứng dụng

Machine Learning và Deep Learning hiện nay đóng vai trò trụ cột trong nhiều lĩnh vực, từ y tế (chẩn đoán bệnh, theo dõi sức khỏe) đến công nghiệp (tự động hóa, dự đoán hỏng hóc thiết bị), thương mại điện tử (hệ thống gợi ý), và đặc biệt là các hệ thống IoT thông minh. Trong các hệ thống giám sát sức khỏe, ví dụ như phát hiện té ngã dựa trên cảm biến gia tốc và nhịp tim, Machine Learning và Deep Learning giúp tự động phát hiện mẫu dữ liệu bất thường và đưa ra cảnh báo nhanh chóng, góp phần nâng cao hiệu quả chăm sóc sức khỏe.

2.5.2 Mạng nơ-ron tích chập (CNNs)

a. Tổng quan về mạng nơ-ron tích chập

Mạng nơ-ron tích chập (Convolutional Nơ-ron Network – CNN) là một trong những kiến trúc deep learning tiêu biểu, được thiết kế đặc biệt để xử lý những dữ liệu có cấu trúc lưới như hình ảnh và video. Nguyên lý cơ bản của CNN lấy cảm hứng từ vỏ não thị giác của sinh vật, nơi các nơ-ron chỉ phản ứng với những vùng cục bộ trong trường nhìn. Nhờ cơ chế chia sẻ tham số (shared weights) của các bộ lọc (filter/kernel) và tính chất bất biến theo dịch chuyển (shift-invariance), CNN vừa giảm thiểu đáng kể số lượng tham số cần huấn luyện, vừa duy trì khả năng nhận diện vật thể dù chúng xuất hiện ở nhiều vị trí khác nhau trong ảnh.

Về cơ cấu, một CNN điển hình gồm nhiều tầng (layers) được xếp chồng lên nhau. Đầu tiên là các lớp tích chập (convolutional layers), nơi mỗi bộ lọc nhỏ—thường có kích thước 3×3 hoặc 5×5 —lần lượt “trượt” (stride) trên ảnh đầu vào, tính toán tích vô hướng và xuất ra các bản đồ đặc trưng (feature maps). Để kiểm soát độ giảm kích thước của feature maps, ta thường thêm padding—hay là viền giá trị bằng 0—vào biên ảnh. Tiếp theo, lớp pooling (thường là max-pooling) được dùng để giảm kích thước không gian, loại bỏ nhiễu và giới hạn tình trạng over-fitting. Sau mỗi phép tích chập và pooling thường xen kẽ một hàm kích hoạt phi tuyến (activation function) như ReLU, giúp mạng học được những biểu diễn phức tạp hơn so với các hàm tuyến tính đơn giản.

Khi luồng dữ liệu tiến sâu hơn qua các tầng, những lớp tích chập đầu tiên chủ yếu học các đặc trưng cơ bản như cạnh, góc cạnh và vùng sáng tối. Các lớp giữa kết hợp những đặc trưng này thành các vùng mô tả cấu trúc phức tạp hơn—chẳng hạn mắt, bánh xe hay cánh cửa. Ở cuối mạng, sau khi đã “làm phẳng” (flatten) các feature maps, các lớp liên kết đầy đủ (fully connected layers) thực hiện quá trình tích hợp toàn bộ thông tin và đưa ra kết quả phân loại cuối cùng bằng hàm Softmax.

Từ LeNet-5 do Yann LeCun đề xuất năm 1998 cho tới AlexNet, VGG, ResNet và gần đây là EfficientNet, CNN không ngừng được cải tiến cả về mặt kiến trúc lẫn phương pháp huấn luyện. LeNet-5 thành công trên bộ dữ liệu chữ viết tay MNIST, trong khi AlexNet thắng lớn tại cuộc thi ImageNet năm 2012 nhờ áp dụng ReLU, dropout và huấn luyện trên GPU để xử lý hàng chục triệu tham số. ResNet mở rộng chiều sâu mạng với “residual connection” giúp giảm vấn đề gradient biến mất, còn EfficientNet đưa ra chiến lược “compound scaling” đồng thời mở rộng độ sâu, độ rộng và độ phân giải ảnh đầu vào để đạt hiệu quả tính toán và độ chính xác cao hơn.

Ứng dụng của CNN rất đa dạng: từ phân loại ảnh, phát hiện và xác định vị trí vật thể, phân đoạn ảnh cho đến nhận dạng khuôn mặt và y học hình ảnh (X-ray, MRI). Ngoài ra, CNN còn được điều chỉnh để xử lý dữ liệu chuỗi như âm thanh, tín hiệu y tế hay thậm chí là xử lý ngôn ngữ tự nhiên. Ưu điểm lớn nhất của CNN là khả năng tự động trích xuất đặc trưng mà không cần thiết kế thủ công, cùng với hiệu quả tính toán nhờ chia sẻ tham số và sử dụng bộ lọc nhỏ. Tuy nhiên, mạng cũng đòi hỏi lượng lớn dữ liệu và tài nguyên GPU để huấn luyện, đồng thời có thể gặp khó khăn khi phải nhận diện vật thể trong các bối cảnh phức tạp, nơi thông tin ngữ cảnh rộng hơn đóng vai trò then chốt.

b. Mạng nơ-ron tích chập 1 chiều (1D CNN)

Định nghĩa CNN 1D: Mạng tích chập 1 chiều (1D CNN) là một kiến trúc mạng nơ-ron sâu chuyên xử lý dữ liệu tuần tự như chuỗi

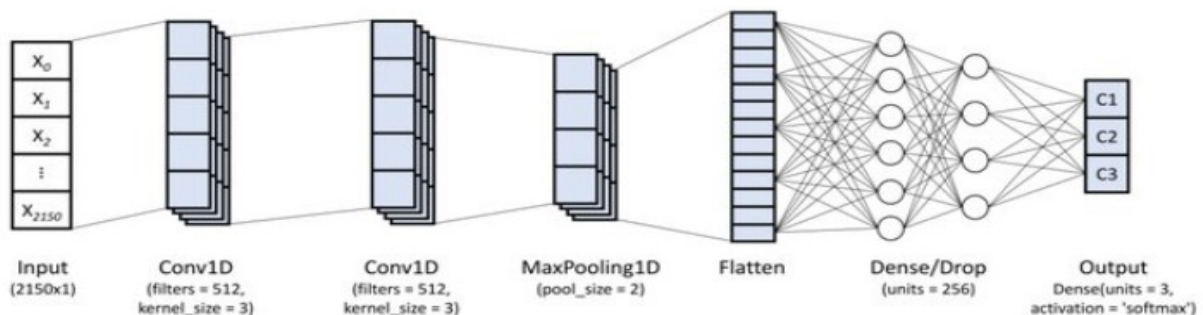
thời gian, tín hiệu hoặc văn bản. Thay vì sử dụng bộ lọc hai chiều như CNN xử lý ảnh, CNN 1D áp dụng các bộ lọc 1D trượt dọc theo một chiều duy nhất của dữ liệu (thường là chiều thời gian) để trích xuất đặc trưng cục bộ. Dữ liệu đầu vào của CNN 1D thường có dạng 3 chiều (batch_size, độ_dài_chuỗi, số_kênh) (ví dụ một chuỗi có nhiều bước thời gian và số lượng biến đi kèm)

Khác biệt với CNN 2D: CNN 2D được thiết kế cho dữ liệu ảnh 2 chiều (chiều rộng $\times$ chiều cao) nên bộ lọc cũng có 2 chiều. Trong khi đó, CNN 1D đơn giản là phiên bản “rút gọn” dùng bộ lọc 1 chiều trượt qua chuỗi 1D. Ví dụ, khi dùng CNN cho dữ liệu chuỗi, ta dùng Conv1D (bộ lọc $1 \times k$) thay vì Conv2D (bộ lọc $k \times k$). Đầu ra của Conv1D là một ma trận 2 chiều (tập hợp các bản đồ đặc trưng theo thời gian và số bộ lọc) thay vì ma trận 3 chiều như Conv2D. Nhờ vậy, CNN 1D phù hợp để phát hiện các mẫu ngắn hạn trong tín hiệu thời gian (đỉnh, dốc, v.v.) mà không cần quan tâm đến cấu trúc không gian như ảnh.

Mạng CNN 1D thường được dùng cho các dạng dữ liệu:

- Tín hiệu y sinh (ECG, EEG)
- Dữ liệu rung động cơ học (vibration)
- Tín hiệu âm thanh, lời nói
- Chuỗi số liệu thời gian khác (sensor readings) .

Cấu trúc tổng quát



Hình 2.9 Cấu trúc tổng quát của CNN 1D

CNN 1D thường bao gồm ba lớp cơ bản tương tự như CNN thông thường: Lớp tích chập 1D (Conv1D), Lớp gộp 1D (Pooling1D), và Lớp kết nối đầy đủ (Fully-connected).

Thông thường, một CNN 1D đơn giản sẽ bao gồm một hoặc nhiều khối Conv1D – Pooling xen kẽ, sau đó là một lớp Flatten để chuyển đổi dữ liệu thành vector, và cuối cùng là một hoặc nhiều lớp Dense để thực hiện nhiệm vụ phân loại hoặc hồi quy.

Lớp Conv1D: Lớp này sử dụng các bộ lọc 1D (kernel) trượt qua chuỗi dữ liệu đầu vào. Tại mỗi vị trí, bộ lọc thực hiện phép tích vô hướng (dot-product) với một phần của chuỗi, từ đó tạo ra các bản đồ đặc trưng (feature maps). Ví dụ, một lớp Conv1D(filters=64, kernel_size=3, activation='relu') có 64 bộ lọc với chiều dài là 3. Các bộ lọc này sẽ quét qua chuỗi dữ liệu và học các đặc trưng như các mẫu ngắn hạn hoặc xu hướng tại mỗi vị trí. Về bản chất, Conv1D tương tự như Conv2D nhưng đã giảm chiều: nó chỉ di chuyển theo một chiều (thời gian), giúp trích xuất các tính chất tuần tự của dữ liệu.

Lớp Pooling1D: thường được đặt sau một hoặc hai lớp Conv1D để giảm kích thước dữ liệu và củng cố thông tin quan trọng. Lớp này sử dụng một bộ lọc trượt (không có tham số cần học) và thực hiện phép gộp (ví dụ: max pooling hoặc average pooling) trên từng vùng nhỏ của đặc trưng đầu vào. Chẳng hạn, MaxPooling1D(pool_size=2) sẽ chọn giá trị lớn nhất trong mỗi cửa sổ có chiều dài 2, giúp giảm độ phân giải của chuỗi đặc trưng đi một nửa. Việc sử dụng Pooling giúp giảm số lượng tham số, chống hiện tượng quá khớp (overfitting) và tập trung vào các đặc trưng nổi bật nhất.

Lớp Fully-connected (Dense)

Sau khi các bản đồ đặc trưng 1D được làm phẳng (flatten) thành một vector, các lớp Dense sẽ thực hiện phân loại hoặc hồi quy cuối cùng. Mỗi nút ở lớp này kết nối với mọi phần tử ở lớp trước đó và thường sử dụng các hàm kích hoạt như softmax để đưa ra xác suất phân lớp. Ví dụ, sau các lớp Conv1D và Pooling, ta có thể thêm Dense(50, activation='relu') và Dense(num_classes, activation='softmax') để dự đoán nhãn cuối cùng.

Mạng CNN 1D có các siêu tham số điều khiển hình thái và độ hoạt động của mỗi lớp. Các tham số then chốt bao gồm:

- **Kích thước bộ lọc (Kernel size):** Đây là độ dài của bộ lọc 1D trên chuỗi dữ liệu đầu vào. Tham số này xác định khu vực mà mỗi bộ lọc sẽ "nhìn thấy" và xử lý. Ví dụ, một kernel_size=3 sẽ xem xét 3 timesteps (bước thời gian) liên tiếp. Kích thước kernel lớn cho phép mô hình nhận diện các mẫu dài hơn nhưng đồng thời làm tăng số lượng tham số cần học, trong khi kích thước quá nhỏ có thể bỏ sót các thông tin dài hạn quan trọng.
- **Số bước nhảy (Stride):** là khoảng di chuyển của bộ lọc giữa mỗi lần quét qua dữ liệu đầu vào. Nếu stride = 1, bộ lọc sẽ dịch chuyển từng vị trí một. Tuy nhiên, nếu stride > 1, bộ lọc sẽ bỏ qua một số vị trí, dẫn đến việc giảm độ dài của đầu ra (output) và kiểm soát mức độ trích xuất đặc trưng. Chẳng hạn, stride=2 giúp giảm

kích thước kết quả còn một nửa ở mỗi lớp tích chập (conv) hoặc gộp (pooling). Stride lớn hơn sẽ tạo ra đầu ra có kích thước nhỏ hơn.

- Số lượng bộ lọc (Filters): Đây là số lượng bộ lọc 1D trong một lớp Conv1D, đồng thời cũng là chiều sâu của đầu ra từ lớp đó. Mỗi bộ lọc được thiết kế để học một đặc trưng khác nhau từ dữ liệu. Ví dụ, filters=64 sẽ tạo ra 64 bản đồ đặc trưng song song từ lớp Conv1D đó. Việc tăng số lượng bộ lọc giúp mô hình có thể học được nhiều đặc trưng phong phú hơn, nhưng cũng làm tăng chi phí tính toán đáng kể.
- Padding (Đệm biên): Padding xác định cách xử lý các phần biên của tín hiệu khi thực hiện phép lọc. Có các tùy chọn padding phổ biến như:
 - "valid": Không thêm đệm, kết quả đầu ra sẽ ngắn hơn.
 - "same": Thêm đệm (thường là số 0) sao cho đầu ra có cùng độ dài với đầu vào (nếu stride=1).
 - "causal": Đối với dữ liệu chuỗi thời gian, loại đệm này bảo toàn tính tuần tự, đảm bảo rằng đầu ra tại một thời điểm chỉ phụ thuộc vào các đầu vào tại hoặc trước thời điểm đó. Padding giúp kiểm soát kích thước của đầu ra và tránh mất thông tin quan trọng ở các mép của chuỗi dữ liệu.

Như vậy, CNN 1D được thiết kế chuyên biệt cho dữ liệu chuỗi hoặc ngữ cảnh thời gian, với các lớp Conv1D và Pooling1D thay vì Conv2D và Pooling2D như trong CNN thông thường. Việc lựa chọn các tham số phù hợp (như kích thước bộ lọc, số bước nhảy, số lượng bộ lọc và padding) là vô cùng quan trọng để mô hình có thể nắm bắt được các đặc trưng quan trọng mà không trở nên quá phức tạp.

c. Cơ chế huấn luyện

Lan truyền xuôi (Forward Propagation):

Quy trình lan truyền xuôi của 1D CNN trải qua các bước chính: lấy tín hiệu đầu vào x , tính tích chập, áp dụng hàm kích hoạt, thực hiện gộp (pooling) tuần tự qua các lớp, và cuối cùng tính toán đầu ra.

Tích chập: Giả sử tín hiệu vào là $h(l-1)$, mỗi bộ lọc $w(l)$ thực hiện tích chập 1D. Ví dụ, đầu ra trước kích hoạt tại vị trí i của bộ lọc j là :

$$z_{j,i}^{(l)} = \sum_{u=1}^F w_{j,u}^{(l)} h_{i+u-1}^{(l-1)} + b_j^{(l)}.$$

Trong đó, F là kích thước bộ lọc, $w_{j,u}^{(l)}$ là trọng số của bộ lọc thứ j tại vị trí u ở lớp l , $h_{i+u-1}^{(l-1)}$ là giá trị đầu vào từ lớp trước đó, và $b_j^{(l)}$ là bias của bộ lọc thứ j ở lớp l . Kết quả $z^{(l)}$ có kích thước L theo công thức đã nêu.

Kích hoạt: Ta áp dụng hàm kích hoạt ϕ (như ReLU, sigmoid,...) lên từng phần tử của $z^{(l)}$:

$$h_{j,i}^{(l)} = \phi(z_{j,i}^{(l)}).$$

Ví dụ, với ReLU: $\phi(z) = \max(0, z)$. Việc này giúp mạng học được các tính năng phi-tuyến.

Pooling: Sau khi tích chập và kích hoạt, ta có một chuỗi $h_j^{(l)}$ cho mỗi bộ lọc j . Tầng pooling lấy chuỗi này chia thành các khối (ví dụ kích thước k với stride S) và chọn giá trị đại diện.

Với max-pooling:

$$p_{j,r}^{(l)} = \max(h_{j,(r-1)S+1}^{(l)}, \dots, h_{j,(r-1)S+k}^{(l)}).$$

Với average-pooling:

$$p_{j,r}^{(l)} = \frac{1}{k} \sum h_{j,(r-1)S+u}^{(l)}.$$

Pooling giúp giảm độ dài chuỗi và làm mịn đặc trưng. Kết quả mỗi chuỗi sau pooling có độ dài nhỏ hơn, tạo thành một ma trận (các cột là kết quả của mỗi bộ lọc)

Flatten và Fully-Connected (FC): Cuối cùng, ma trận đầu ra của tầng pooling cuối cùng được làm phẳng thành một vectơ f (bằng cách ghép các cột hoặc hàng lại). Vectơ này được đưa vào mạng FC. Mỗi phần tử đầu ra cuối cùng y_i của mạng là:

$$y_i = g\left(\sum_j W_{ij} f_j + b_i\right),$$

trong đó W là ma trận trọng số và g là hàm kích hoạt tầng cuối (sigmoid hoặc softmax cho phân loại, hoặc tuyến tính cho hồi quy).

Lan truyền ngược (Backpropagation) và cập nhật trọng số:

Quy trình lan truyền ngược thực hiện tính đạo hàm của hàm mất mát L theo các tham số, từ lớp đầu ra lan ngược về đầu vào, và cập nhật trọng số để giảm lỗi. Các bước chính như sau:

Tính đạo hàm hàm mất mát ở đầu ra: Giả sử mạng thực hiện tác vụ hồi quy, sử dụng hàm mất mát MSE:

$$L = \frac{1}{2} \sum_i (y_i - t_i)^2,$$

trong đó t_i là giá trị mục tiêu. Từ đó, đạo hàm theo đầu ra mạng y_i là $\partial L / \partial y_i = (y_i - t_i)$.

Lan ngược qua tầng Flatten: Gradient $\delta(f)$ được định hình lại (reshape) thành ma trận như đầu ra của tầng pooling trước đó.

Lan truyền qua lớp Pooling:

Với average-pooling, lỗi được chia đều cho các phần tử trong khối. Ví dụ nếu khối có k phần tử, mỗi phần tử nhận $\delta(\text{next})/k$.

Với max-pooling, lỗi được truyền đầy đủ cho phần tử lớn nhất đã được chọn ở bước forward và bằng 0 với các phần tử khác (cần lưu lại chỉ số “chiến thắng” trong bước forward).

Lan truyền qua lớp Tích chập: Sau khi có lỗi δ tại đầu ra của mỗi bộ lọc tích chập (sau pooling), ta cập nhật trọng số bộ lọc. Mỗi trọng số $w_{u,l}$ trong bộ lọc thu được gradient bằng tích (correlation) giữa lỗi của đầu ra và giá trị đầu vào tương ứng

Cập nhật trọng số: Sau khi tính các gradient $\partial L / \partial W$ và $\partial L / \partial b$ cho tất cả các tầng, ta cập nhật tham số bằng phương pháp tối ưu (ví dụ Gradient Descent hoặc Adam):

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b},$$

với η là tốc độ học (learning rate).

Quá trình này lặp lại cho từng batch/lần huấn luyện. Tổng kết, quá trình lan truyền ngược trong 1D CNN về cơ bản tuân theo quy tắc đạo hàm theo chuỗi (chain rule) giống như CNN 2D, nhưng

cần lưu ý đến dimension 1D của tensor. Phương pháp này giúp điều chỉnh trọng số bộ lọc và bias sao cho lỗi giảm dần qua các lần cập nhật (hàm mất mát giảm).

Khác biệt giữa 1D CNN và 2D CNN:

Về mặt toán học, nguyên lý lan truyền xuôi/ngược giữa 1D và 2D CNN tương tự (đều dựa trên tích chập, pooling, hàm kích hoạt và lan truyền chuỗi đạo hàm), nhưng khác nhau ở thứ tự bậc tensor và ứng dụng. Điểm khác biệt chính gồm:

Kích thước lõi tích chập: 1D CNN sử dụng bộ lọc 1 chiều (ví dụ kích thước $k \times 1$) trượt dọc theo một trục của dữ liệu (thời gian hoặc không gian 1 chiều). Trong khi đó, 2D CNN dùng bộ lọc 2D ($k \times k$) trượt trên ma trận đầu vào (hình ảnh). Điều này làm cho công thức toán giống nhau về bản chất nhưng khác số chiều tổng hợp.

Độ phức tạp tính toán: 1D convolution có độ phức tạp ít hơn 2D convolution. Ví dụ, tích chập 1D cho tín hiệu độ dài N với bộ lọc kích thước K mất $O(NK)$ phép tính nhân-cộng cho mỗi bộ lọc. Trong khi đó, tích chập 2D với ảnh kích thước $N \times N$ và filter $K \times K$ mất $O(N^2K^2)$. Như vậy, dưới cùng cấu hình bộ lọc và kích thước dữ liệu, 1D CNN có độ phức tạp thấp hơn đáng kể, cho phép huấn luyện và suy luận nhanh hơn.

Không gian tham số: Tương tự, số lượng tham số của mạng 1D thường thấp hơn vì mỗi bộ lọc 1D ít trọng số hơn (ít chiều) so với bộ lọc 2D cùng kích thước. Điều này làm 1D CNN phù hợp với dữ liệu thứ tự (chuỗi, tín hiệu) có ít mẫu hoặc cần triển khai nhẹ (ví dụ nhúng, thời gian thực).

Ứng dụng dữ liệu: 1D CNN thường dùng cho dữ liệu theo chuỗi thời gian hoặc chuỗi ký tự (ví dụ phân loại tín hiệu EEG/ECG, audio, văn bản). 2D CNN chủ yếu cho ảnh/video. Quy trình lan truyền của cả hai đều theo chuỗi lớp tương tự, chỉ khác ở chiều lọc và độ lớn dữ liệu vào.

Tóm lại, thuật toán lan truyền xuôi/ngược cơ bản không đổi giữa CNN 1D và CNN 2D, chỉ khác ở số chiều tính toán. Vì vậy, kiến thức lan truyền ngược của CNN 2D (sử dụng chain rule qua tích chập 2D) có thể áp dụng nguyên vẹn cho 1D với việc điều chỉnh chỉ số và phép tính tương ứng.

d. Hàm mất mát và hàm kích hoạt

Hàm mất mát (loss function)

Hàm mất mát là một đại lượng định lượng mức độ sai khác giữa đầu ra dự đoán của mạng nơ-ron và giá trị mục tiêu thực tế. Mục tiêu của quá trình huấn luyện là tối thiểu hóa giá trị hàm mất mát, giúp mô hình học được các tham số tối ưu. Việc lựa chọn hàm mất mát phụ thuộc vào loại bài toán, cụ thể như hồi quy

(regression) hay phân loại (classification). Dưới đây là các hàm mất mát phổ biến:

- Mean Squared Error (MSE) – Hàm mất mát cho bài toán hồi quy
 - Công dụng: MSE thường được sử dụng trong các bài toán hồi quy, nơi mô hình dự đoán giá trị liên tục (ví dụ: dự đoán nhiệt độ, giá nhà).
 - Công thức:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

y_i : giá trị thực tế (ground truth) – tức là nhãn hoặc giá trị thật trong tập dữ liệu.

$\hat{y}_i$: giá trị dự đoán (predicted value) – đầu ra do mô hình dự đoán.

- Đặc điểm:
 - MSE phạt mạnh các sai số lớn do sử dụng bình phương.
 - Giúp mô hình tập trung giảm thiểu các sai lệch lớn, tuy nhiên dễ bị ảnh hưởng bởi các điểm ngoại lai (outlier).
- Binary Cross-Entropy (Log-Loss) – Hàm mất mát cho phân loại nhị phân
 - Công dụng: Dùng cho các bài toán phân loại 2 lớp (ví dụ: phát hiện té ngã – có té ngã hoặc không).
 - Công thức:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- Đặc điểm:
 - Đánh giá sự khác biệt giữa xác suất dự đoán và nhãn thực tế.
 - Thích hợp cho bài toán phân loại nhị phân.
- Categorical Cross-Entropy – Hàm mất mát cho phân loại đa lớp
 - Công dụng: Sử dụng khi bài toán có nhiều hơn hai lớp (ví dụ: phân loại nhiều trạng thái sức khỏe).

- Công thức:

$$L = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^C y_{ij} \log(\hat{y}_{ij})$$

- Đặc điểm:
 - Cho phép mô hình tối ưu xác suất trên nhiều lớp.
 - Được dùng phổ biến trong các mạng nơ-ron sâu cho bài toán phân loại.

Hàm kích hoạt (Activation Function)

Hàm kích hoạt được sử dụng trong các nơ-ron của mạng để đưa tính phi tuyến vào mô hình, cho phép mạng học được các mối quan hệ phức tạp trong dữ liệu.

- ReLU (Rectified Linear Unit):
 - Định nghĩa: $f(x) = \max(0, x)$, tức là giữ nguyên phần dương của đầu vào và đặt phần âm bằng 0.
 - Ưu điểm: Đơn giản, hiệu quả và giúp mạng tránh hiện tượng gradient biến mất (vanishing gradient) ở phía giá trị dương. Nó cho phép quá trình cập nhật diễn ra nhanh chóng với các nơ-ron được kích hoạt.
 - Nhược điểm: Khi đầu vào âm, đạo hàm bằng 0 có thể khiến nơ-ron "chết" (không còn học được) do không có gradient để cập nhật.
 - Ứng dụng phổ biến: Được ưa chuộng cho các tầng ẩn nhờ tính đơn giản và khả năng hội tụ nhanh.
- Sigmoid (Hàm Logistic):
 - Định nghĩa: $f(x) = 1/(1+e^{-x})$, đầu ra nằm trong khoảng (0,1) và có đồ thị dạng chữ S mềm mại.
 - Ưu điểm: Giá trị đầu ra có thể được hiểu như xác suất, nên thường dùng ở tầng đầu ra cho bài toán phân loại nhị phân.
 - Nhược điểm:
 - Dễ bị bão hòa ở hai đầu khi x rất lớn hoặc rất nhỏ, dẫn đến đạo hàm rất nhỏ (gây ra vấn đề gradient biến mất) và làm chậm quá trình huấn luyện.
 - Đầu ra không đối xứng quanh 0, có thể ảnh hưởng đến hiệu quả tối ưu hóa.

- Tanh (Hàm Hyperbolic Tangent):

- Định nghĩa:

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- cho kết quả trong khoảng $(-1, 1)$.
- Ưu điểm: Có tính chất zero-centered (đầu ra đối xứng quanh 0) nên thường hội tụ tốt hơn Sigmoid khi dùng làm tầng ẩn.
- Nhược điểm: Vẫn gặp vấn đề bão hòa và gradient biến mất tương tự Sigmoid khi x lớn. Tanh từng được dùng rộng rãi trước khi ReLU trở nên phổ biến.

- Softmax:

- Định nghĩa: Với vector đầu vào $(z_1, \dots, z_C)$, Softmax tính

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

- Ứng dụng: Được sử dụng ở tầng cuối cùng cho các bài toán phân loại đa lớp.
- Đặc điểm: Biến đổi các giá trị đầu ra thành một vector xác suất có tổng bằng 1, trong đó mỗi phần tử $\sigma(z)_i$ là xác suất của lớp i. Điều này giúp dễ dàng kết hợp với hàm mất mát Entropy chéo cho đa lớp.

2.5.3 TensorFlow Lite

TensorFlow Lite (TFLite) là một thư viện được thiết kế đặc biệt để chạy suy luận Học máy (Machine Learning - ML) trực tiếp trên các thiết bị, tập trung vào kích thước nhỏ gọn và hiệu suất cao.

a. Cấu trúc và thành phần kỹ thuật của TensorFlow Lite

Quy trình điển hình của TFLite bao gồm việc chuyển đổi một mô hình TensorFlow hoặc Keras gốc sang định dạng tệp TFLite được tối ưu hóa, sau đó tải và chạy mô hình đó trong môi trường thời gian chạy (runtime) tích hợp trên thiết bị. Các thành phần chính của TFLite bao gồm:

TensorFlow Lite Converter: Đây là một chương trình hoạt động ngoại tuyến, có chức năng chuyển đổi các mô hình TensorFlow (ví dụ: SavedModel, Keras H5) thành tệp định dạng .tflite. Định dạng .tflite là một dạng FlatBuffer được tối ưu hóa cho suy luận trên thiết bị.

TensorFlow Lite Model File: Là tệp .tflite thực tế. Tệp này chứa đồ thị tính toán của mô hình cùng với các trọng số đã được tối ưu hóa để triển khai hiệu quả.

Interpreter (API C++): Là bộ giải thích hay còn gọi là môi trường thời gian chạy nhỏ gọn trên thiết bị. Interpreter có nhiệm vụ nạp tệp mô hình và thực thi các toán tử (operator) cần thiết. Ví dụ, Interpreter có thể chỉ chiếm khoảng 300KB mã, nhỏ hơn đáng kể so với các giải pháp khác như TensorFlow Mobile.

Trên thiết bị, Interpreter chạy từng toán tử trong đồ thị mô hình. Một tính năng nổi bật của TFLite là khả năng chỉ nạp những toán tử cần thiết cho mô hình đang chạy, giúp giảm dung lượng bộ nhớ.

TensorFlow Lite cũng hỗ trợ Delegate – các plugin cho phép tăng tốc phần cứng. Chẳng hạn, trên nền tảng Android, Interpreter có thể sử dụng Android NNAPI (Nơ-ron Networks API) hoặc GPU delegate để chuyển một số toán tử sang xử lý trên các chip chuyên dụng. Tương tự, trên iOS, có thể sử dụng Core ML delegate. Ngoài ra, các nhà phát triển còn có thể viết các kernel tùy chỉnh bằng C++ để bổ sung các toán tử riêng cho Interpreter nếu cần.

TensorFlow Lite còn có một phiên bản đặc biệt là TFLite Micro dành cho vi điều khiển (MCU). TFLite Micro là một môi trường thời gian chạy tối giản, được viết bằng C/C++, dành riêng cho các thiết bị có tài nguyên rất hạn chế. Nó chỉ hỗ trợ một tập hợp con các toán tử thiết yếu, cho phép chạy các mô hình rất nhỏ (thường chỉ vài KB) cho các tác vụ đơn giản như phân loại hình ảnh hoặc âm thanh.

Như vậy, kiến trúc của TFLite bao gồm một chuỗi chuyển đổi mô hình (từ TensorFlow sang Converter và cuối cùng là tệp .tflite) và một môi trường thời gian chạy trên thiết bị (gồm Interpreter C++, Thư viện Kernel và các Delegates), đảm bảo quá trình suy luận diễn ra gọn nhẹ và linh hoạt.

b. Quy trình chuyển đổi và triển khai mô hình

Quy trình triển khai một mô hình ML với TensorFlow Lite thường bao gồm các bước sau:

Chuyển đổi mô hình

- Đầu tiên, bạn cần có một mô hình đã được huấn luyện từ các framework như TensorFlow/Keras hoặc PyTorch.
- Đối với TensorFlow (SavedModel hoặc tf.keras): Bạn sử dụng `tf.lite.TFLiteConverter` để xuất ra tệp .tflite. Ví dụ, mã chuyển đổi có thể trông như thế này:
`converter=tf.lite.TFLiteConverter.from_saved_model(dir)` và `tflite_model = converter.convert()`. Ngoài ra, bạn cũng có thể dùng công cụ dòng lệnh `tflite_convert`.
- Đối với PyTorch: Bạn có thể chuyển đổi mô hình PyTorch sang định dạng ONNX trước, sau đó từ ONNX sang TFLite. Hoặc, với PyTorch 2.1 trở lên, bạn có thể sử dụng thư viện `ai-edge-torch` để chuyển đổi trực tiếp sang định dạng LiteRT (.tflite).

Tối ưu hóa

- Sau khi đã có tệp TFLite, bạn có thể áp dụng các kỹ thuật tối ưu hóa để giảm kích thước và cải thiện hiệu suất. Các kỹ thuật phổ biến bao gồm:
- Quantization (Lượng tử hóa): Kỹ thuật này giảm số bit của trọng số (và đôi khi cả đầu ra) của mô hình. Điều này có thể giúp kích thước tệp nhỏ hơn khoảng 4 lần và tăng tốc độ chạy từ 2-3 lần trên CPU. Có nhiều chế độ lượng tử hóa như:
 - Dynamic Range Quantization: Chỉ lượng tử hóa trọng số.
 - Full Integer Quantization (int8/uint8): Lượng tử hóa toàn bộ mô hình sang kiểu số nguyên 8-bit.
 - Float16 Quantization: Lượng tử hóa trọng số sang kiểu dấu phẩy động 16-bit. Việc này giúp giảm dung lượng và cải thiện độ trễ. Tuy nhiên, cần lưu ý rằng lượng tử hóa có thể làm giảm nhẹ độ chính xác của mô hình, đặc biệt với các mạng nhỏ hoặc phức tạp.
- Pruning (Cắt tỉa): Kỹ thuật này thường được thực hiện trong quá trình huấn luyện để loại bỏ các kết nối không quan trọng trong mạng, từ đó giảm kích thước mô hình. Sau khi cắt tỉa, có thể kết hợp thêm lượng tử hóa để tối ưu hóa hơn nữa.

Triển khai lên thiết bị

- Tệp .tflite đã được tối ưu hóa sẽ được tích hợp vào ứng dụng của bạn trên thiết bị mục tiêu.
- Trên Android: Bạn có thể thêm mô hình vào thư mục assets/ của ứng dụng và sử dụng thư viện TFLite với API Java/Kotlin để tải và chạy. Google khuyến nghị sử dụng Runtime qua Google Play Services để giảm kích thước APK, hoặc đóng gói thư viện chuẩn nếu bạn cần kiểm soát chặt chẽ hơn.
- Trên iOS: Bạn thêm dependency thông qua CocoaPods (ví dụ: pod 'TensorFlowLite') và sử dụng API Swift/ObjC của TFLite.
- Trên Linux/Embedded: Bạn có thể gọi trực tiếp API C++ của TFLite.
- Với thiết bị nhúng (MCU) dùng TensorFlow Lite Micro: Bạn cần chuyển tệp .tflite thành một mảng byte C/C++ (thường dùng công cụ như xxd), sau đó gọi API của TFLite Micro để nạp dữ liệu và thực hiện suy luận.

c. Ưu điểm và nhược điểm của TensorFlow Lite

Ưu điểm

- Hiệu quả trên thiết bị: TFLite được thiết kế đặc biệt cho suy luận trên thiết bị nhúng hoặc di động, tối ưu hóa cả về hiệu năng và kích thước. Thư viện lõi chỉ nặng vài trăm KB, và tệp

mô hình sau khi lượng tử hóa có thể nhỏ hơn khoảng 4 lần so với bản dấu phẩy động ban đầu.

- Chạy ngoại tuyến: TFLite cho phép mô hình chạy hoàn toàn trên thiết bị mà không cần kết nối internet. Điều này giúp bảo vệ quyền riêng tư của người dùng và tăng độ tin cậy do không phụ thuộc vào kết nối mạng.
- Tăng tốc phần cứng: Hỗ trợ tăng tốc phần cứng thông qua các delegates (ví dụ: GPU delegate, NNAPI trên Android, Core ML trên iOS), giúp tăng tốc độ suy luận đáng kể.
- Đa nền tảng và đa ngôn ngữ: Cung cấp API cho nhiều nền tảng (Android, iOS, Linux, vi điều khiển) và hỗ trợ các ngôn ngữ lập trình phổ biến như Java/Kotlin, Swift, C++ và Python.
- Quản lý bộ nhớ hiệu quả: Các tính năng quản lý bộ nhớ tĩnh (ví dụ: cấp phát tensor) giúp TFLite hoạt động mượt mà trên các thiết bị có tài nguyên hạn chế.

Nhược điểm

- Giảm độ chính xác sau tối ưu hóa: Việc tối ưu hóa mô hình, đặc biệt là lượng tử hóa, có thể làm giảm nhẹ độ chính xác. Trong một số trường hợp, mô hình được lượng tử hóa hậu huấn luyện có thể kém chính xác hơn đáng kể, nhất là với các mạng nhỏ hoặc quá phức tạp.
- Hạn chế về toán tử: TFLite không hỗ trợ tất cả các toán tử có trong TensorFlow gốc. Nếu mô hình chứa các toán tử không được hỗ trợ, nhà phát triển có thể phải sử dụng Flex delegate (cho phép gọi runtime TensorFlow đầy đủ) hoặc viết toán tử tùy chỉnh. Điều này làm tăng độ phức tạp và kích thước của môi trường thời gian chạy. Quá trình chuyển đổi có thể gặp lỗi nếu mô hình sử dụng các lớp, RNN hoặc phép toán không phổ biến.
- API không hoàn toàn đồng nhất: Các API giữa các nền tảng có thể không hoàn toàn đồng nhất (ví dụ: sự khác biệt giữa Kotlin và Swift, hoặc sự sẵn có của các delegate). Điều này đôi khi yêu cầu nhà phát triển phải viết mã riêng cho từng nền tảng.

d. Các ứng dụng thực tế của TensorFlow Lite

TensorFlow Lite đã được ứng dụng rộng rãi trong nhiều lĩnh vực trên các thiết bị di động và nhúng, đưa khả năng AI vào các sản phẩm một cách nhẹ nhàng và hiệu quả.

Thị giác máy tính:

- Nhận dạng ảnh và video thời gian thực: Ví dụ, một ứng dụng Android mẫu có thể sử dụng TFLite để phân loại âm thanh

(nhận dạng tiếng vỗ tay, tiếng nói) theo thời gian thực ngay trên điện thoại.

- OCR (Nhận diện ký tự quang học) di động: Google đã giới thiệu ví dụ về ứng dụng Android sử dụng TFLite để phát hiện vùng chứa văn bản (text detection) và nhận dạng ký tự trên ảnh, giúp trích xuất chữ từ logo sản phẩm. Quy trình này thường gồm hai bước: phát hiện vùng chữ và sau đó nhận dạng ký tự bằng mô hình TFLite trên thiết bị.

Xử lý ngôn ngữ tự nhiên (NLP):

- TFLite được dùng để triển khai các mô hình trả lời câu hỏi (question answering), phân loại văn bản, hoặc các tính năng Smart Reply trên thiết bị di động.
- Google cung cấp các ứng dụng tham khảo (Android/iOS) sử dụng TFLite với các mô hình BERT nhẹ (như MobileBERT, ALBERT) để trả lời câu hỏi hoặc gợi ý trả lời tự động. Điều này cho phép thực hiện chức năng dự đoán văn bản ngay trên điện thoại mà không cần gửi dữ liệu lên máy chủ.

Chăm sóc sức khỏe:

- TFLite được sử dụng để tạo ra các công cụ AI di động hỗ trợ chẩn đoán. Ví dụ, nhóm Nghiên cứu Google đã dùng TensorFlow Lite để xây dựng hệ thống siêu âm thai trên điện thoại. Hệ thống này giúp nhân viên y tế ở vùng sâu kiểm tra tuổi thai và tình trạng bào thai bằng cách xử lý video siêu âm ngay trên thiết bị.
- Ở quy mô nhỏ hơn, TFLite Micro đang được ứng dụng trong các thiết bị TinyML. Một ví dụ là mPOD DxTrack, một thiết bị đọc que thử y tế giá rẻ (như que thử thai hoặc xét nghiệm nhanh COVID-19). DxTrack sử dụng TFLite Micro để nhận diện và phân loại kết quả trên que thử chỉ trong vài giây, mang lại chẩn đoán nhanh chóng và loại bỏ sai lệch do đọc bằng mắt thường.

Hệ thống IoT và Edge:

TFLite cũng được dùng trong các hệ thống IoT, ví dụ trên Raspberry Pi hoặc các thiết bị Edge. Các ứng dụng bao gồm theo dõi camera an ninh, phát hiện đồ vật, hoặc xử lý giọng nói (ví dụ: các trợ lý ảo chạy ngoại tuyến) ngay tại chỗ.

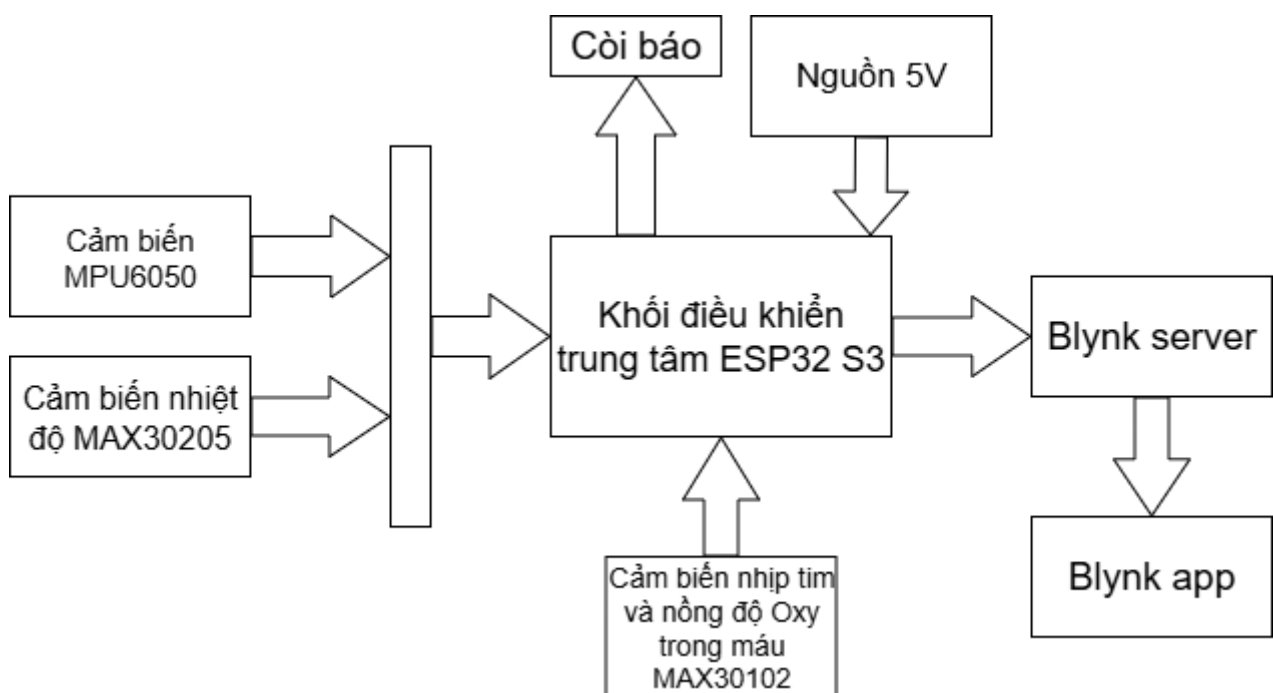
CHƯƠNG 3: TÌNH TOÁN VÀ THIẾT KẾ MÔ HÌNH

3.1 Giới thiệu mô hình

Mô hình của đề tài có những chức năng chính sau:

- Có khả năng thu thập dữ liệu về nhiệt độ cơ thể.
- Có khả năng thu thập dữ liệu về nhịp tim.
- Có khả năng thu thập dữ liệu về nồng độ Oxy trong máu.
- Có khả năng cảnh báo khi xác định có tình huống té ngã một cách chính xác thông qua các dữ liệu về gia tốc và góc quay được thu thập và được mô hình Deep Learning xử lý.
- Có thể giúp theo dõi các thông số về sức khỏe và nhận cảnh báo có té ngã từ xa qua ứng dụng Blynk trên điện thoại.

Từ các đặc tính yêu cầu của đề tài nhóm thực hiện đã thiết kế mô hình với các khối chính như sau:



Hình 3.1 Sơ đồ khối tổng quát của mô hình.

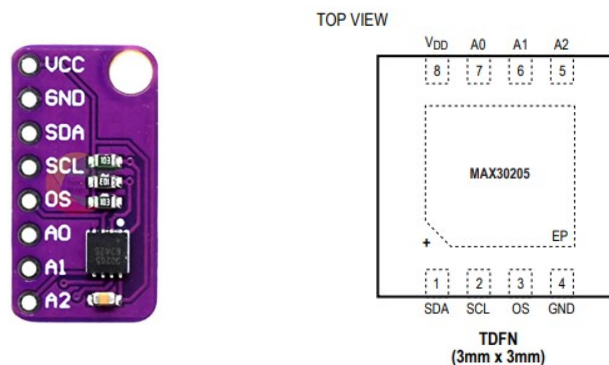
Các khối của mô hình bao gồm:

- Cảm biến MPU6050: Dùng để thu thập dữ liệu về gia tốc và góc quay khi có chuyển động bất thường để mô hình Deep Learning xử lý và đưa ra kết luận
- Cảm biến nhiệt độ MAX30205: Được đặt tiếp xúc với cơ thể để thu thập dữ liệu nhiệt độ một cách chính xác nhất.

- Cảm biến MAX30102: Dùng thu thập dữ liệu về nhịp tim và nồng độ Oxy trong máu. Cảm biến được gắn cố định ở ngón trỏ của người dùng và được nối với khối điều khiển trung tâm qua các dây dẫn.
- Còi báo: Dùng để báo động khi có tình huống té ngã xảy ra.
- Blynk server: Nhận các dữ liệu từ khối xử lý trung tâm và truyền đến ứng dụng Blynk trên điện thoại.
- Blynk app: Ứng dụng trên điện thoại. Giúp người thân của người sử dụng có thể theo dõi các thông số về sức khỏe và nhận được các cảnh báo khi phát hiện có bất thường.
- Ngoài ra, một mô hình Deep Learning sẽ được tích hợp để chạy trực tiếp trên khối điều khiển trung tâm. Mô hình này chỉ chạy khi khối xử lý trung tâm phát hiện những bất thường trong chuyển động nhờ các dữ liệu được gửi về từ cảm biến MPU6050.

3.2 Thiết kế tính toán từng khối

3.2.1 Cảm biến nhiệt độ MAX30205



Hình 3.2 Cảm biến MAX30205

Đặc điểm kỹ thuật của cảm biến MAX20205

- Là cảm biến đo nhiệt độ y sinh có độ chính xác cao, sử dụng giao tiếp I²C , giúp dễ dàng kết nối với vi điều khiển thông qua hai chân SDA và SCL.
- Độ phân giải đo nhiệt độ là 16 bit.
- Dải đo nhiệt độ từ 0°C đến +50°C, phù hợp cho các ứng dụng y tế như theo dõi thân nhiệt người.
- Có thể đạt độ chính xác 0,1°C trong khoảng nhiệt độ cơ thể từ 37°C đến 39°C, đáp ứng tốt yêu cầu trong các hệ thống theo dõi sức khỏe.
- Không cần hiệu chuẩn hoặc linh kiện bên ngoài để hoạt động, cảm biến tích hợp sẵn mạch ADC và mạch hiệu chỉnh nội bộ.
- Nguồn hoạt động linh hoạt từ 2,7V đến 3,3V, tiêu thụ dòng điện rất thấp, thích hợp cho các thiết bị đeo hoặc hệ thống dùng pin.

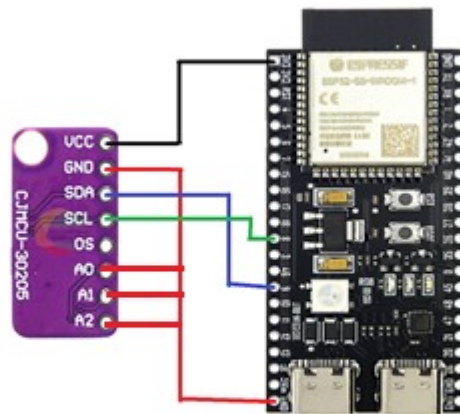
- Thời gian chuyển đổi nhiệt độ nhanh, điển hình trong khoảng 44-55ms cho mỗi lần đọc.

Cảm biến nhiệt độ MAX30205 cung cấp dữ liệu nhiệt độ dạng số với độ phân giải cao qua giao tiếp I²C tương thích chuẩn 2-wire. Cảm biến này sử dụng hai đường tín hiệu là SDA (data) và SCL (clock), cùng với nguồn cấp và mass để kết nối với vi điều khiển.

Địa chỉ I²C của cảm biến MAX30205 có thể được cấu hình bằng các chân A0, A1 và A2. Cụ thể, A0 và A1 có thể được nối đến nguồn, GND, SDA hoặc SCL, còn A2 có thể nối đến nguồn hoặc GND, cho phép lựa chọn lên đến 32 địa chỉ khác nhau. Tính năng này giúp có thể kết nối nhiều cảm biến MAX30205 trên cùng một bus I²C mà không bị xung đột địa chỉ.

Mỗi thiết bị MAX30205 hoạt động độc lập với địa chỉ I²C riêng biệt và cho phép triển khai hệ thống giám sát nhiệt độ đa điểm một cách linh hoạt. Với thiết kế giao tiếp I²C tiêu chuẩn, MAX30205 phù hợp cho các ứng dụng yêu cầu độ chính xác cao trong đo nhiệt độ cơ thể người hoặc môi trường xung quanh.

Sơ đồ kết nối chân cảm biến MAX30205 với ESP32:



Hình 3.3 Kết nối MAX30205 với ESP32

Chân VCC: nối nguồn 3.3V

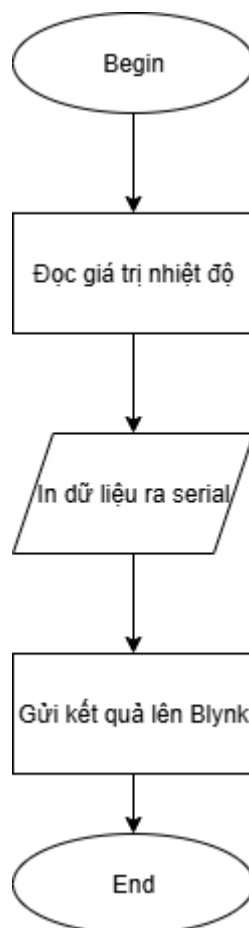
Chân GND: nối GND

Chân SDA: nối Pin 9

Chân SCL: nối Pin 8

Các chân A0, A1, A2: Cùng nối với GND để chọn địa chỉ là 0x48 (theo như datasheet)

Lưu đồ giải thuật hàm đọc nhiệt độ `updateTemperature()`:



Hình 3.4 Lưu đồ giải thuật hàm `updateTemperature()`

3.2.2 Cảm biến nhịp tim và nồng độ Oxy trong máu MAX30102

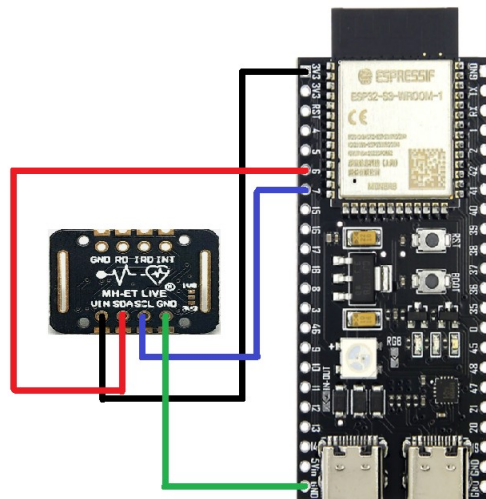


Hình 3.5 Cảm biến MAX30102

Đặc điểm kỹ thuật của MAX30102:

- Sử dụng giao tiếp I²C giúp dễ dàng kết nối với vi điều khiển qua hai chân SDA và SCL.
- Tích hợp sẵn bộ phát hồng ngoại (bước sóng 880nm) và LED đỏ (bước sóng 660nm) cùng photodiode để thu nhận tín hiệu phản xạ từ mao mạch dưới da.
- Hỗ trợ nhiều mức cấu hình mẫu và độ phân giải với bộ ADC 18 bit tích hợp bên trong, cho phép đo chính xác tín hiệu PPG (Photoplethysmogram).
- Dải điện áp hoạt động từ 1.8V (cho lõi) và từ 3.3V (cho giao tiếp I/O), phù hợp với các vi điều khiển phổ biến hiện nay.
- Tốc độ lấy mẫu linh hoạt, có thể cấu hình tần số lấy mẫu từ 50Hz đến 1000Hz, đáp ứng tốt các ứng dụng đo liên tục trong thời gian thực.

Sơ đồ nối chân cảm biến MAX30102 với ESP32:



Hình 3.6 Kết nối MAX30102 với ESP32

Chân VCC: nối nguồn 3.3V

Chân GND: nối GND

Chân SDA: nối Pin 6

Chân SCL: nối Pin7

Vì khi kết nối MAX30102 chung bus I²C với MAX30205 thì không thể kết nối được với cảm biến nên nhóm chọn kết nối bus I²C riêng cho MAX30102.

Nhóm dùng thư viện SparkFun MAX3010x Pulse and Proximity Sensor (được phát triển bởi SparkFun) thay cho thư viện MAX30100lib nhằm khắc phục lỗi treo dữ liệu khi gửi dữ liệu lên Blynk.

Các hàm có sẵn trong thư viện SparkFun MAX3010x Pulse and Proximity Sensor :

- **checkForBeat(particleSensor.getIR())**: Xác định xem có nhịp tim hay không từ tín hiệu hồng ngoại IR. Trả về true khi phát hiện nhịp tim.
- **particleSensor.check()**: Đọc dữ liệu mới từ cảm biến MAX30102 và lưu vào bộ đệm nội bộ.
- **particleSensor.available()**: Kiểm tra có dữ liệu mới trong buffer không.
- **particleSensor.nextSample()**: Chuyển con trỏ dữ liệu nội bộ sang sample tiếp theo trong buffer.
- **maxim_heart_rate_and_oxygen_saturation()**: Hàm xử lý tín hiệu từ ánh sáng đỏ và tia hồng ngoại (với số mẫu được quy định) bằng thuật toán để tính nhịp tim và nồng độ Oxy trong máu.

Phương pháp tính nhịp tim và nồng độ Oxy trong máu:

- **Nhịp tim:**

+Nhịp tim được xác định dựa trên tín hiệu hồng ngoại thu được từ cảm biến MAX30102. Mỗi khi cảm biến ghi nhận một nhịp đập, thời gian giữa hai nhịp liên tiếp sẽ được dùng để tính ra số nhịp tim trên phút (BPM - Beats Per Minute) theo công thức:

$$\text{BPM} = 60s/\Delta t$$

Trong đó, Δt là thời gian giữa hai nhịp tim liên tiếp (đơn vị giây). Thời gian này được tính bằng cách lấy hiệu giữa hiện tại và thời điểm xảy ra nhịp tim trước đó.

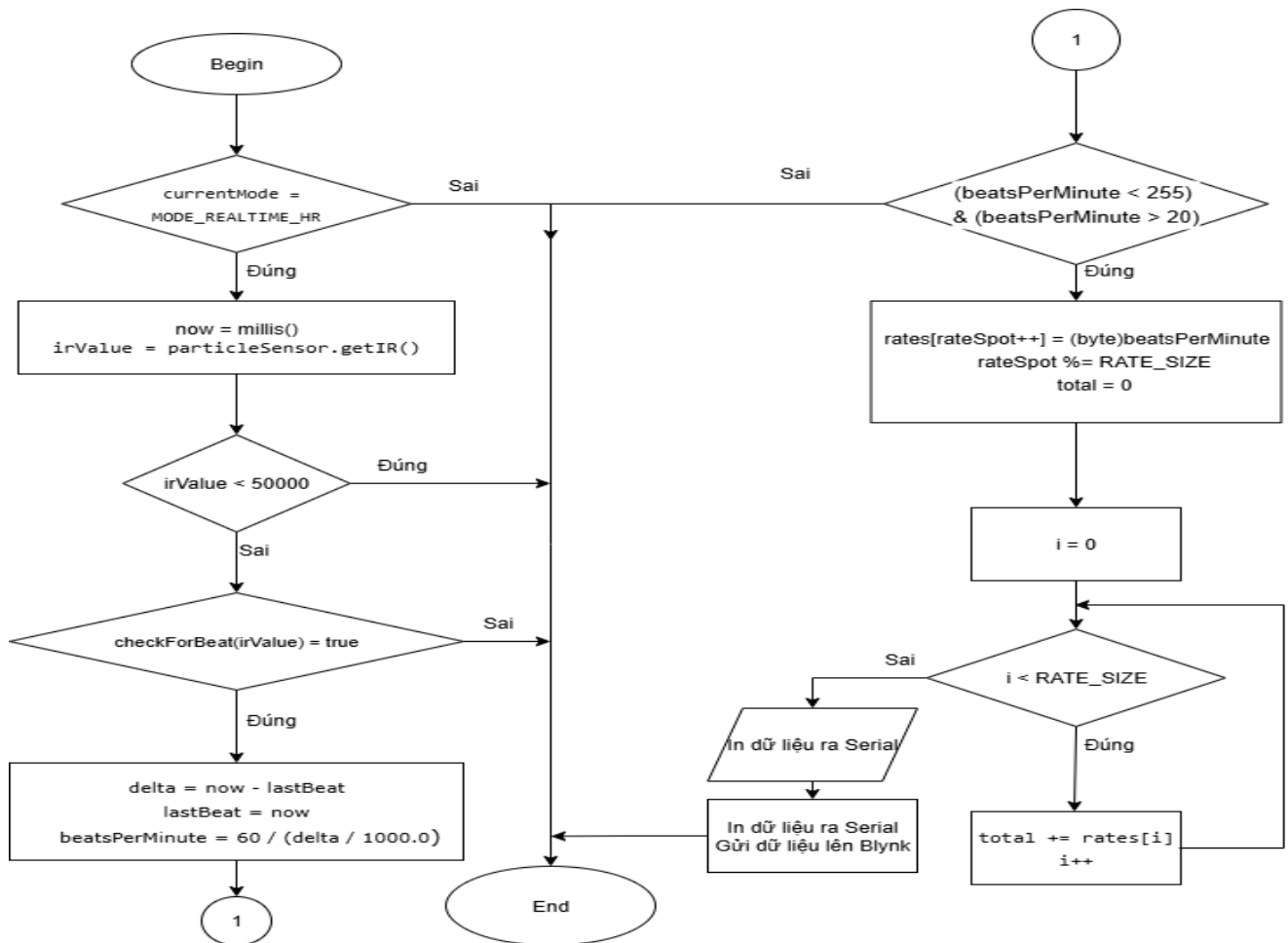
+Sau khi tính được BPM, giá trị này được lưu vào một mảng `rates[]` để lấy trung bình nhằm giảm nhiễu và dao động đột ngột. Cụ thể:

+Mảng `rates[]` có kích thước cố định (`RATE_SIZE = 4`) (tính nhịp tim trung bình của 4 giá trị gần nhất)

+Vị trí lưu hiện tại được đánh dấu bằng `rateSpot` và sẽ quay vòng.

+Tính trung bình cộng của toàn bộ mảng được lấy làm giá trị nhịp tim trung bình (`beatAvg`).

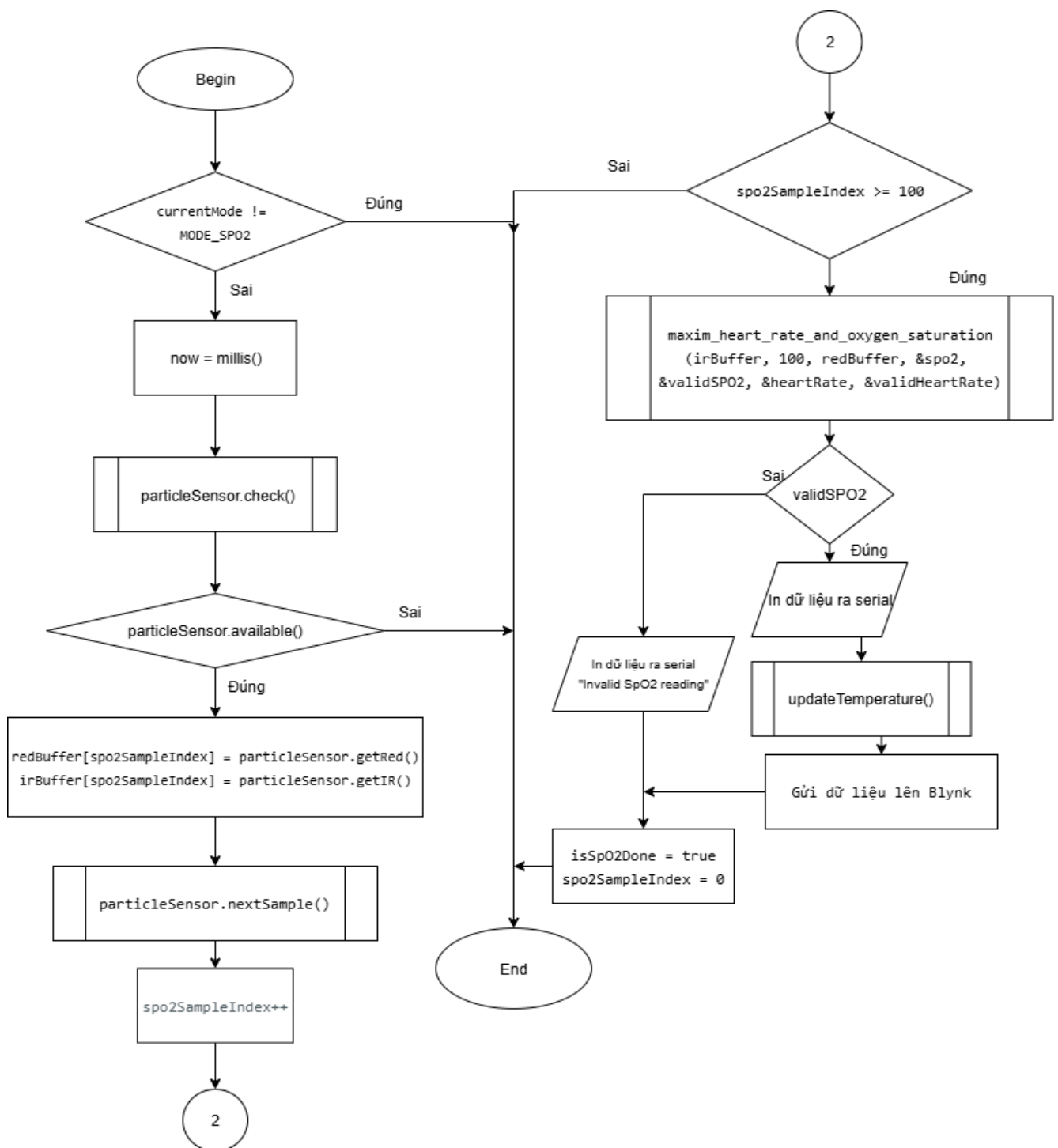
Lưu đồ giải thuật của hàm tính nhịp tim checkRealtimeHeartRate():



Hình 3.7 Lưu đồ giải thuật của hàm checkRealtimeHeartRate()

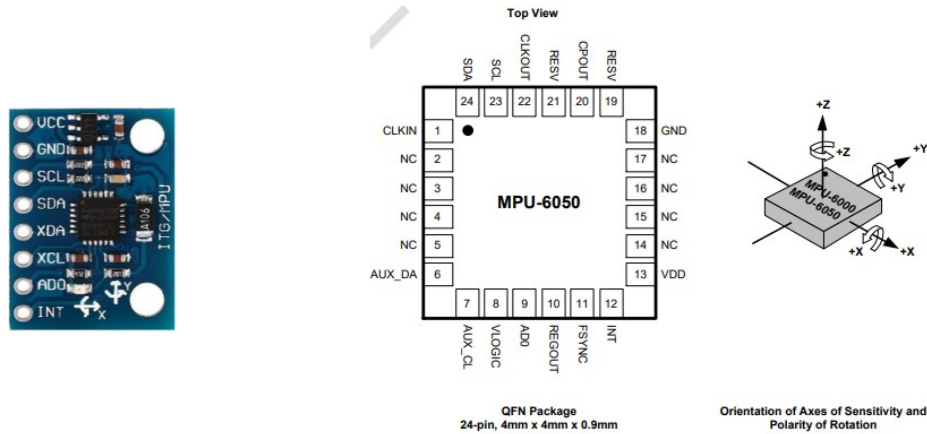
- **Nồng độ Oxy:** Vi điều khiển sẽ lấy lần lượt giá trị của ánh sáng đỏ và tia hồng ngoại và lưu vào hai mảng redBuffer[] và irBuffer[], mỗi mảng có kích thước 100 phần tử. Khi có đủ 100 phần tử thì hàm maxim_heart_rate_and_oxygen_saturation() sẽ được gọi để tính nồng độ Oxy trong máu.

Lưu đồ giải thuật hàm tính nồng độ Oxy trong máu
updateSpO2Measurement():



Hình 3.8 Lưu đồ giải thuật hàm updateSpO2Measurement()

3.2.3 Cảm biến gia tốc và góc quay MPU6050



Hình 3.9 Cảm biến MPU6050

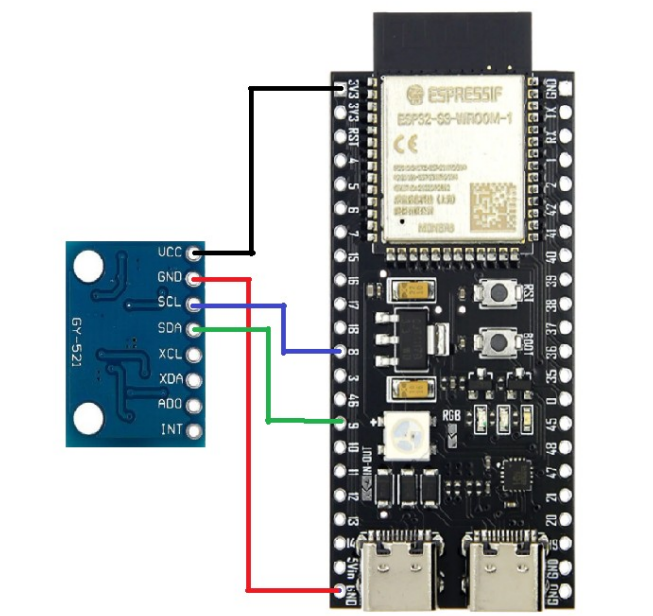
Đặc điểm kỹ thuật của MAX30102:

- MPU6050 là cảm biến tích hợp con quay hồi chuyển 3 trục (gyroscope) và gia tốc kế 3 trục (accelerometer) trong cùng một chip, giúp đo chuyển động và góc quay của vật thể theo thời gian thực.
- Giao tiếp thông qua chuẩn I²C , dễ dàng kết nối với các vi điều khiển như Arduino, ESP32 thông qua hai chân SDA và SCL.
- Có 4 dải đo gia tốc và 4 dải đo góc quay:

Đại lượng	Dải đo	Độ nhạy
Gia tốc	$\pm 2g$	16384 LSB/g
	$\pm 4g$	8192 LSB/g
	$\pm 8g$	4096 LSB/g
	$\pm 16g$	2048 LSB/g
Góc quay	$\pm 250^\circ/s$	131 LSB/($^\circ/s$)
	$\pm 500^\circ/s$	65.5 LSB/($^\circ/s$)
	$\pm 1000^\circ/s$	32.8 LSB/($^\circ/s$)
	$\pm 2000^\circ/s$	16.4 LSB/($^\circ/s$)

- Tích hợp bộ chuyển đổi tín hiệu tương tự sang số (ADC) 16 bit cho mỗi trục.
- Dải điện áp hoạt động từ 2.375V đến 3.46V.
- Nhiệt độ hoạt động ổn định trong khoảng $-40^\circ C$ đến $+85^\circ C$, hỗ trợ tốt trong nhiều môi trường sử dụng khác nhau.

Sơ đồ nối chân cảm biến MAX30102 với ESP32:



Hình 3.10 Kết nối MPU6050 với ESP32

Chân VCC: nối nguồn 3.3V

Chân GND: nối GND

Chân SDA: nối Pin 9

Chân SCL: nối Pin8

MPU6050 sẽ được kết nối chung bus I²C với cảm biến MAX30205

Chuẩn hóa dữ liệu từ MPU6050:

+Cảm biến MPU6050 trả về dữ liệu gia tốc và góc quay dưới dạng giá trị thô tương ứng với dải đo đã được cấu hình. Để sử dụng trong các tính toán vật lý và mô hình học máy, các giá trị này cần được chuyển đổi sang đơn vị chuẩn: m/s² cho gia tốc và °/s cho vận tốc góc.

+Công thức chung:

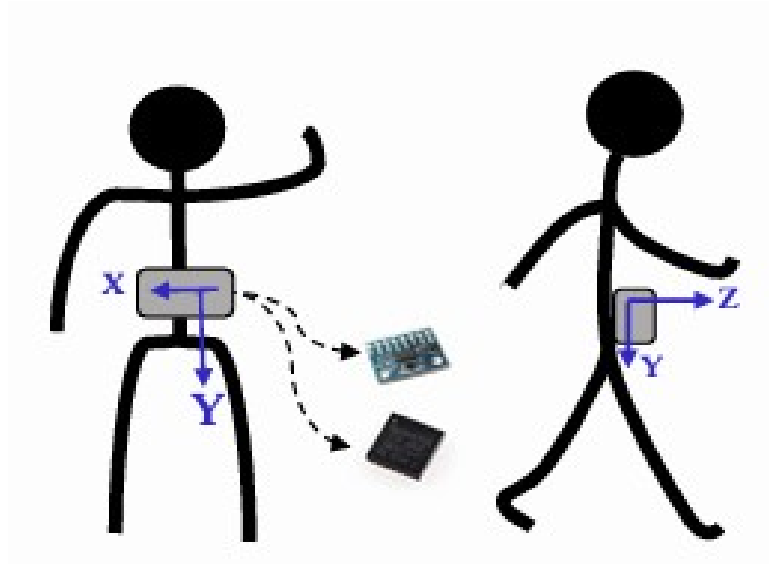
$$\text{Gia tốc (g)} = \frac{\text{Dữ liệu thô}}{\text{Độ nhạy (LSB / g)}}$$

$$\text{Tốc độ quay (°/s)} = \frac{\text{Dữ liệu thô}}{\text{Độ nhạy (LSB / (°/s))}}$$

+Phải lấy độ nhạy ứng với dải đo đã được cấu hình.

+Ví dụ: Cấu hình cảm biến với dải đo là ±8g và ±500 °/s thì phải chia cho độ nhạy có giá trị 4096 LSB/g và 65.5 LSB/(°/s)

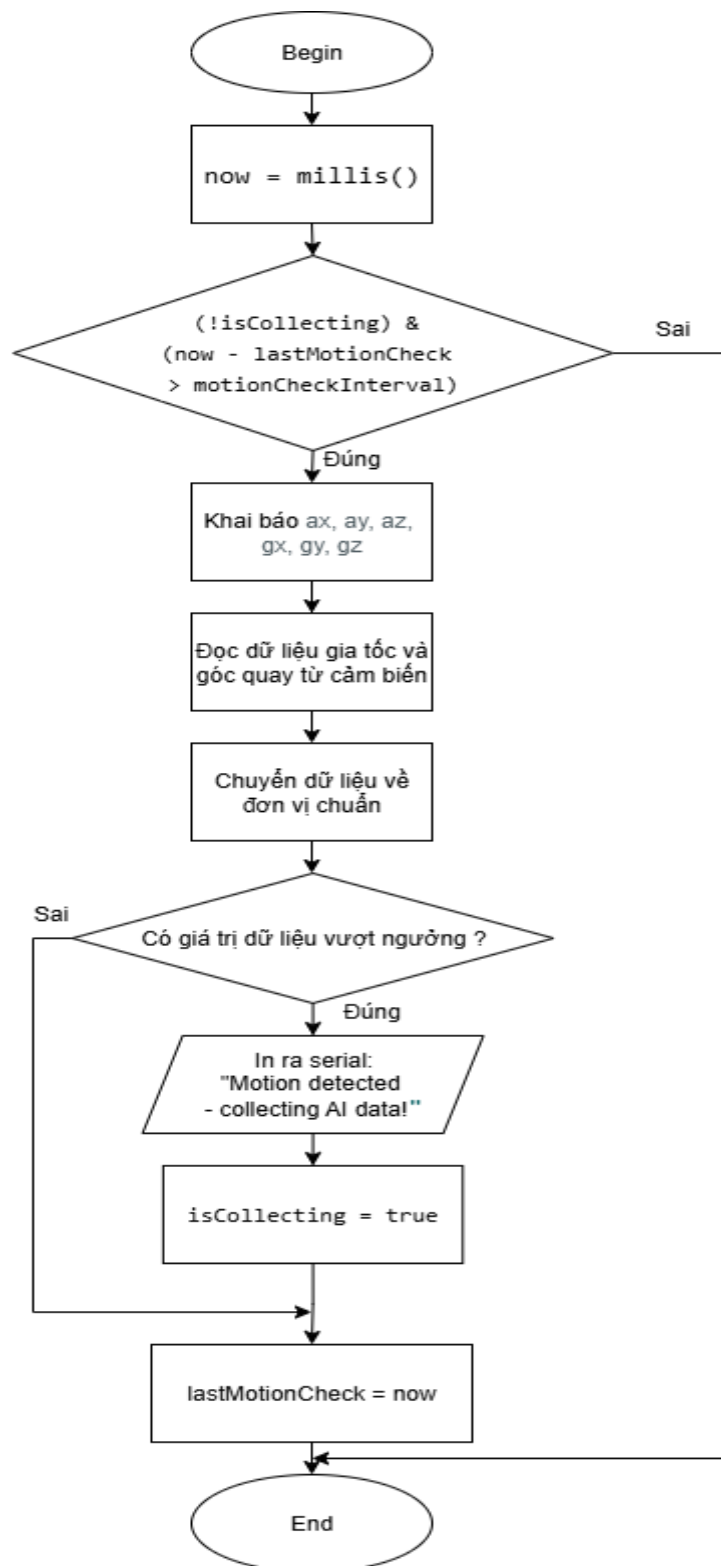
Cảm biến sẽ được gắn cố định trên thiết bị đeo ở eo với chiều của các Vec-tơ gia tốc như hình minh họa dưới đây:



Hình 3.11 Chiều Vec-tơ gia tốc của cảm biến MPU6050

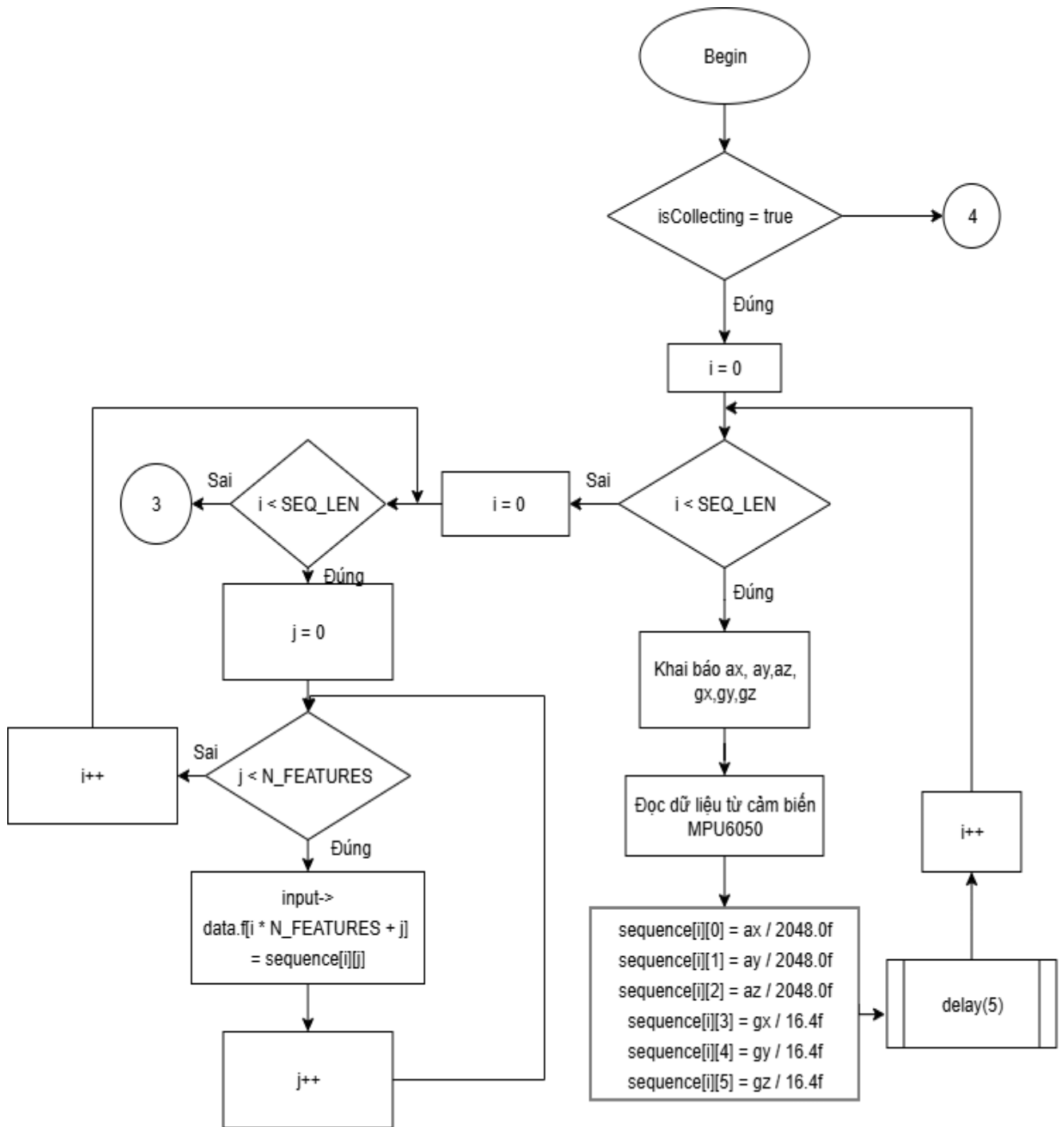
Nguyên lý hoạt động: Hàm `checkMotionAndStartCollection()` sẽ được gọi sao mỗi 100ms để kiểm tra liên tục các dữ liệu về gia tốc và góc quay, khi có dữ liệu vượt ngưỡng thì biến `isCollecting = true` qua đó cho phép hàm `collectDataForAI()` hoạt động để thu thập dữ liệu với tần số lấy mẫu 200Hz và chạy mô hình dự đoán.

Lưu đồ giải thuật của hàm checkMotionAndStartCollection()

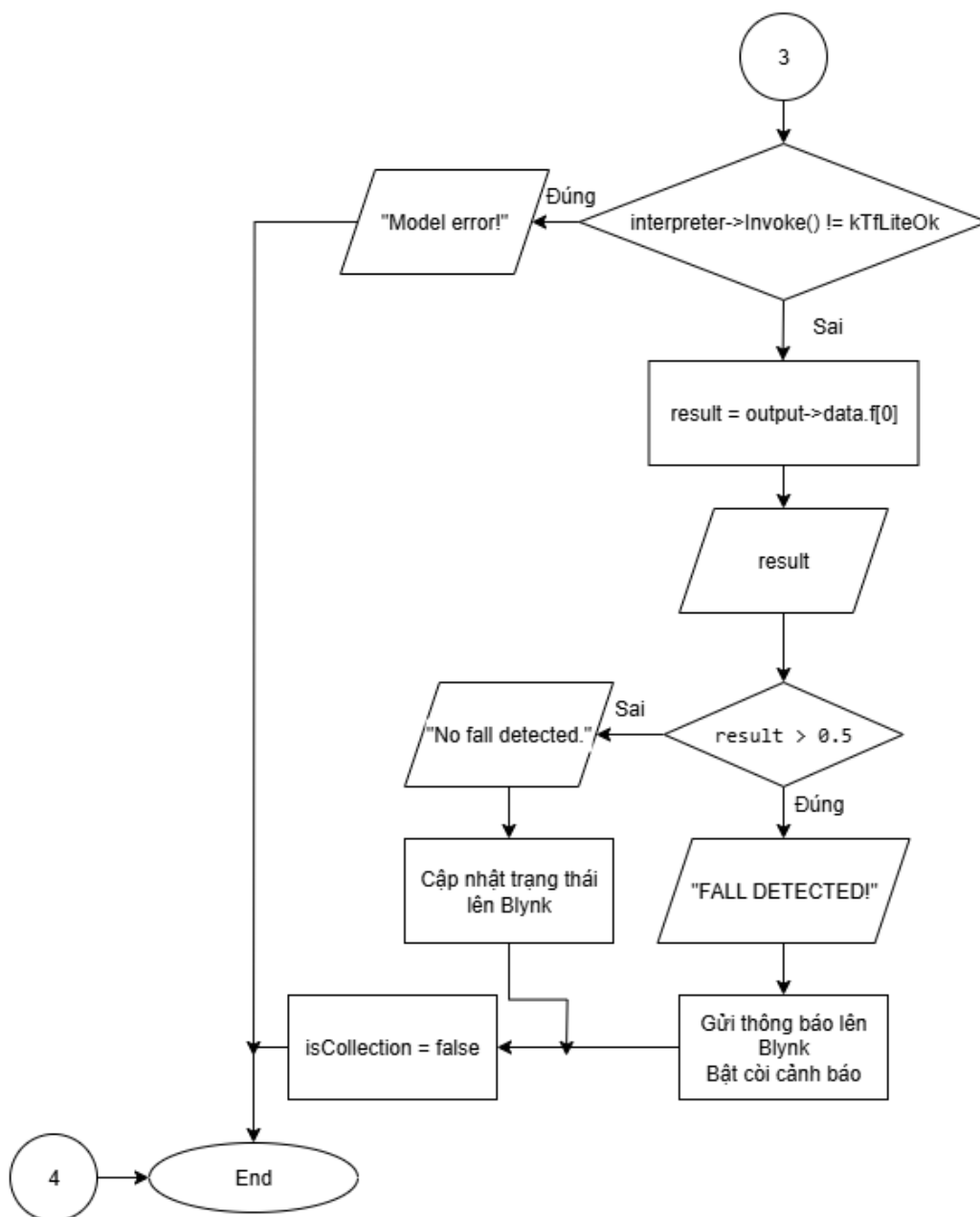


Hình 3.12 Lưu đồ giải thuật hàm checkMotionAndStartCollection()

Lưu đồ giải thuật hàm collectDataForAI()



Hình 3.13a Lưu đồ giải thuật cấu hàm collectDataForAI()



Hình 3.13b Lưu đồ giải thuật của hàm *collectDataForAI()*

3.2.4 Mô hình Deep Learning

a. Chuẩn bị dữ liệu huấn luyện

Nhóm chọn tập dữ liệu SisFall. Tập dữ liệu SisFall là một trong những bộ dữ liệu công khai được sử dụng rộng rãi trong nghiên cứu phát hiện té ngã và giám sát hoạt động của con người, đặc biệt hướng đến nhóm đối tượng là người cao tuổi. SisFall được xây dựng bởi nhóm nghiên cứu tại Đại học Antioquia (Colombia) nhằm cung cấp dữ liệu thực nghiệm phục vụ cho việc đánh giá và phát triển các thuật toán nhận diện té ngã. Tập dữ liệu bao gồm 4510 file tín hiệu, được thu thập từ 38 đối tượng (15 người cao tuổi và 23 người trẻ) khi thực hiện 19 loại hoạt động thường ngày khác nhau, bao gồm các hành vi bình thường (như đi bộ, ngồi xuống, đứng dậy,...) và 15 tình huống té ngã (té về phía trước, sau, trượt, ngã ngồi...). Mỗi mẫu dữ liệu được ghi nhận từ 2 cảm biến gia tốc (cảm

biến ADXL345 và cảm biến MMA8451Q)và 1 cảm biến con quay hồi chuyển 3 trục (ITG3200) gắn trên cơ thể, với tần số lấy mẫu 200 Hz.

Do dữ liệu trong tập SisFall dữ liệu thô chưa được chuẩn hóa nên việc cần làm đầu tiên là phải chuẩn hóa dữ liệu lại theo các đơn vị chuẩn là g và °/s với công thức:

$$\text{Gia tốc (g)} = \frac{\text{Dữ liệu thô}}{\text{Độ nhạy (LSB / g)}}$$

$$\text{Tốc độ quay (°/s)} = \frac{\text{Dữ liệu thô}}{\text{Độ nhạy (LSB / (°/s))}}$$

Vì các cảm biến dùng trong việc thu thập dữ liệu cho tập SisFall được cấu hình như sau: cảm biến ADXL345 với độ phân giải 13 bit và dải đo $\pm 16g$, cảm biến ITG3200 với độ phân giải 16 bit và dải đo $\pm 2000^\circ/s$ nên độ nhạy của gia tốc và tốc góc quay lần lượt là 256 (theo datasheet ADXL345) và 14.375 (theo datasheet ITG3200).

Dữ liệu của cảm biến MMA8451Q sẽ bị loại bỏ vì cảm biến MPU6050 dùng trong mô hình chỉ có 3 trục gia tốc hướng và 3 trục góc quay nên phải loại bỏ 1 cảm biến để dữ liệu ngõ vào có cùng kích thước với dữ liệu được huấn luyện. Các dữ liệu về hoạt động đi bộ và chạy bộ cũng bị loại bỏ vì nhóm nhận thấy các hoạt động đó không dễ bị nhầm lẫn với té ngã.

Quá trình phân đoạn dữ liệu:

+Vì tập trong tập SisFall các hành động có thời gian lấy mẫu khác nhau nhưng mô hình Deep Learning lại yêu cầu ngõ vào có cùng kích thước nên ta cần phân đoạn lại dữ liệu. Thay vì dùng phương pháp “cửa sổ trượt” tuy đơn giản nhưng tốn tài nguyên và thiếu tính chính xác thì nhóm chọn phương pháp “điểm tác động”. Nguyên lý của phương pháp này là xác định mẫu có tổng độ lớn gia tốc lớn nhất (SVM) vì khi đó lực tác động lên cơ thể là lớn nhất sau đó cắt lấy 600 bắt đầu từ mẫu (thay vì các mẫu xung quanh như trong các nghiên cứu [6] và [5]). Gia tốc của điểm tác động lớn nhất được tính bằng công thức.

$$SVM = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

+Phương pháp này không chỉ giúp tiết kiệm tài nguyên mà còn loại bỏ các mẫu không cần thiết giúp tăng độ chính xác so với phương pháp “cửa sổ trượt” thông thường trong quá trình huấn luyện.

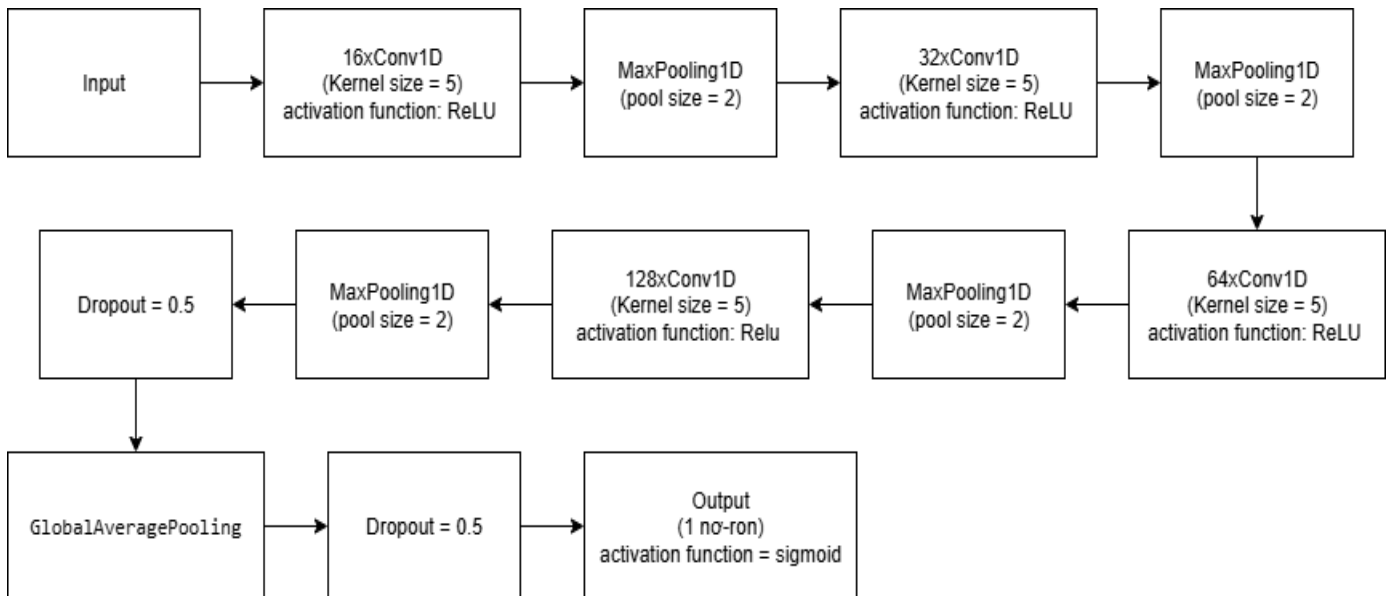
Dán nhãn dữ liệu theo kiểu nhị phân: Các hoạt động té ngã sẽ được gán nhãn là 1, còn các hoạt động thường ngày khác sẽ được gán nhãn là 0. Kiểu dán nhãn này sẽ biến mô hình thành mô hình phân loại nhị phân.

Dữ liệu sẽ được chia ra theo tỉ lệ 80% cho quá trình huấn luyện và 20% còn lại dùng để đánh giá.

b. Xây dựng mô hình Deep Learning

Mô hình Deep Learning được xây dựng dùng mạng nơ-ron tích chập (CNNs) bao gồm các lớp tích chập 1 chiều Conv1D thường được dùng cho các bài toán có dữ liệu dạng chuỗi theo thời gian (được dùng trong [6]).

Cấu trúc của mô hình Deep Learning:



Hình 3.14 Cấu trúc mô hình Deep Learning

Mô hình có:

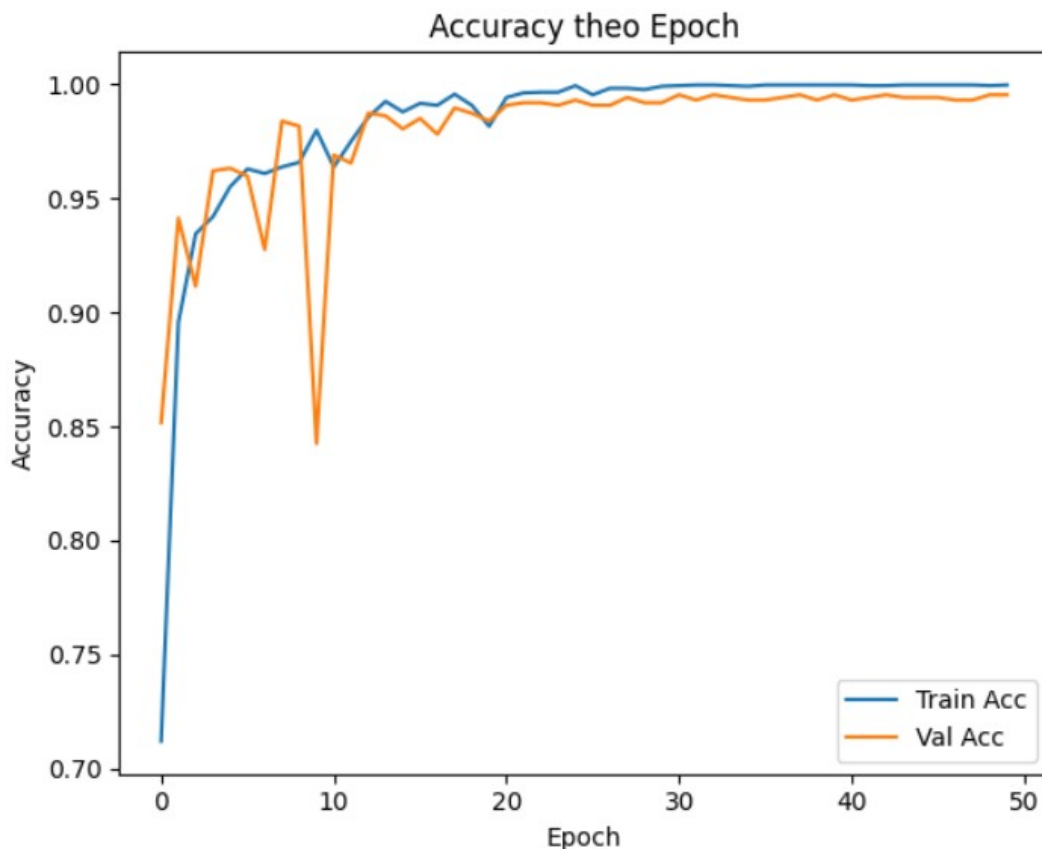
- Input layer: Lớp đầu vào có kích thước là (600×6) với 600 mẫu tín hiệu và mỗi mẫu có 6 đặc trưng theo gia tốc và góc quay.
- Lớp Conv thứ nhất: Với 16 kernel có kích thước là 5 và hàm kích hoạt ReLU sẽ cho ngõ ra có kích thước là (596×16) .
- Lớp MaxPooling thứ nhất: Với pool size là 2 sẽ cho ngõ ra có kích thước (298×16) .
- Lớp Conv thứ hai: Với 32 kernel có kích thước là 5 và hàm kích hoạt ReLU sẽ cho ngõ ra có kích thước là (294×32) .
- Lớp MaxPooling thứ nhất: Với pool size là 2 sẽ cho ngõ ra có kích thước (147×32) .
- Lớp Conv thứ ba: Với 64 kernel có kích thước là 5 và hàm kích hoạt ReLU sẽ cho ngõ ra có kích thước là (143×64) .
- Lớp MaxPooling thứ ba: Với pool size là 2 sẽ cho ngõ ra có kích thước (71×64) .
- Lớp Conv thứ tư: Với 128 kernel có kích thước là 5 và hàm kích hoạt ReLU sẽ cho ngõ ra có kích thước là (67×128) .
- Lớp MaxPooling thứ tư: Với pool size là 2 sẽ cho ngõ ra có kích thước (33×128) .
- Dropout với tỷ lệ là 0.5 nhằm giảm hiện tượng overfitting bằng cách ngẫu nhiên loại bỏ 50% các node trong quá trình huấn luyện.

- Lớp GlobalAveragePooling1D: Lớp này thực hiện trung bình tất cả giá trị trong mỗi feature map, biến đổi tensor từ dạng (33 x 128) thành kích thước (1 x 128). Lớp này có tác dụng thay cho lớp kết nối đầy đủ (fully connected) giúp giảm đáng kể số lượng tham số của mô hình.
- Tiếp tục Dropout là 0.5 nhằm tăng cường khả năng tổng quát hóa mô hình.
- Lớp Dense ngõ ra: Vì là mô hình phân loại nhị phân nên ngõ ra sẽ có một nơ-ron và dùng hàm kích hoạt sigmoid sẽ trả về giá trị từ 0 tới 1.
- Tổng số lượng tham số của mô hình 54609.

Quá trình huấn luyện mô hình được thực hiện với các thông số như sau:

- Batch size: 16
- Hàm tối ưu: Adam
- Hàm mất mát: Binary Cross-Entropy
- Epoch: 50
- Learning rate scheduler:
ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience = 3, min_lr = 1e-5, verbose = 1)

Kết quả sau quá trình huấn luyện:



Hình 3.15 Biểu đồ sự thay đổi độ chính xác trên tập huấn luyện và đánh giá theo epoch

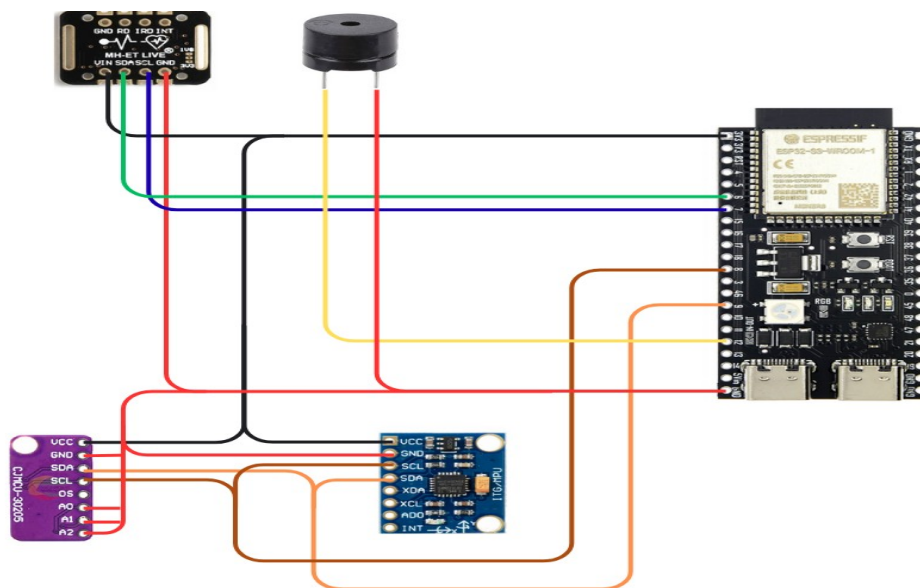
Mô hình đạt được độ chính xác trên tập đánh giá là 99.54% và không có sự chênh lệch với tập huấn luyện nên mô hình không bị overfitting. Tuy nhiên, đây không phản ánh lên tính chính xác của mô hình trong quá trình sử dụng.

Sau khi quá trình huấn luyện kết thúc mô hình sẽ được chuyển đổi và lưu dưới dạng TensorFlow Lite. Nhưng nếu muốn triển khai mô hình lên ESP32 ta phải chuyển đổi một lần nữa từ file TensorFlow Lite đó thành một mảng kiểu unsigned char[] trong ngôn ngữ C để có thể thêm vào đoạn mã chương trình và triển khai lên ESP32.

3.2.5 Khối điều khiển trung tâm

Khối điều khiển trung tâm là một ESP32 có nhiệm vụ điều khiển toàn bộ chức năng của mô hình gồm: xử lý dữ liệu nhiệt độ, nhịp tim, nồng độ Oxy trong máu từ các cảm biến; phát hiện bất thường trong chuyển động, chạy mô hình suy luận khi phát hiện bất thường, cảnh báo khi phát hiện té ngã và gửi dữ liệu lên ứng dụng Bynk để theo dõi từ xa.

Sơ đồ nối chân toàn bộ mô hình:



Hình 3.16 Sơ đồ kết nối chân toàn bộ mô hình

Các chân kết nối theo như các phần đã nêu ở trên. Chân dương của còi nối với pin 13 của ESP32

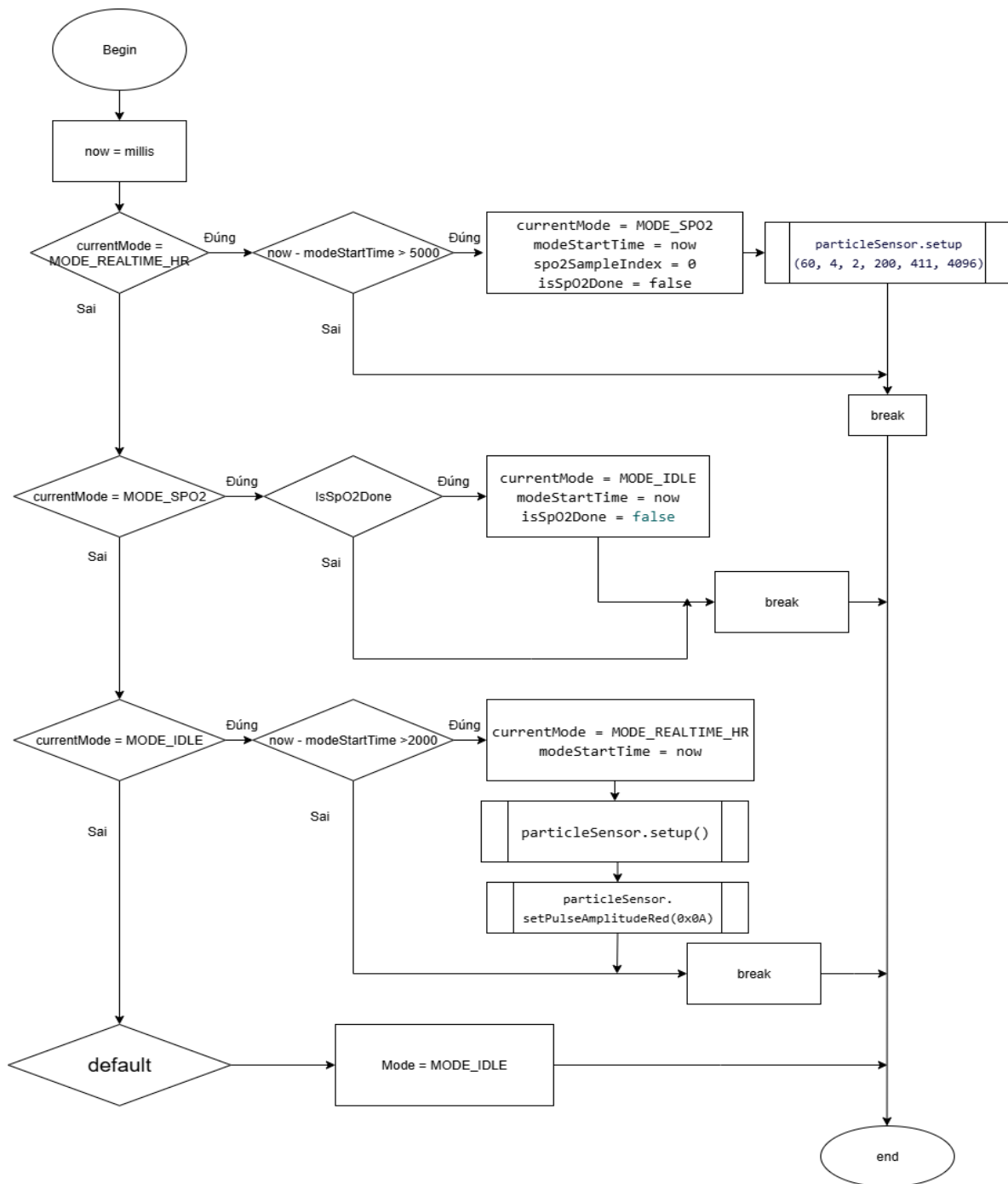
Nguyên lý hoạt động của khối điều khiển trung tâm:

+Hàm manageSensorModes() đóng vai trò điều phối các chế độ hoạt động của cảm biến MAX30102 dựa trên giá trị của biến currentMode. Cụ thể, khi hệ thống đang ở chế độ đo nhịp tim, hàm checkRealtimeHeartRate() sẽ được gọi để thu thập và xử lý dữ liệu nhịp tim theo thời gian thực. Ngược lại, khi chuyển sang chế độ đo nồng độ oxy trong máu (SpO₂), hệ thống sẽ thực thi hàm

updateSpO2Measurement(). Ở chế độ này, để đảm bảo độ chính xác trong tính toán, cảm biến cần thu thập đủ 100 mẫu liên tục, với mỗi mẫu tương ứng với một vòng lặp chương trình. Việc này giúp duy trì tính liên tục của phép đo và tránh bị gián đoạn nếu xảy ra tình huống khẩn cấp như té ngã.

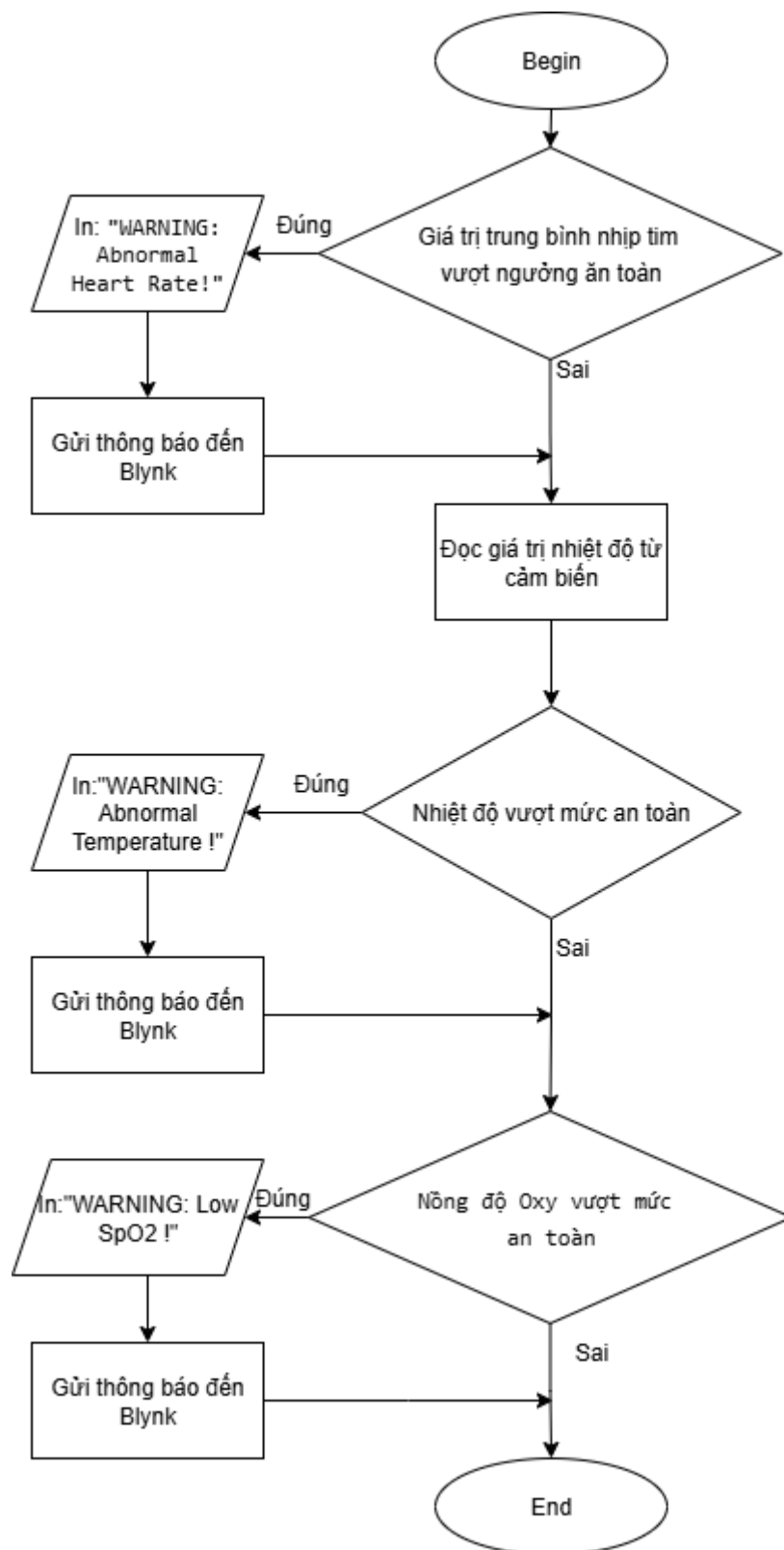
+Song song với đó, khối điều khiển trung tâm cũng liên tục theo dõi dữ liệu thu được từ cảm biến MPU6050. Nếu phát hiện giá trị vượt ngưỡng đã định nghĩa, mô hình học sâu sẽ được kích hoạt để xác định xem có tình huống té ngã xảy ra hay không. Trong trường hợp té ngã được phát hiện, hệ thống sẽ ngay lập tức kích hoạt còi cảnh báo và gửi thông báo đến ứng dụng Blynk. Bên cạnh đó, nếu bất kỳ chỉ số sức khỏe nào vượt qua ngưỡng an toàn đã được thiết lập, một cảnh báo cũng sẽ được gửi đến người thân hoặc người chăm sóc. Trước khi đưa ra cảnh báo, hệ thống sẽ chờ 30 giây để đảm bảo dữ liệu đo được đã ổn định, nhằm hạn chế cảnh báo sai lệch.

Lưu đồ giải thuật hàm manageSensorModes():



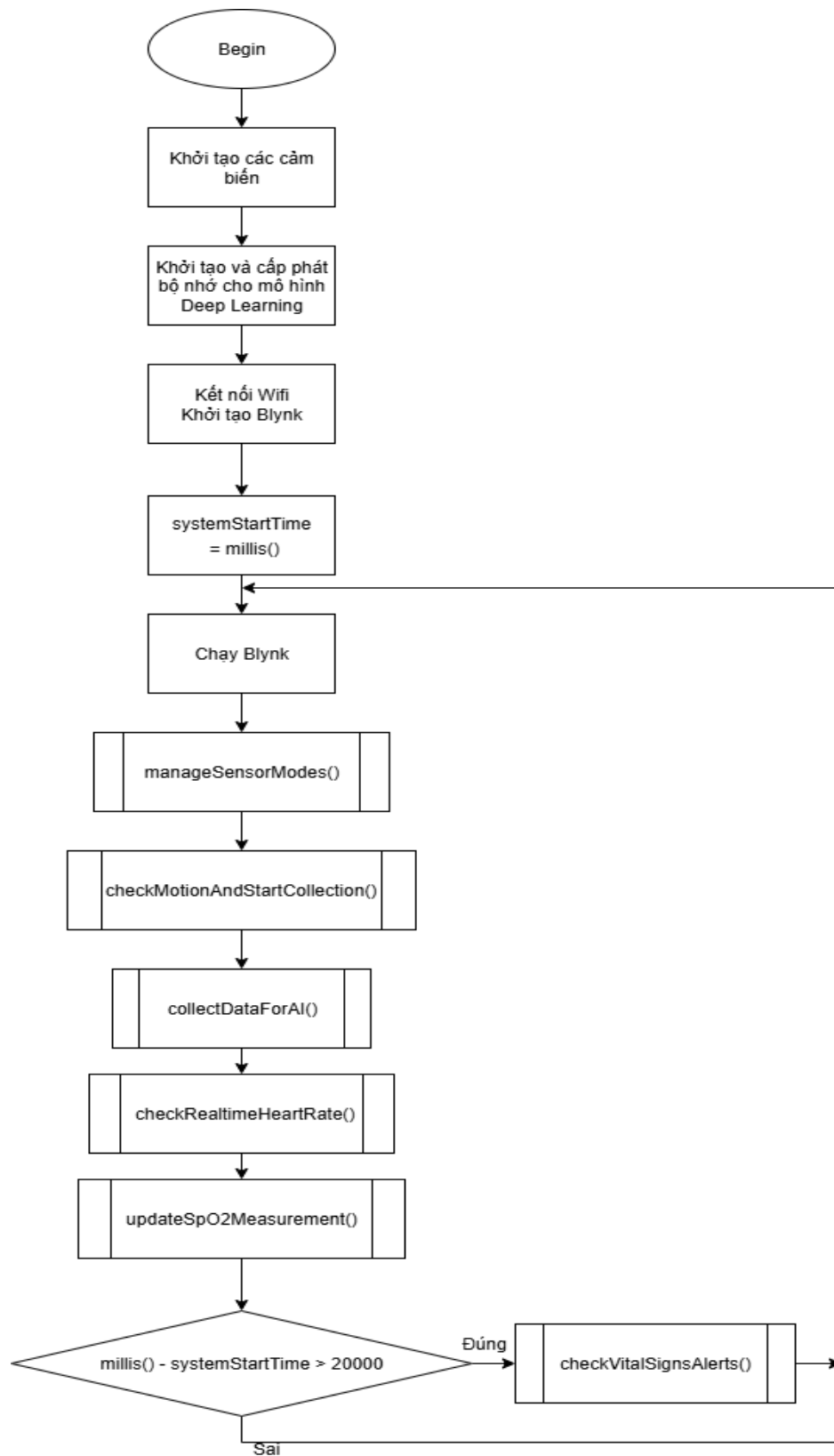
Hình 3.17 Lưu đồ giải thuật hàm manageSensorModes()

Lưu đồ giải thuật hàm gửi cảnh báo về chỉ số sức khỏe
checkVitalSignsAlerts()



Hình 3.18 Lưu đồ giả thuật của hàm checkVitalSignsAlerts()

Lưu đồ giải thuật của chương trình chính



Hình 3.19 Lưu đồ giải thuật của chương trình chính

CHƯƠNG 4: GIỚI THIỆU PHẦN MỀM

4.1 Ngôn ngữ lập trình

4.1.1 Python

a. Giới thiệu

Python là một ngôn ngữ lập trình đa năng, được ưa chuộng trong lĩnh vực khoa học dữ liệu và học máy. Nó nổi bật nhờ cú pháp đơn giản, dễ học và một hệ sinh thái thư viện phong phú như Pandas, NumPy, SciPy, scikit-learn, TensorFlow, Keras, Matplotlib, Seaborn, và Plotly.

b. Các thư viện numpy, TensorFlow

Trong hệ thống phát hiện té ngã sử dụng dữ liệu cảm biến, NumPy và TensorFlow là hai thư viện Python đóng vai trò cực kỳ quan trọng. NumPy được dùng để xử lý và tiền xử lý dữ liệu dưới dạng mảng số học, trong khi TensorFlow chuyên về xây dựng, huấn luyện và triển khai các mô hình học sâu dựa trên dữ liệu chuỗi thời gian.

Đặc điểm và tính năng chính của NumPy:

- NumPy nổi bật với khả năng xử lý mảng số học hiệu quả, là nền tảng cho nhiều thư viện khoa học dữ liệu khác:
- Cấu trúc dữ liệu ndarray: Điểm mạnh nhất của NumPy là cấu trúc dữ liệu ndarray (mảng N chiều) hiệu năng cao. Mảng NumPy là một "view" trên bộ nhớ, nơi tất cả các phần tử trong cùng một mảng phải cùng kiểu dữ liệu, giúp tối ưu hóa việc truy cập và xử lý.
- Tính toán vector hóa và Broadcasting: NumPy hỗ trợ các phép toán vector hóa và broadcasting. Điều này cho phép thực hiện các phép toán trên toàn bộ mảng hoặc giữa các mảng có kích thước khác nhau một cách cực kỳ hiệu quả, loại bỏ nhu cầu sử dụng vòng lặp Python truyền thống, từ đó tăng tốc độ tính toán đáng kể.
- Hàm toán học phong phú: Thư viện này cung cấp một loạt các hàm toán học, đại số tuyến tính và các hàm tích chập (ví dụ: `numpy.convolve` để thực hiện tích chập giữa hai dãy số).
- Thống kê và tiền xử lý dữ liệu: NumPy giúp dễ dàng tính toán các đặc trưng thống kê cơ bản như trung bình, độ lệch chuẩn, giá trị RMS, cũng như chuẩn hóa dữ liệu chỉ bằng các phép toán trên mảng.
- Tích hợp tốt với các thư viện khác: NumPy là xương sống của nhiều thư viện phổ biến. Các cấu trúc dữ liệu của Pandas và Matplotlib đều dựa trên NumPy; tất cả chuỗi dữ liệu trong Matplotlib và Pandas đều tự động được chuyển thành

ndarray. SciPy cũng được xây dựng trên NumPy, cung cấp thêm các công cụ khoa học chuyên sâu như tối ưu hóa, xử lý tín hiệu và xác suất.

Đặc điểm và tính năng chính của TensorFlow:

- TensorFlow là một thư viện mạnh mẽ dành cho học máy và học sâu, đặc biệt trong việc xây dựng và triển khai các mô hình phức tạp:
- Kiến trúc đồ thị tính toán và Eager Execution: TensorFlow có kiến trúc dựa trên đồ thị tính toán, trong đó dữ liệu được biểu diễn dưới dạng các tensors (mảng đa chiều). Ban đầu (TensorFlow 1.x), các phép tính được xây dựng thành đồ thị tĩnh trước khi thực thi. Tuy nhiên, TensorFlow 2.x đã hỗ trợ chế độ eager execution, cho phép thực thi lệnh ngay lập tức, giúp quá trình gỡ lỗi và phát triển trở nên thuận tiện hơn nhiều.
- API cấp cao (Keras) và Pipeline dữ liệu (tf.data): TensorFlow cung cấp các API cấp cao như Keras để định nghĩa và huấn luyện các mô hình mạng nơ-ron một cách dễ dàng. Ngoài ra, tf.data giúp xây dựng các pipeline dữ liệu hiệu quả, tối ưu hóa việc nạp và xử lý dữ liệu cho quá trình huấn luyện.
- Các lớp cơ bản cho học sâu: Các lớp cơ bản trong tf.keras.layers như Dense (lớp kết nối đầy đủ), LSTM (mạng hồi quy dài hạn) hay Conv1D/2D (lớp tích chập 1D/2D) hỗ trợ việc xây dựng các mô hình học sâu phức tạp như CNN và RNN, phù hợp cho dữ liệu chuỗi thời gian, hình ảnh hoặc video.
- Hỗ trợ phần cứng đa dạng: TensorFlow được thiết kế để chạy trên nhiều loại phần cứng khác nhau, bao gồm CPU, GPU (với công nghệ CUDA/SYCL để tăng tốc đáng kể), và cả TPU của Google.
- Công cụ trực quan hóa (TensorBoard): Thư viện này tích hợp công cụ TensorBoard mạnh mẽ, cho phép trực quan hóa toàn bộ quá trình huấn luyện. Người dùng có thể theo dõi các chỉ số quan trọng như loss, accuracy, xem đồ thị mô hình, và phân tích các embedding.
- Triển khai trên thiết bị di động và nhúng (TensorFlow Lite): TensorFlow Lite (LiteRT) là phiên bản rút gọn của TensorFlow, được thiết kế đặc biệt cho các thiết bị di động và nhúng. Các mô hình được tối ưu hóa và nén để chạy hiệu quả trên các thiết bị có cấu hình thấp, mở rộng khả năng ứng dụng của AI đến các thiết bị IoT và biên.

Vai trò của NumPy trong tiền xử lý dữ liệu

- Trong hệ thống phát hiện té ngã, NumPy đóng vai trò quan trọng trong việc lưu trữ và xử lý tín hiệu cảm biến thô (như dữ liệu từ gia tốc kế và con quay hồi chuyển) dưới dạng mảng. Nhờ các phép toán vector hóa, NumPy có thể xử lý các Lọc tín hiệu: Thực hiện các phép cộng hoặc nhân trên mảng để loại bỏ nhiễu hoặc làm mịn dữ liệu.
- Trượt cửa sổ (sliding windows): Chia chuỗi thời gian dài thành các đoạn con (windows) để phân tích độc lập.
- Tính toán các đặc trưng thống kê: Dễ dàng tính toán các giá trị như RMS (root-mean-square), giá trị tối đa, trị tuyệt đối trung bình, v.v.
- Chuẩn hóa dữ liệu: Các kỹ thuật chuẩn hóa đơn giản và hiệu quả như trừ trung bình và chia độ lệch chuẩn được thực hiện nhanh chóng bằng NumPy.
- Các đặc trưng sơ cấp này, cùng với các thuật toán như lọc số hay phân tích tần số, sẽ được dùng làm đầu vào cho mô hình học máy.

Vai trò của TensorFlow trong mô hình học sâu

- TensorFlow chịu trách nhiệm chính trong việc xây dựng, huấn luyện và đánh giá các mô hình học sâu (ví dụ: mạng LSTM, CNN) dựa trên các đặc trưng cảm biến đã được trích xuất.
- Huấn luyện mô hình hiệu quả: Nhờ khả năng tự động tính đạo hàm, các thuật toán tối ưu hóa (như Adam, SGD) và tích hợp GPU, TensorFlow giúp huấn luyện các mạng nơ-ron phức tạp một cách hiệu quả và nhanh chóng.
- Hệ sinh thái hỗ trợ triển khai thực tế: Ngoài ra, hệ sinh thái TensorFlow còn hỗ trợ các công cụ như TensorFlow Serving để phục vụ mô hình trong môi trường sản xuất, hay các thư viện mở rộng như TinyML và Edge TPU để đưa mô hình AI vào các ứng dụng thực tế trên thiết

4.1.2 C/C++

a. Giới thiệu

Ngôn ngữ **C và C++** chiếm ưu thế tuyệt đối trong lập trình nhúng, chiếm đến hơn 95% mã nguồn trong các hệ thống nhúng hiện nay. Lý do là C/C++ tối ưu hiệu năng và kiểm soát phần cứng rất tốt. Ngôn ngữ này biên dịch trực tiếp xuống mã máy, cho phép lập trình viên thao tác bộ nhớ và ngoại vi ở mức thấp (sử dụng con trỏ, truy xuất thanh ghi) để tinh chỉnh hiệu suất và độ trễ của hệ thống. C/C++ cũng tương thích ngược với mã C có sẵn (legacy code), do đó các thư viện phần cứng (driver) thường được viết bằng C/C++ để dễ tích hợp và tái sử dụng. Vì vậy, hầu hết firmware, driver và giao tiếp phần cứng trên vi điều khiển đều được viết bằng

C/C++. Ví dụ, ESP-IDF – nền tảng lập trình chính thức cho ESP32 – mặc định sử dụng ngôn ngữ C/C++ và tích hợp thư viện chuẩn (Newlib) cùng các API phần cứng theo chuẩn của GCC. Thực tế, Espressif khuyến nghị phát triển ứng dụng trên ESP32 bằng C/C++ (hỗ trợ C++23) để tận dụng mọi tính năng của phần cứng.

Trong khi C/C++ thống trị thị trường nhúng, các ngôn ngữ cấp cao như Python hay Java cũng bắt đầu được sử dụng trong một số ứng dụng nhúng. Python (qua MicroPython) có cú pháp thân thiện, tiện lợi cho người mới, nhưng nó là ngôn ngữ thông dịch nên chiếm nhiều bộ nhớ và thực thi chậm hơn. Ví dụ, Python dễ đọc và tái sử dụng mã nhờ thư viện phong phú, nhưng mã Python thường kém hiệu quả khi chạy trên vi điều khiển so với C/C++. Java (đặc biệt Java ME) ít phổ biến trên các vi điều khiển vì yêu cầu máy ảo và bộ nhớ lớn, thường chỉ dùng cho các hệ nhúng có phần cứng mạnh (như smartphone hoặc gateway IoT). Tóm lại, tuy Python và Java có ưu điểm về năng suất lập trình, nhưng C/C++ vẫn là lựa chọn hàng đầu cho các hệ thống nhúng thực thụ bởi khả năng tối ưu tài nguyên, độ ổn định lâu dài và hỗ trợ thao tác phần cứng trực tiếp.

b. Đặc điểm của C/C++ cho lập trình nhúng

Truy cập bộ nhớ trực tiếp: C/C++ cho phép sử dụng con trỏ và ghi đè thanh ghi để thao tác trực tiếp với phần cứng. Ví dụ, lập trình viên có thể lấy địa chỉ của một thanh ghi GPIO và ghi giá trị vào đó, điều mà khó thực hiện được trong ngôn ngữ cấp cao khác. Khả năng này giúp tối ưu tối đa hiệu suất, giảm thiểu độ trễ. Như Ryu Ishihara nhận định, C++ “cung cấp khả năng truy cập cấp thấp tới tài nguyên phần cứng” để tối ưu hiệu suất và giảm thiểu sử dụng bộ nhớ. Cũng nhờ đó, việc viết device driver hay giao tiếp phần cứng (UART, SPI, I²C, v.v.) trở nên hiệu quả và nhanh chóng.

Hiệu suất và tối ưu: Mã C/C++ được biên dịch thành mã máy tối ưu, tạo mã thực thi gọn và nhanh, thích hợp cho hệ thống nhúng tài nguyên hạn chế. Nhiều nghiên cứu chỉ ra rằng chương trình nhúng viết bằng C/C++ chạy nhanh hơn, chiếm ít bộ nhớ hơn so với cùng thuật toán viết bằng Python hay Java. Đặc biệt trong các ứng dụng thời gian thực (real-time), yêu cầu phản hồi tức thì, khả năng tối ưu hóa ở mức mã máy là rất quan trọng. C/C++ cho phép lập trình viên thiết lập các tùy chọn tối ưu (như -O2) và dùng các kỹ thuật tối ưu thấp như inline assembly để đạt được hiệu quả cao nhất.

Hỗ trợ lập trình hướng đối tượng (C++): C++ kế thừa toàn bộ ưu điểm của C và bổ sung khả năng lập trình hướng đối tượng. Tính năng OOP như lớp, đa hình, kế thừa giúp tổ chức mã nguồn rõ ràng, tái sử dụng cao và dễ mở rộng. Chuyên gia Jacob Beningo nhận xét C++ giúp “tính mở rộng cao và tái sử dụng mã” trong phát triển nhúng, nhờ các khuôn mẫu và thừa kế của nó. Ví dụ, ta có thể định nghĩa một lớp cơ sở cho một loại cảm biến rồi kế thừa thêm các lớp

con cho từng model cụ thể, tăng khả năng bảo trì và giảm sai sót. Chuẩn thư viện C++ cũng rất phong phú (vector, algorithm, string, v.v.), hỗ trợ việc phát triển nhanh và an toàn.

Thư viện chuẩn và thư viện phần cứng C/C++ tích hợp chặt chẽ với các thư viện chuẩn và thư viện phần cứng. Trên ESP32, Espressif dùng thư viện chuẩn Newlib cho C trong ESP-IDF, và cũng hỗ trợ nhiều thư viện của bên thứ ba. Đồng thời, hầu hết MCU có thư viện HAL (Hardware Abstraction Layer) hoặc CMSIS (trên ARM) để dễ dàng cấu hình ngoại vi. ESP-IDF cung cấp HAL và driver cấp thấp cho các ngoại vi (GPIO, UART, Wi-Fi, Bluetooth...) cho phép điều khiển phần cứng ở nhiều mức trừu tượng khác nhau.

c. Cấu trúc chương trình C/C++ cơ bản trong vi điều khiển

Một chương trình nhúng điển hình bằng C/C++ thường có cấu trúc sau: hàm main() (hoặc app_main() trong ESP-IDF) khởi tạo hệ thống rồi chạy trong vòng lặp vô hạn để giữ thiết bị hoạt động liên tục.

```
int main(void) {  
    // 1. Khởi tạo hệ thống (khởi tạo ngoại vi, thiết lập bộ đếm  
    thời gian, ...  
    System_Init();  
    // 2. Vòng lặp chính lặp vô hạn  
    while (1) {  
        // Thực hiện nhiệm vụ chính (đọc cảm biến, xử lý, điều  
        khiển, ...)  
        DoMainTasks();  
        // Có thể chèn lệnh tối ưu, sleep, chờ ngắt...  
    }  
    return 0; // Trong nhúng, thường không bao giờ đến được  
    đây.  
}
```

Trên các hệ thống vi điều khiển (MCU), sau khi hoàn tất các tác vụ khởi tạo ban đầu, phần lớn nội dung chương trình sẽ được đặt trong một vòng lặp vô hạn while(1). Điều này là cần thiết để đảm bảo thiết bị hoạt động liên tục. Nếu hàm main() kết thúc, vi điều khiển có thể rơi vào trạng thái không xác định, dẫn đến các hành vi không mong muốn hoặc hệ thống bị treo (crash).

Ngoài hàm `main()`, lập trình nhúng thường sử dụng biến toàn cục và cấu trúc (structs) để lưu trữ trạng thái của hệ thống và truy cập các thiết bị ngoại vi. Các thanh ghi cấu hình phần cứng, ví dụ như thanh ghi điều khiển GPIO hay bộ đếm thời gian, thường được ánh xạ trực tiếp vào vùng nhớ vật lý thông qua con trỏ, hoặc được truy cập gián tiếp qua các hàm/macro do thư viện HAL (Hardware Abstraction Layer) cung cấp. Điều này cho phép lập trình viên dễ dàng cấu hình chế độ hoạt động, tần số xung nhịp và các thông số khác của phần cứng.

d. Công cụ và môi trường lập trình C/C++ cho vi điều khiển

Việc phát triển phần mềm nhúng C/C++ yêu cầu các công cụ biên dịch và môi trường phát triển tích hợp (IDE) chuyên dụng.

Trình biên dịch: Hầu hết các vi điều khiển (MCU) sử dụng GCC (GNU Compiler Collection) hoặc một biến thể của nó. Ví dụ, ESP32 sử dụng Xtensa GCC (`xtensa-esp32-elf-gcc`) làm công cụ biên dịch chính. Các MCU ARM thường dùng ARM GCC. Trình biên dịch chuyển đổi mã C/C++ thành mã máy tối ưu, và thường đi kèm với các công cụ phụ trợ như linker (trình liên kết) và debugger (công cụ gỡ lỗi). Các ví dụ khác như Keil (ARMCC) hay MPLAB XC (cho PIC) cũng phổ biến, nhưng về cơ bản mọi công cụ đều phục vụ biên dịch mã nhúng với các thư viện chuẩn.

IDE và môi trường phát triển: Có nhiều IDE hỗ trợ lập trình nhúng C/C++. Với ESP32 cụ thể, có thể kể đến: Arduino IDE (dùng Arduino core cho ESP32, lập trình C++ đơn giản), PlatformIO (hỗ trợ đa nền tảng, tích hợp trong VS Code), Espressif IDE/VS Code Extension (bản đồ CMake của ESP-IDF), và Espressif IoT Development Framework (ESP-IDF) kết hợp với trình soạn thảo như VS Code hoặc Eclipse. Ngoài ra, một số IDE khác như Keil μ Vision (chủ yếu cho ARM Cortex-M) hay STM32CubeIDE (cho dòng STM32) phổ biến trong cộng đồng nhúng. Các IDE này thường tích hợp sẵn công cụ biên dịch (GCC, Clang), nạp mã (flash), và gỡ lỗi (debug) thuận tiện.

Thư viện hỗ trợ phần cứng: Nền tảng phần mềm thường đi kèm với các thư viện phong phú. Trên ESP32/ESP-IDF, có FreeRTOS (hệ điều hành thời gian thực) được tích hợp sẵn, nên lập trình viên sử dụng API của FreeRTOS để quản lý tác vụ, bộ đếm thời gian, queues... Bên cạnh đó, ESP-IDF cung cấp các thư viện HAL (Hardware Abstraction Layer) và driver cho từng ngoại vi (GPIO, ADC, SPI, I²C, UART, Wi-Fi, Bluetooth, v.v.) để dễ cấu hình phần cứng. Ví dụ, có module `esp_wifi` để điều khiển Wi-Fi, module `driver/gpio.h` để điều khiển chân, tất cả đều viết bằng C/C++. Các thư viện khác như lwIP (giao thức mạng TCP/IP), cJSON (xử lý JSON), hay driver cảm biến (ví dụ cảm biến nhiệt độ DHT, IMU) cũng có

sẵn hoặc do cộng đồng cung cấp, giúp giảm thiểu công việc phải viết lại từ đầu.

Lợi ích môi trường phát triển: Các công cụ trên giúp tối ưu hoá quy trình phát triển. Ví dụ, PlatformIO tích hợp quản lý thư viện và build nhiều board cùng lúc. Arduino IDE rất dễ tiếp cận với người mới (đã tích hợp sẵn thư viện C++ tiêu chuẩn). ESP-IDF với CMake cho phép tùy chỉnh sâu, sử dụng C và C++ kết hợp (mặc định C++23). Nhờ đó, lập trình viên có thể chọn công cụ và thư viện phù hợp với dự án của mình, nhưng điểm chung là tất cả đều dùng C/C++ làm ngôn ngữ nền tảng.

e. Ưu điểm và Hạn chế của C/C++ trong Lập trình Nhúng

Ưu điểm:

C/C++ là ngôn ngữ gần sát phần cứng, biên dịch ra mã máy hiệu năng cao, giúp ESP32 chạy xử lý rất nhanh và hiệu quả. Cả hai ngôn ngữ đều hỗ trợ các tính năng cấp cao (OOP, thư viện STL cho C++, con trỏ, typedef, v.v.) cho phép xây dựng phần mềm phức tạp dễ tái sử dụng và bảo trì. Thư viện và driver cho ESP32 rất phong phú (ESP-IDF, Arduino-ESP32, ESP-DSP, v.v.) và cộng đồng lập trình viên rộng lớn thường xuyên chia sẻ mã nguồn, tư liệu. Cộng đồng hỗ trợ mạnh giúp giải quyết vấn đề nhanh hơn. C/C++ không có cơ chế thu dọn bộ nhớ tự động (garbage collector) nên lập trình viên có toàn quyền điều khiển việc cấp phát và giải phóng bộ nhớ, tối ưu được tài nguyên hệ thống.

Hạn chế:

C/C++ phức tạp và đòi hỏi lập trình viên có hiểu biết sâu về phần cứng cũng như quản lý bộ nhớ. Việc phải tự quản lý con trỏ và cấp phát/giải phóng thủ công rất dễ dẫn đến lỗi tràn bộ nhớ, truy cập ngẫu nhiên hay gián đoạn nghiêm trọng nếu không cẩn thận. Ngoài ra, C++ (và cả C) không có cơ chế xử lý lỗi như Garbage Collector, nên nếu dùng sai bộ nhớ có thể gây crash ứng dụng. Việc biên dịch dự án C++ lớn cũng có thể mất nhiều thời gian và kích thước nhị phân có thể lớn nếu sử dụng nhiều tính năng nâng cao không cần thiết. Chính vì vậy, người phát triển nhúng cần kiên nhẫn và test kỹ, tránh dùng các tính năng C++ tốn tài nguyên như exception hay template quá mức.

4.2 Phần mềm viết chương trình điều khiển (Arduino IDE)

4.2.1. Giới thiệu chung về Arduino IDE

Arduino IDE (Môi trường phát triển tích hợp) là công cụ phần mềm mã nguồn mở dành riêng cho lập trình các board mạch Arduino. Arduino IDE đơn giản hóa quá trình phát triển ứng dụng vi điều khiển, giúp cả người mới bắt đầu và chuyên gia dễ dàng viết, biên dịch và nạp chương

trình vào board. Phần mềm này hỗ trợ lập trình bằng ngôn ngữ Arduino (dựa trên C/C++) và chạy trên Windows, MacOS, Linux với giao diện thân thiện. Nhờ tính đơn giản và linh hoạt, Arduino IDE đã trở thành công cụ phổ biến trong phát triển các hệ thống nhúng và ứng dụng IoT.

Đối tượng chính sử dụng Arduino IDE bao gồm sinh viên, giáo viên, người mới làm quen với lập trình nhúng cũng như các maker, kỹ sư phát triển prototype. Một cộng đồng toàn cầu gồm học sinh, sinh viên, người đam mê (hobbyist), lập trình viên và các chuyên gia đã chọn Arduino IDE cho các dự án của mình. Nhờ đó, kiến thức và thư viện hỗ trợ phong phú từ cộng đồng Arduino giúp người học và nhà phát triển nhanh chóng triển khai ý tưởng.

Đối tượng sử dụng chủ yếu gồm:

- Sinh viên, giáo viên: Trong đào tạo vi điện tử và lập trình nhúng.
- Makers/hobbyists: Tự học, làm dự án DIY (như robot, cảm biến, điều khiển đèn).
- Kỹ sư, chuyên gia IoT: Lập trình nguyên mẫu thiết bị thông minh.
- Người mới học lập trình: Giao diện đơn giản và khả năng demo nhanh giúp họ dễ tiếp cận.

4.2.2. Đặc điểm và chức năng chính của Arduino IDE

Arduino IDE có giao diện đồ họa trực quan và đơn giản, với thanh menu chức năng (mở/tạo tập tin, lưu, biên dịch, nạp) và cửa sổ soạn thảo mã lớn. Giao diện này rất thân thiện, giúp người dùng mới làm quen nhanh chóng. Arduino IDE tích hợp các thành phần cần thiết (editor, compiler, cửa sổ gỡ lỗi) trong cùng một cửa sổ, nên không cần chạy nhiều chương trình riêng biệt. Ví dụ, hình minh họa bên trên cho thấy cửa sổ Arduino IDE với ví dụ chương trình Blink (nháy đèn LED), trong đó có các nút điều khiển biên dịch và nạp code.

Giao diện đơn giản, dễ sử dụng: Thiết kế chuẩn (menu, toolbar, khu vực viết code) được người dùng đánh giá trực quan, phù hợp cả người chưa có nhiều kinh nghiệm. Các công cụ cơ bản như Compile (Verify), Upload (nạp), Serial Monitor... được đặt ở gần các phím điều khiển, tạo thuận tiện khi thao tác.

Hỗ trợ nhiều vi điều khiển và board mạch: Arduino IDE hỗ trợ tất cả các board Arduino chính thức (Uno, Mega, Nano, Leonardo, ...) và thông qua Board Manager cho phép cài thêm core khác (ví dụ ESP32, STM32, Arduino Pro, v.v.). Thực tế, nhiều dòng board mới ra đời đều hướng tới tương thích với Arduino IDE, cho phép cộng đồng dễ dàng lập trình các thiết bị mới bằng môi trường quen thuộc.

Tích hợp trình biên dịch và nạp code: Arduino IDE sử dụng hệ toolchain GNU và thư viện AVR Libc để biên dịch mã nguồn, và sử dụng công cụ avrdude (hoặc trình nạp tương ứng) để nạp chương trình qua cổng USB. Người dùng chỉ cần bấm nút “Verify” để biên dịch và “Upload” để nạp code lên board mà không cần cấu hình phức tạp.

Thư viện phong phú: Arduino IDE đi kèm nhiều thư viện chuẩn (Wire, SPI, Serial, WiFi, BLE, cảm biến DHT, BMP, v.v.) giúp lập trình tương tác phần cứng nhanh chóng. Người dùng có thể cài đặt thêm hàng ngàn thư viện từ Library Manager bên trong IDE, mở rộng khả năng kết nối cảm biến, giao tiếp I²C /SPI, WiFi, Bluetooth... Thư viện chuẩn với các hàm đơn giản cho phép điều khiển phần cứng dễ dàng mà không phải lập trình từ mức bit thấp.

Serial Monitor (Giám sát nối tiếp): Arduino IDE tích hợp cửa sổ Serial Monitor để nhận/gửi dữ liệu nối tiếp giữa board và máy tính, hỗ trợ gỡ lỗi hoặc theo dõi dữ liệu thời gian thực. Phiên bản mới (IDE 2.x) còn bổ sung thêm công cụ Serial Plotter để vẽ đồ thị các giá trị cảm biến từ dữ liệu Serial giúp phân tích trực quan. Hai công cụ này (Monitor và Plotter) cho phép người dùng quan sát đồng thời giá trị số và đồ thị trong quá trình chạy chương trình.

4.2.3. Cấu trúc chương trình trong Arduino IDE

Trong môi trường Arduino IDE, mọi **sketch (chương trình)** tối thiểu bao gồm hai hàm cơ bản: `setup()` và `loop()`. Theo định nghĩa của Arduino:

- **setup():** Đây là hàm khởi tạo, chỉ chạy một lần khi board bật nguồn hoặc reset. Tại đây, chúng ta thường dùng để cấu hình chế độ chân (ví dụ: `pinMode()`), khởi động giao tiếp (ví dụ: `Serial.begin()`), hoặc thiết lập ban đầu cho cảm biến, module, v.v. Sau khi hàm này thực thi xong, chương trình sẽ chuyển sang hàm `loop()`.
- **loop():** Đây là hàm chính, chạy lặp vô hạn sau khi `setup()` hoàn tất. Mọi công việc xử lý chính của chương trình (như đọc cảm biến, điều khiển ngõ ra, tính toán, giao tiếp...) đều được đặt trong `loop()`, nên chương trình sẽ lặp đi lặp lại mãi mãi miễn là board còn được cấp nguồn.

Ngoài hai hàm này, đầu file sketch thường bao gồm các câu lệnh **#include** để đưa các thư viện cần thiết vào (ví dụ: `#include <Wire.h>`, `#include <WiFi.h>`...), cùng với các khai báo biến và chân GPIO. Trong hàm `setup()`, chúng ta sử dụng `pinMode(chân, OUTPUT/INPUT)` để đặt chế độ hoạt động cho chân (đầu vào hoặc đầu ra) trước khi sử dụng.

Ví dụ một chương trình đơn giản điều khiển LED tích hợp trên chân 13:

```
#define LED_PIN 13

void setup() {

    pinMode(LED_PIN, OUTPUT);    // Thiết lập chân 13 là
    ngõ ra

}

void loop() {

    digitalWrite(LED_PIN, HIGH);    // Bật LED

    delay(1000);                    // Đợi 1 giây

    digitalWrite(LED_PIN, LOW);     // Tắt LED

    delay(1000);                    // Đợi 1 giây }
```

Ví dụ trên thực hiện việc nháy LED mỗi giây một lần. Mã nguồn này minh họa cách sử dụng pinMode() để khai báo chân và digitalWrite() để gửi tín hiệu HIGH/LOW, cùng hàm delay() tạo độ trễ.

4.2.4. Các tính năng hỗ trợ phát triển

Quản lý thư viện (Library Manager): Arduino IDE tích hợp công cụ Library Manager cho phép tìm kiếm, cài đặt và cập nhật các thư viện phần mềm dễ dàng ngay trong IDE. Người dùng chỉ cần vào Sketch → Include Library → Manage Libraries, rồi tìm tên thư viện (ví dụ “DHT sensor”) và cài đặt tự động. Điều này giúp mở rộng nhanh khả năng của dự án (thêm thư viện WiFi, cảm biến, đồ họa, ...) mà không phải tự build thư viện thủ công.

Serial Plotter: Công cụ mới trong Arduino IDE 2.x, cho phép vẽ đồ thị theo thời gian của nhiều biến/tín hiệu nối tiếp. Ví dụ khi đọc giá trị cảm biến liên tục, ta có thể hiển thị đường đồ thị trực quan ngay trong IDE, thuận tiện cho hiệu chuẩn hoặc phân tích dữ liệu thời gian thực. Ngoài ra, vẫn có cửa sổ Serial Monitor truyền thống để xem giá trị số và gửi lệnh cơ bản.

Hỗ trợ đa nền tảng nhúng: Arduino IDE không chỉ giới hạn ở các board Arduino cổ điển mà còn hỗ trợ các platform khác thông qua core tương ứng. Ví dụ, ta có thể cài đặt ESP32 Core để lập trình các board ESP32, hoặc STM32 Core cho các board STM32... thông qua Board Manager của IDE. Nhờ vậy, một IDE có thể làm việc với nhiều loại chip khác nhau, đơn giản hóa quá trình chuyển đổi giữa các platform.

Tích hợp với PlatformIO và plugin mở rộng: Mặc dù Arduino IDE hoạt động độc lập, nó cũng tương thích với nhiều công cụ phát triển khác. Người dùng có thể dùng Arduino IDE như một plugin trong PlatformIO (một môi trường phát triển đa nền tảng), hoặc sử dụng các tiện ích mở rộng (extension) trên VS Code, Eclipse Theia... để làm việc với mã Arduino. Điều này cho phép tận dụng các tính năng nâng cao (gỡ lỗi phần cứng, quản lý dự án) của các IDE khác trong khi vẫn giữ nguyên ngôn ngữ và thư viện Arduino.

Tổng kết: Arduino IDE là một công cụ linh hoạt, dễ sử dụng cho việc phát triển ứng dụng nhúng và IoT. Với giao diện đơn giản, bộ công cụ tích hợp (biên dịch, nạp firmware, quản lý thư viện, Serial Monitor...), và khả năng mở rộng cao, nó giúp người học và nhà phát triển nhanh chóng xây dựng và thử nghiệm các hệ thống vi điều khiển mà không đòi hỏi nhiều thiết lập phức tạp. Nhờ có Arduino IDE, nhiều ý tưởng sáng tạo đã được hiện thực hoá thành các prototype và sản phẩm trong cộng đồng nhúng trên toàn thế giới.

4.3 Giao diện người dùng trên Blynk

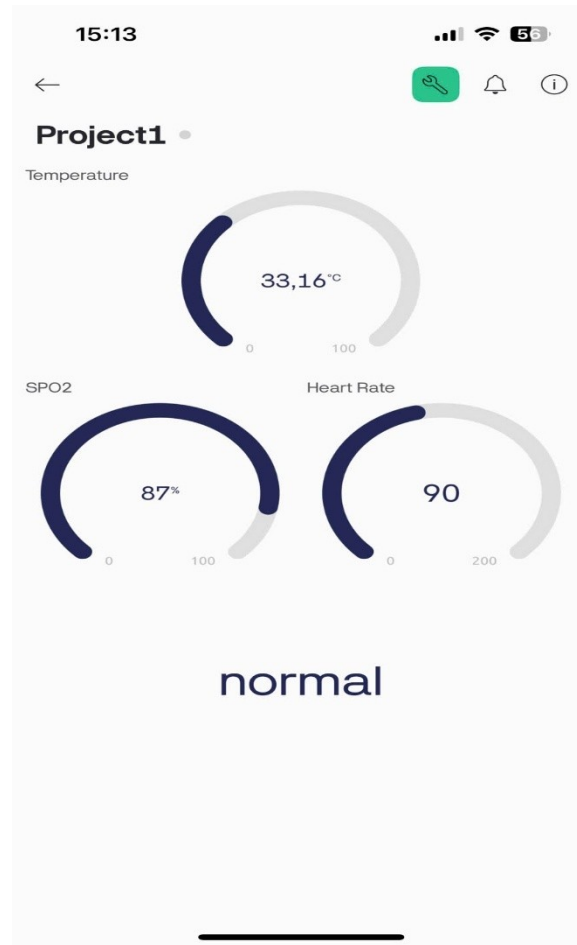
4.3.1 Giới thiệu nền tảng Blynk

Blynk IoT là một nền tảng phần mềm IoT đầy đủ cho phép thiết kế nguyên mẫu, triển khai và quản lý thiết bị điện tử từ xa ở mọi quy mô. Nền tảng này giúp kết nối phần cứng với đám mây, xây dựng ứng dụng di động (iOS/Android), giám sát dữ liệu thời gian thực và điều khiển thiết bị từ xa mà không cần mã hóa phức tạp. Giao diện ứng dụng Blynk cho phép thiết kế dashboard bằng cách kéo-thả (drag-and-drop) widget: chỉ cần chọn widget từ “Widget Box” rồi cấu hình chức năng. Blynk hỗ trợ đa dạng widget: ví dụ Controllers như nút bấm (Button), thanh trượt (Slider) để gửi lệnh, và Displays như đèn LED ảo, biểu đồ (Chart), đồng hồ đo (Gauge) để trực quan hóa dữ liệu từ thiết bị.

4.3.2 Ứng dụng trong đồ án

Trong dự án này, hệ thống sử dụng các cảm biến đo đặc thông số sức khỏe và hoạt động: cảm biến nhiệt độ MAX30205 đo thân nhiệt, cảm biến nhịp tim MAX30102 đo nhịp tim, và cảm biến MPU6050 (gia tốc kế 3 trục + con quay hồi chuyển) theo dõi chuyển động để phát hiện té ngã. Dữ liệu đọc được từ các cảm biến trên ESP32 được gửi đến ứng dụng Blynk để hiển thị và giám sát. Cụ thể, giá trị nhiệt độ và nhịp tim liên tục được hiển thị trên giao diện Blynk, còn các mẫu gia tốc được phân tích bằng mô hình học máy để xác định trạng thái (đứng, ngồi, đi bộ hoặc té ngã). Khi mô hình phát hiện sự kiện té ngã, hệ thống sẽ kích hoạt thông báo cảnh báo (gửi push notification) đến ứng dụng trên điện thoại. Nhờ đó, người dùng hoặc người chăm sóc có thể giám sát sức khỏe từ xa và nhận cảnh báo kịp thời khi phát sinh sự cố.

4.3.3 Cấu trúc giao diện Blynk đã xây dựng



Hình 4.1 Giao diện của ứng dụng Blynk đã xây dựng

Ứng dụng Blynk IoT được thiết kế trực quan với các widget phù hợp cho dự án sức khỏe. Ở giao diện trên, hệ thống sử dụng đồng hồ đo hình tròn (Gauge) để hiển thị giá trị cảm biến và tự động gửi thông báo cảnh báo khi phát hiện té ngã. Khi người dùng ngã, ứng dụng sẽ hiển thị cảnh báo “Fall detected” cùng thông tin dữ liệu liên quan.

- Widget Nhiệt độ: Sử dụng widget dạng gauge (đồng hồ đo hình tròn) để hiển thị giá trị nhiệt độ cơ thể theo thời gian thực. Người dùng có thể nhìn nhanh giá trị nhiệt độ dưới dạng kim đồng hồ hoặc vòng tròn quay. Blynk hỗ trợ widget Gauge cho mục đích này.
- Widget Nhịp tim: Sử dụng widget gauge (hoặc Value Display nếu muốn) để hiển thị nhịp tim đo từ cảm biến MAX30102. Giá trị nhịp tim cập nhật liên tục và được hiển thị trực quan trên một vòng đo hoặc số. Widget Gauge trong Blynk phù hợp để trực quan hóa các đại lượng số dạng liên tục.
- Widget Trạng thái té ngã: Sử dụng widget Văn bản (Text) để hiển thị trạng thái ngã hay an toàn. Ví dụ, khi mô hình xác định “Ngã”, widget sẽ hiện chữ “Fall” (hoặc “Ngã”) với màu

đỏ, còn khi bình thường hiển “OK” với màu xanh. Widget Text trong Blynk cho phép hiển thị các thông báo dạng ký tự tự do.

Ngoài ra, hệ thống sử dụng tính năng cảnh báo (Notifications) của Blynk để gửi thông báo đẩy (push notification) đến điện thoại khi phát hiện té ngã. Blynk cung cấp cơ chế thiết lập sự kiện và gửi thông báo (in-app alert, email hoặc SMS) một cách linh hoạt. Trong dự án, khi cảm biến ghi nhận mẫu té ngã, thiết bị sẽ kích hoạt sự kiện và Blynk app sẽ hiển thị cảnh báo trên điện thoại người dùng.

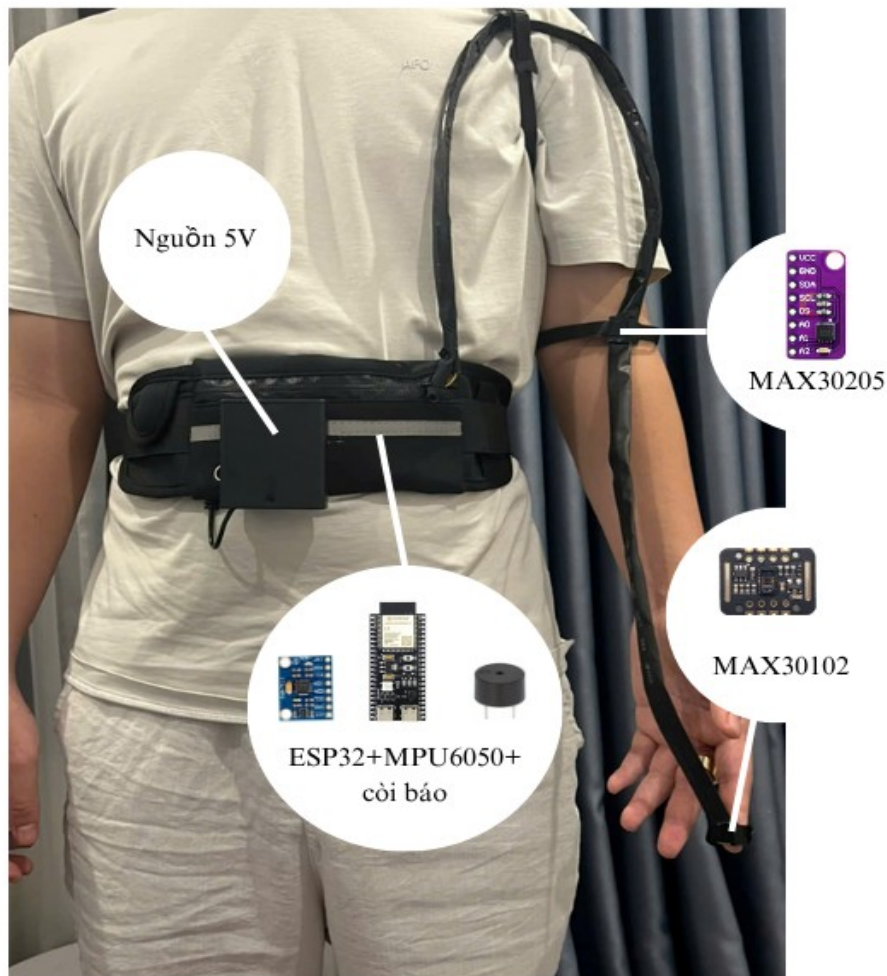
CHƯƠNG 5: KẾT QUẢ THI CÔNG

5.1. Hình ảnh sản phẩm thực tế:

Sau quá trình thi công thì nhóm đã làm ra được thành phẩm như hình bên dưới gồm ESP32 và cảm biến MPU6050 được gắn cố định ở một túi đeo ở eo và ESP32 sẽ được kết nối với cảm biến MAX30102 gắn cố định ở ngón tay và MAX30205 ở cánh tay thông qua dây dẫn.



Hình 5.1 Sản phẩm khi được người dùng đeo lên người



Hình 5.2 Vị trí của các thành phần trong hệ thống

Các thông số sẽ được theo dõi từ xa qua ứng dụng Blynk

5.2. Kết quả thực nghiệm

Nhóm sẽ tiến hành kiểm thử mô hình bằng cách mô phỏng lại các hoạt động té ngã và các hoạt động khác khi đang đeo hệ thống trên người và ghi lại kết quả dự đoán, đồng thời cũng xem xét các chỉ số về sức khỏe được thu thập.

Kết quả sẽ được ghi vào “CONFUSION MATRIX”, là một bảng dùng để ghi lại kết quả của các mô hình phân loại.

Dự đoán	Té ngã	Hoạt động bình thường
Thực tế		
Té ngã	30	0
Hoạt động bình thường	2	32

Bảng 5.1 Confusion matrix

Kết quả sao quá trình thí nghiệm cho thấy mô hình có khả năng phân loại giữa té ngã và các hoạt động khác đạt 96,87% và thời gian phản hồi khi có tình huống té ngã xảy ra từ 4-5 giây. Tuy nhiên ở các tình huống té ngã chỉ là giả vờ, không phải là tình huống ngã thật sự nên có thể xảy ra sai số ngoài thực tế. Các chức năng thu thập thông tin về sức khỏe cũng hoạt động tốt.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Những vấn đề đã giải quyết trong đồ án

- Nguyên cứu, triển khai mô hình Deep Learning trên ESP32.
- Tìm hiểu, sử dụng các cảm biến phù hợp dùng cho việc theo dõi sức khỏe.
- Lập trình cho vi điều khiển ESP32.
- Dùng Blynk để theo dõi dữ liệu từ xa.

6.2. Ưu điểm, khuyết điểm của mô hình.

+Ưu điểm:

- Có thể phân loại té ngã và các hoạt động thường ngày với độ chính xác cao.
- Có thể giúp theo dõi sức khỏe từ xa.
- Chi phí phát triển hợp lý (nhỏ hơn 500000 đồng).

+Khuyết điểm:

- Tổng thể mô hình có rườm rà, chưa được nhỏ gọn, có thể gây khó chịu cho người dùng.
- Thời gian phản hồi khi có té ngã chưa nhanh.
- Phải kết nối với Wifi mới có thể gửi được dữ liệu.

6.3. Hướng phát triển.

- Tích hợp thêm tính năng dự đoán đột quỵ khi té ngã (kết hợp với dữ liệu điện não theo nghiên cứu [13])
- Tự phát triển một ứng dụng theo dõi riêng thay vì Blynk.
- Cải tiến mô hình cho nhỏ gọn hơn.
- Thêm chức năng gửi SMS khi không có Wifi.

Phụ lục:

Mã python

+Tiền xử lý dữ liệu

```
import os
import numpy as np

def read_signal_file(filepath):
    data = []
    with open(filepath, 'r', encoding='utf-8', errors='ignore') as f:
        for line in f:
            line = line.strip().replace('; ', '')
            parts = line.split(',')
            parts = [p.strip() for p in parts if p.strip() != '']

            if len(parts) >= 6:
                try:
                    values = list(map(float, parts[:6]))
                    data.append(values)
                except ValueError:
                    continue
    return np.array(data)

def segment_signal(data, window_size=600, stride=600):
    segments = []
    for i in range(0, len(data) - window_size + 1, stride):
        segment = data[i:i + window_size]
        segments.append(segment)
    return np.array(segments)

def get_label_from_filename(filename):
    fall_tags = ['F01', 'F02', 'F03', 'F04', 'F05', 'F06', 'F07', 'F08', 'F09', 'F10', 'F11', 'F12',
                'F13', 'F14', 'F15']
    if any(tag in filename for tag in fall_tags):
        return 1
    return 0
data_dir = './SisFall4'

X = []
y = []

for root, dirs, files in os.walk(data_dir):
    for filename in files:
        if filename.endswith('.txt'):
            filepath = os.path.join(root, filename)
            data = read_signal_file(filepath)
```



```

print(f"{filename}: {data.shape[0]} mẫu")

if data.shape[0] < 600:
    print(f"Bỏ qua {filename}: quá ít mẫu ({data.shape[0]})")
    continue

segments = segment_signal(data)
label = get_label_from_filename(filename)
X.extend(segments)
y.extend([label] * len(segments))

X = np.array(X)
y = np.array(y)

print("Tổng số sample:", X.shape)
print("Shape mỗi sample:", X[0].shape)
print("Số lượng nhãn 0 (ADL):", np.sum(y == 0))
print("Số lượng nhãn 1 (Fall):", np.sum(y == 1))

```

+ Mô hình và huấn luyện

```

from tensorflow.keras import layers, models, Input
import tensorflow as tf

model = models.Sequential([
    layers.Conv1D(16, kernel_size=5, activation='relu', input_shape=(600, 6)),
    layers.MaxPooling1D(pool_size=2),
    layers.Conv1D(32, kernel_size=5, activation='relu'),
    layers.MaxPooling1D(pool_size=2),
    layers.Conv1D(64, kernel_size=5, activation='relu'),
    layers.MaxPooling1D(pool_size=2),
    layers.Conv1D(128, kernel_size=5, activation='relu'),
    layers.MaxPooling1D(pool_size=2),
    layers.Dropout(0.5),
    layers.GlobalAveragePooling1D(),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

model.summary()

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

from sklearn.model_selection import train_test_split
from tensorflow.keras.callbacks import ReduceLROnPlateau

```

```

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=3,
    min_lr=1e-5,
    verbose=1

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

history=model.fit(X_train, y_train, epochs=50, batch_size=16, validation_data=(X_test,
y_test), callbacks =[reduce_lr])

```

+Chuyển mô hình thành file .tflite

```

import tensorflow as tf
converter = tf.lite.TFLiteConverter.from_keras_model(model)

tflite_model = converter.convert()
tflite_model_name = 'model4'
open(tflite_model_name + '.tflite','wb').write(tflite_model)

```

+Chuyển mô hình thành file .h

```

# Function: Convert some hex value into an array for C programming
def hex_to_c_array(hex_data, var_name):

    c_str = ""

    # Create header guard
    c_str += '#ifndef ' + var_name.upper() + '_H\n'
    c_str += '#define ' + var_name.upper() + '_H\n\n'

    # Add array length at top of file
    c_str += '\nunsigned int ' + var_name + '_len = ' + str(len(hex_data)) + ';\n'

    # Declare C variable
    c_str += 'unsigned char ' + var_name + '[] = {'
    hex_array = []
    for i, val in enumerate(hex_data) :

        # Construct string from hex
        hex_str = format(val, '#04x')

        # Add formatting so each line stays within 80 characters
        if (i + 1) < len(hex_data):
            hex_str += ','
        if (i + 1) % 12 == 0:
            hex_str += '\n '

```

```

hex_array.append(hex_str)

# Add closing brace
c_str += '\n ' + format(' '.join(hex_array)) + '\n};\n\n'

# Close out header guard
c_str += '#endif //' + var_name.upper() + '_H'

return c_str
c_model_name = 'model4'
with open(c_model_name + '.h', 'w') as file:
    file.write(hex_to_c_array(tflite_model, c_model_name))

```

Code trên Arduino IDE

```

#include <MAX30105.h>
#include <heartRate.h>
#include <spo2_algorithm.h>

// ===== FINAL CODE: MPU6050 + MAX30105 + MLX90614 + Firebase + AI
// FALL DETECTION =====
#define BLYNK_TEMPLATE_ID "TMPL6SW0dqH9J"
#define BLYNK_TEMPLATE_NAME "Project1"
#define BLYNK_AUTH_TOKEN "xK0vy3ojBbq8p2JdZH59S8B0Gy-8swU-"
#include <Wire.h>
#include <WiFi.h>
#include <FirebaseESP32.h>
#include "MPU6050.h"
#include "MAX30105.h"
#include "spo2_algorithm.h"
#include <Adafruit_MLX90614.h>
#include "model4.h"
#include <Chirale_TensorFlowLite.h>
#include "tensorflow/lite/micro/all_ops_resolver.h"
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include "heartRate.h"
#include <BlynkSimpleEsp32.h>
#include "Protocentral_MAX30205.h"
MAX30205 tempSensor;
const byte RATE_SIZE = 4;
byte rates[RATE_SIZE];
byte rateSpot = 0;
long lastBeat = 0;
float beatsPerMinute;
int beatAvg;
#define WIFI_SSID "BS12 0804"
#define WIFI_PASSWORD "79797979"
FirebaseData firebaseData;
FirebaseAuth auth;

```

```

FirebaseConfig config;
#define BUZZER_PIN 12

MPU6050 mpu;
MAX30105 particleSensor;
Adafruit_MLX90614 mlx = Adafruit_MLX90614();
#define TEMP_HIGH_THRESHOLD 38.0
#define TEMP_LOW_THRESHOLD 33.0
#define SPO2_LOW_THRESHOLD 95
#define HR_HIGH_THRESHOLD 120
#define HR_LOW_THRESHOLD 40
// ===== TENSORFLOW LITE =====
#define SEQ_LEN 600
#define N_FEATURES 6
unsigned long systemStartTime = 0;
const tflite::Model* model = tflite::GetModel(model4_data);
tflite::AllOpsResolver resolver;
constexpr int kTensorArenaSize = 110 * 1024;
uint8_t* tensor_arena = nullptr;
tflite::MicroInterpreter* interpreter;
TfLiteTensor* input;
TfLiteTensor* output;
float (*sequence)[N_FEATURES] = nullptr;

const float acc_threshold = 1.2;
const float gyro_threshold = 150.0;
bool isCollecting = false;
enum SensorMode {
    MODE_REALTIME_HR,
    MODE_SPO2,
    MODE_IDLE
};

SensorMode currentMode = MODE_REALTIME_HR;
unsigned long modeStartTime = 0;

uint32_t irBuffer[100];
uint32_t redBuffer[100];
int32_t spo2 = 0, heartRate = 0;
int8_t validSPO2 = 0, validHeartRate = 0;
int spo2SampleIndex = 0;
bool isSpO2Done = false;

unsigned long lastMotionCheck = 0;
unsigned long lastHeartRateCheck = 0;
unsigned long lastSpO2Check = 0;
unsigned long lastTempCheck = 0;

const unsigned long motionCheckInterval = 100;
const unsigned long tempCheckInterval = 5000;

```

```

void setup() {
  Serial.begin(115200);
  Wire.begin(9, 8);
  Wire1.begin(6, 7);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  mpu.initialize();
  mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_16);
  mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_2000);

  particleSensor.begin(Wire1, I2C_SPEED_FAST);
  particleSensor.setup();
  particleSensor.setPulseAmplitudeRed(0x0A);
  tempSensor.scanAvailableSensors();

  tempSensor.begin();
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  tensor_arena = (uint8_t*)heap_caps_malloc(kTensorArenaSize, MALLOC_CAP_SPIRAM);
  sequence = (float(*)[N_FEATURES])heap_caps_malloc(sizeof(float) * SEQ_LEN *
N_FEATURES, MALLOC_CAP_SPIRAM);
  if (!tensor_arena || !sequence) { Serial.println("Memory allocation failed!"); while (1); }
  interpreter = new tfLite::MicroInterpreter(model, resolver, tensor_arena,
kTensorArenaSize);
  if (interpreter->AllocateTensors() != kTfLiteOk) { Serial.println("Tensor allocation
failed!"); while (1); }
  input = interpreter->input(0);
  output = interpreter->output(0);
  Blynk.begin(BLYNK_AUTH_TOKEN, WIFI_SSID, WIFI_PASSWORD);
  Blynk.virtualWrite(V3, "normal");
  Serial.println("Setup complete!\nPlace finger on sensor for heart rate...");
  systemStartTime = millis();
}

void manageSensorModes() {
  unsigned long now = millis();
  switch (currentMode) {
    case MODE_REALTIME_HR:
      if (now - modeStartTime > 5000) {
        currentMode = MODE_SPO2;
        modeStartTime = now;
        spo2SampleIndex = 0;
        isSpO2Done = false;
        particleSensor.setup(60, 4, 2, 200, 411, 4096);
        Serial.println(" Switching to SpO2 mode...");
      }
      break;
    case MODE_SPO2:
      if (isSpO2Done) {
        currentMode = MODE_IDLE;
        modeStartTime = now;
        isSpO2Done = false;
      }
  }
}

```

```

        Serial.println("Switching to idle mode...");
    }
    break;
case MODE_IDLE:
    if (now - modeStartTime > 2000) {
        currentMode = MODE_REALTIME_HR;
        modeStartTime = now;
        particleSensor.setup();
        particleSensor.setPulseAmplitudeRed(0x0A);
        Serial.println("Switching to real-time heart rate mode...");
    }
    break;
default:
    currentMode = MODE_IDLE;
    break;
}
}
}

void checkMotionAndStartCollection() {
    unsigned long now = millis();
    if (!isCollecting && now - lastMotionCheck > motionCheckInterval) {
        int16_t ax, ay, az, gx, gy, gz;
        mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
        float accX = ax / 2048.0f, accY = ay / 2048.0f, accZ = az / 2048.0f;
        float gyroX = gx / 16.4f, gyroY = gy / 16.4f, gyroZ = gz / 16.4f;
        if (abs(accX) > acc_threshold || abs(accY) > acc_threshold || abs(accZ) > acc_threshold
||
        abs(gyroX) > gyro_threshold || abs(gyroY) > gyro_threshold || abs(gyroZ) >
gyro_threshold) {
            Serial.println("Motion detected - collecting AI data!");
            isCollecting = true;
        }
        lastMotionCheck = now;
    }
}

void collectDataForAI() {
    if (!isCollecting) return;
    for (int i = 0; i < SEQ_LEN; i++) {
        int16_t ax, ay, az, gx, gy, gz;
        mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
        sequence[i][0] = ax / 2048.0f;
        sequence[i][1] = ay / 2048.0f;
        sequence[i][2] = az / 2048.0f;
        sequence[i][3] = gx / 16.4f;
        sequence[i][4] = gy / 16.4f;
        sequence[i][5] = gz / 16.4f;
        delay(5);
    }
    for (int i = 0; i < SEQ_LEN; i++)
        for (int j = 0; j < N_FEATURES; j++)

```

```

    input->data.f[i * N_FEATURES + j] = sequence[i][j];
if (interpreter->Invoke() != kTfLiteOk) {
    Serial.println("Model error!");
    return;
}
float result = output->data.f[0];
Serial.printf("AI Result: %.3f\n", result);

if (result > 0.5f) {
    Serial.println("FALL DETECTED!");
    Blynk.virtualWrite(V3, "fall");
    digitalWrite(BUZZER_PIN, HIGH);
    Blynk.logEvent("warning", "Phát hiện té ngã");
} else {
    Serial.println("No fall detected.");
    Blynk.virtualWrite(V3, "normal");
}
isCollecting = false;
}

void checkRealtimeHeartRate() {
    if (currentMode != MODE_REALTIME_HR) return;
    unsigned long now = millis();
    long irValue = particleSensor.getIR();
    if (irValue < 50000) return;
    if (checkForBeat(irValue)) {
        long delta = now - lastBeat;
        lastBeat = now;
        beatsPerMinute = 60 / (delta / 1000.0);
        if (beatsPerMinute > 20 && beatsPerMinute < 255) {
            rates[rateSpot++] = (byte)beatsPerMinute;
            rateSpot %= RATE_SIZE;
            long total = 0;
            for (byte i = 0; i < RATE_SIZE; i++) total += rates[i];
            beatAvg = total / RATE_SIZE;
            Serial.printf("BPM: %.1f | Avg: %d\n", beatsPerMinute, beatAvg);
            Blynk.virtualWrite(V0, beatAvg);
        }
    }
}

void updateSpO2Measurement() {
    if (currentMode != MODE_SPO2) return;
    unsigned long now = millis();
    particleSensor.check();
    if (particleSensor.available()) {
        redBuffer[spo2SampleIndex] = particleSensor.getRed();
        irBuffer[spo2SampleIndex] = particleSensor.getIR();
        particleSensor.nextSample();
        Serial.printf("SpO2 sample: %d/100\n", spo2SampleIndex + 1);
    }
}

```

```

    spo2SampleIndex++;
    if (spo2SampleIndex >= 100) {
        maxim_heart_rate_and_oxygen_saturation(irBuffer, 100, redBuffer, &spo2, &validSPO2,
&heartRate, &validHeartRate);
        if (validSPO2) {
            updateTemperature();
            Serial.printf("SpO2: %ld%% \n", spo2);
            Blynk.virtualWrite(V1, spo2);
        } else {
            Serial.println("Invalid SpO2 reading");
        }
        isSpO2Done = true;
        spo2SampleIndex = 0;
    }
}

void updateTemperature() {
    float temp = tempSensor.getTemperature();
    Serial.printf("Body Temperature (MAX30205): %.2f°C\n", temp);
    Blynk.virtualWrite(V2, temp);
}

void checkVitalSignsAlerts() {
    // Nhịp tim
    if (beatAvg > 0 && (beatAvg < HR_LOW_THRESHOLD || beatAvg >
HR_HIGH_THRESHOLD)) {
        Serial.println("WARNING: Abnormal Heart Rate!");
        Blynk.logEvent("warning", "Cảnh báo: Nhịp tim bất thường");
    }

    // Nhiệt độ
    float temp = tempSensor.getTemperature();
    if (temp < TEMP_LOW_THRESHOLD || temp > TEMP_HIGH_THRESHOLD) {
        Serial.println(" WARNING: Abnormal Temperature!");
        Blynk.logEvent("warning", "Cảnh báo: Nhiệt độ cơ thể bất thường");
    }

    // SpO2
    if (spo2 > 0 && spo2 < SPO2_LOW_THRESHOLD) {
        Serial.println("WARNING: Low SpO2!");
        Blynk.logEvent("warning", "Cảnh báo: Nồng độ oxy trong máu thấp");
    }
}

void loop() {
    Blynk.run();
    manageSensorModes();
    checkMotionAndStartCollection();
    collectDataForAI();
}

```



```
checkRealtimeHeartRate();  
updateSpO2Measurement();  
if (millis() - systemStartTime > 30000) {  
    checkVitalSignsAlerts();  
}  
}
```

Tài liệu tham khảo

- [1]. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, "SisFall: A fall and movement dataset" *Sensors*, 2017.
- [2]. InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification Revision 3.4*, 2013. [URL]: <https://www.invensense.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>
- [3]. Maxim Integrated, *MAX30205: Human Body Temperature Sensor*, 2016. [URL]: <https://datasheets.maximintegrated.com/en/ds/MAX30205.pdf>
- [4]. Maxim Integrated, *MAX30102: High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health*, 2018. [URL]: <https://datasheets.maximintegrated.com/en/ds/MAX30102.pdf>
- [5]. Mohammad Saleh Hoseinzadeh, Ali Ekhlasi. "A Wireless Body Temperature and Oxygen Saturation Monitoring system based on Android Smartphones". International Conference on Internet of Things and Applications (IoT), IEEE, 2022.
- [6]. E. Casilari, R. Lora-Rivera, and F. García-Lago, "A Wearable Fall Detection System Using Deep Learning", *Advances and Trends in Artificial Intelligence. From Theory to Practice*, Springer, 2019
- [7]. G. Wang, Q. Li, L. Wang, Y. Zhang, and Z. Liu, "Elderly fall detection with an accelerometer using lightweight neural networks", *Electronics*, 2019
- [8]. T. de Quadros, A. E. Lazzaretti, and F. K. Schneider, "A movement decomposition and machine learning-based fall detection system using wrist wearable device," *IEEE Sensors Journal*, 2018
- [9]. A.Najmurrokhman, Kusnandar, U. Komarudin, and A. Wibisono, "Development of Falling Notification System for Elderly Using MPU6050 Sensor and Short Message Service," in *Proc. 2nd Int. Seminar of Science and Applied Technology (ISSAT)*, 2021
- [10]. S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, and D. J. Inman, "1D convolutional neural networks and applications: A survey," *Mechanical Systems and Signal Processing*, ScienceDirect, 2021.
- [11]. Workshop HME ITB, "Integrate a Pre-Trained TensorFlow Lite Model With An ESP32 Program To Classify Sensor MPU6050 Data In Real-Time," *Medium*, May 11, 2023. [URL]: <https://medium.com/@workshopitb/integrate-a-pre-trained-tensorflow-lite-model-with-an-esp32-program-to-classify-sensor-mpu6050-data-2728dc6de9f2>

- [12]. PGS TS. Trương Ngọc Sơn, *Giáo trình trí tuệ nhân tạo cơ sở và ứng dụng*, Nhà xuất bản Đại học Quốc gia TP. Hồ Chí Minh, 2020.
- [13]. Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Shlomi Regev, Rocky Rhodes, Tiezhen Wang, Pete Warden, *TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems*. Proceedings of Machine Learning and Systems 3 (MLSys 2021)
- [14]. Yoon-A Choi, Sejin Park , Jong-Arm Jun 1 , Chee Meng Benjamin Ho, Cheol-Sig Pyo, Hansung Lee, Jaehak Yu ,*Machine-Learning-Based Elderly Stroke Monitoring System Using Electroencephalography Vital Signals*. Applies sciences, 2021.